

1. Генеративно-состязательные сети (GAN)

TensorFlow реализация cGAN - условный генератор изображений

1.1 Импорты

```
In [1]: import warnings
warnings.filterwarnings('ignore')

import os
import glob
import time
from pathlib import Path
from datetime import datetime
from typing import Dict, List, Tuple, Optional, Union, Any
from typing import Optional, List, Tuple, Protocol
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from torch.utils.data import Dataset, DataLoader
from enum import Enum
import numpy as np
from IPython.display import clear_output, display
import io
import base64
import matplotlib.pyplot as plt
from PIL import Image
from numpy import zeros, ones
from numpy.random import randn, randint
from keras.models import Sequential
from tensorflow.keras.losses import BinaryCrossentropy, MeanSquaredError
from tensorflow.keras.losses import SparseCategoricalCrossentropy
from tensorflow.keras.layers import (
    Input, Dense, Conv2D, Conv2DTranspose, BatchNormalization,
    LeakyReLU, Dropout, Flatten, Reshape, Embedding, Concatenate,
    GlobalAveragePooling2D, Lambda, Add, Multiply, Layer
)
from keras.layers import BatchNormalization
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.models import Model
import tensorflow as tf
import sys
import logging
import kagglehub

import os
import random
from typing import List, Tuple, Dict, Optional, Union, Any
from pathlib import Path
import shutil
```

```

import cv2
import numpy as np
import pandas as pd
from PIL import Image
from tqdm import tqdm
from sklearn.model_selection import train_test_split
from copy import deepcopy

import torch
import torch.nn as nn
import torch.optim as optim
from transformers import AutoImageProcessor, MobileViTForImageClassification
from torchvision.models import resnet18 as ResNet18

```

2025-06-02 17:31:10.765376: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF\_ENABLE\_ONEDNN\_OPTS=0`.

2025-06-02 17:31:10.779032: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register cuFFT factory: Attempting to register factory for plugin cuFFT when one has already been registered

WARNING: All log messages before absl::InitializeLog() is called are written to STDERR

E0000 00:00:1748885470.795052 469442 cuda_dnn.cc:8579] Unable to register cudNN factory: Attempting to register factory for plugin cuDNN when one has already been registered

E0000 00:00:1748885470.799882 469442 cuda_blas.cc:1407] Unable to register cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has already been registered

W0000 00:00:1748885470.812262 469442 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.

W0000 00:00:1748885470.812276 469442 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.

W0000 00:00:1748885470.812278 469442 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.

W0000 00:00:1748885470.812279 469442 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.

2025-06-02 17:31:10.816645: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.

To enable the following instructions: AVX2 AVX512F AVX512_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

```
In [2]: logger = logging.getLogger(__name__)
handler = logging.StreamHandler()
formatter = logging.Formatter(
    '%(asctime)s - %(name)s - %(levelname)s - %(message)s'
)
handler.setFormatter(formatter)
logger.addHandler(handler)
logging.basicConfig(level=logging.INFO)
```

1.2 Загрузка датасета

```
In [3]: path = kagglehub.dataset_download("nikolasgegenava/cat-breeds")
logger.info(f"Path to dataset files: {path}")
```

```
2025-06-01 20:07:48,072 - __main__ - INFO - Path to dataset files: /home/stu
dent/.cache/kagglehub/datasets/nikolasgegenava/cat-breeds/versions/1
INFO: __main__:Path to dataset files: /home/student/.cache/kagglehub/dataset
s/nikolasgegenava/cat-breeds/versions/1
```

1.3 Предобработка датасета

```
In [3]: class DatasetStrategy(Enum):
    FLAT = "flat"
    HIERARCHICAL = "hierarchical"

    @dataclass
    class LabelMapping:
        class_to_idx: Dict[str, int]
        idx_to_class: Dict[int, str]

        @classmethod
        def from_classes(cls, classes: List[str]) -> "LabelMapping":
            class_to_idx = {class_name: idx for idx, class_name in enumerate(sorted(classes))}
            idx_to_class = {idx: class_name for class_name, idx in class_to_idx.items()}
            return cls(class_to_idx=class_to_idx, idx_to_class=idx_to_class)

    @dataclass
    class ImageData:
        path: Path
        label: Optional[int] = None
        class_name: Optional[str] = None
```

```
In [4]: class ImageDatasetLoader:

    def __init__(
        self,
        data_path: Union[str, Path],
        target_size: Tuple[int, int] = (256, 256),
        normalization_range: Tuple[float, float] = (-1.0, 1.0),
        strategy: DatasetStrategy = DatasetStrategy.HIERARCHICAL,
        force_rgb: bool = True
    ) -> None:
```

```

self.data_path = Path(data_path)
self.target_size = target_size
self.normalization_range = normalization_range
self.strategy = strategy
self.force_rgb = force_rgb
self.label_mapping: Optional[LabelMapping] = None
self._cached_image_data: Optional[List[ImageData]] = None
self._last_pattern: Optional[str] = None
self._last_limit: Optional[int] = None

self._validate_inputs()

def _validate_inputs(self) -> None:
    if not self.data_path.exists():
        raise FileNotFoundError(f>Data path does not exist: {self.data_path}</pre>
</div>
</div>
```



```

) -> List[ImageData]:
    image_paths = list(self.data_path.glob(pattern))

    if not image_paths:
        logger.warning(f"No images found with pattern '{pattern}' in {self.data_path}")
        return []

    if limit is not None and limit > 0:
        image_paths = image_paths[:limit]

    return [ImageData(path=path) for path in image_paths]

def _get_hierarchical_image_data(
    self,
    pattern: str,
    limit: Optional[int]
) -> List[ImageData]:
    classes = self._discover_classes()
    self.label_mapping = LabelMapping.from_classes(classes)

    all_image_data = []

    for class_name in classes:
        class_dir = self.data_path / class_name
        image_paths = list(class_dir.glob(pattern))

        if not image_paths:
            logger.warning(f"No images found in class '{class_name}' with pattern '{pattern}'")
            continue

        label = self.label_mapping.class_to_idx[class_name]
        class_image_data = [
            ImageData(path=path, label=label, class_name=class_name)
            for path in image_paths
        ]
        all_image_data.extend(class_image_data)

    if limit is not None and limit > 0:
        all_image_data = all_image_data[:limit]

    logger.info(f"Found {len(all_image_data)} images across {len(classes)} classes")
    return all_image_data

def _standardize_image_format(self, img: Image.Image) -> Image.Image:
    if self.force_rgb:
        if img.mode == 'RGBA':
            background = Image.new('RGB', img.size, (255, 255, 255))
            background.paste(img, mask=img.split()[-1])
            return background
        elif img.mode in ('L', 'P'):
            return img.convert('RGB')
        elif img.mode != 'RGB':
            return img.convert('RGB')
    return img

def _preprocess_image(

```

```

        self,
        image_path: Path,
        crop_box: Optional[Tuple[int, int, int, int]] = None
    ) -> np.ndarray:
        try:
            with Image.open(image_path) as img:
                img = self._standardize_image_format(img)

                if crop_box is not None:
                    img = img.crop(crop_box)

                img = img.resize(self.target_size, Image.LANCZOS)
                img_array = np.array(img, dtype=np.float32)

                if len(img_array.shape) == 2:
                    img_array = np.stack([img_array] * 3, axis=-1)

                return self._normalize_image(img_array)

        except Exception as e:
            logger.error(f"Failed to process image {image_path}: {e}")
            raise

    def _normalize_image(self, img_array: np.ndarray) -> np.ndarray:
        min_val, max_val = self.normalization_range
        img_normalized = img_array / 127.5 - 1.0
        return np.clip(img_normalized, min_val, max_val)

    def load_dataset(
        self,
        pattern: str = "*.png",
        limit: Optional[int] = None,
        crop_box: Optional[Tuple[int, int, int, int]] = None,
        batch_size: Optional[int] = None,
        return_labels: bool = True
    ) -> Union[np.ndarray, Tuple[np.ndarray, np.ndarray]]:
        image_data_list = self.get_image_data(pattern, limit)

        if not image_data_list:
            if return_labels and self.strategy == DatasetStrategy.HIERARCHICAL:
                return np.array([]), np.array([])
            return np.array([])

        logger.info(f"Loading dataset with {len(image_data_list)} images")

        if batch_size is not None:
            images, labels = self._load_in_batches(image_data_list, crop_box)
        else:
            images, labels = self._load_all_at_once(image_data_list, crop_box)

        if return_labels and self.strategy == DatasetStrategy.HIERARCHICAL:
            return images, labels
        return images

    def _load_all_at_once(
        self,

```

```

        image_data_list: List[ImageData],
        crop_box: Optional[Tuple[int, int, int, int]]
    ) -> Tuple[np.ndarray, Optional[np.ndarray]]:
        processed_images = []
        labels = []

        for i, image_data in enumerate(image_data_list):
            if i % 1000 == 0:
                logger.info(f"Processed {i}/{len(image_data_list)} images")

            try:
                processed_img = self._preprocess_image(image_data.path, crop_box)
                processed_images.append(processed_img)

                if image_data.label is not None:
                    labels.append(image_data.label)
            except Exception as e:
                logger.warning(f"Skipping corrupted image {image_data.path}: {e}")
                continue

        if not processed_images:
            images = np.array([])
            labels_array = np.array([])
        else:
            images = np.array(processed_images, dtype=np.float32)
            labels_array = np.array(labels, dtype=np.int64) if labels else None

        logger.info(f"Dataset loaded successfully. Images shape: {images.shape}")
        if labels_array is not None and labels_array.size > 0:
            logger.info(f"Labels shape: {labels_array.shape}")

        return images, labels_array

    def _load_in_batches(
        self,
        image_data_list: List[ImageData],
        crop_box: Optional[Tuple[int, int, int, int]],
        batch_size: int
    ) -> Tuple[np.ndarray, Optional[np.ndarray]]:
        all_image_batches = []
        all_label_batches = []

        for i in range(0, len(image_data_list), batch_size):
            batch_data = image_data_list[i:i + batch_size]
            batch_images = []
            batch_labels = []

            for image_data in batch_data:
                try:
                    processed_img = self._preprocess_image(image_data.path, crop_box)
                    batch_images.append(processed_img)

                    if image_data.label is not None:
                        batch_labels.append(image_data.label)
                except Exception as e:
                    logger.warning(f"Skipping corrupted image {image_data.path}: {e}")

```

```

        continue

    if batch_images:
        batch_images_array = np.array(batch_images, dtype=np.float32)
        all_image_batches.append(batch_images_array)

    if batch_labels:
        batch_labels_array = np.array(batch_labels, dtype=np.int)
        all_label_batches.append(batch_labels_array)

    batch_num = i // batch_size + 1
    total_batches = (len(image_data_list) + batch_size - 1) // batch_size
    logger.info(f"Processed batch {batch_num}/{total_batches}")

    if not all_image_batches:
        images = np.array([])
        labels = np.array([])
    else:
        images = np.concatenate(all_image_batches, axis=0)
        labels = np.concatenate(all_label_batches, axis=0) if all_label_batches else None

    logger.info(f"Dataset loaded successfully. Images shape: {images.shape}")
    if labels is not None and labels.size > 0:
        logger.info(f"Labels shape: {labels.shape}")

    return images, labels

def get_class_distribution(self) -> Optional[Dict[str, int]]:
    if self._cached_image_data is None:
        return None

    distribution = {}
    for image_data in self._cached_image_data:
        if image_data.class_name:
            distribution[image_data.class_name] = distribution.get(image_data.class_name, 0) + 1

    return distribution

```

```

In [5]: def load_image_dataset_with_info(
    data_path: Union[str, Path],
    target_size: Tuple[int, int] = (256, 256),
    pattern: str = "/*.png",
    limit: Optional[int] = None,
    crop_box: Optional[Tuple[int, int, int, int]] = None,
    normalization_range: Tuple[float, float] = (-1.0, 1.0),
    batch_size: Optional[int] = None,
    strategy: DatasetStrategy = DatasetStrategy.HIERARCHICAL,
    return_labels: bool = True,
    force_rgb: bool = True
) -> Tuple[Union[np.ndarray, Tuple[np.ndarray, np.ndarray]], Optional[LabelMap]]:
    loader = ImageDatasetLoader(
        data_path=data_path,
        target_size=target_size,
        normalization_range=normalization_range,
        strategy=strategy,
        force_rgb=force_rgb
    )
    images, labels = loader.load_data(limit=limit)
    return images, labels

```

```

)

dataset = loader.load_dataset(
    pattern=pattern,
    limit=limit,
    crop_box=crop_box,
    batch_size=batch_size,
    return_labels=return_labels
)

return dataset, loader.label_mapping, loader.get_class_distribution()

```

1.5 Посмотрим на датасет

```

In [7]: class DatasetVisualizer:

    def __init__(self, logger: Optional[logging.Logger] = None):
        self.logger = logger or logging.getLogger(__name__)

    def plot_images(
        self,
        dataset: np.ndarray,
        labels: Optional[np.ndarray] = None,
        class_names: Optional[Dict[int, str]] = None,
        rows: int = 6,
        cols: int = 6,
        figsize: Tuple[int, int] = (15, 16),
        title: str = 'Dataset Images'
    ) -> None:
        self._validate_dataset(dataset)

        total_plots = rows * cols
        images_to_plot = min(len(dataset), total_plots)

        self.logger.info(f"Dataset shape: {dataset.shape}, Plotting {images_to_plot} images")

        fig, axes = self._create_subplot_grid(len(dataset), rows, cols, figsize=figsize)

        try:
            self._plot_images_on_axes(dataset, axes, images_to_plot, labels, class_names)
            self._finalize_plot(fig, title)
            self.logger.info("Visualization completed successfully")

        except Exception as ex:
            self.logger.error(f"Plotting failed: {ex}")
            plt.close(fig)
            raise

    def _validate_dataset(self, dataset: np.ndarray) -> None:
        if dataset.size == 0:
            self.logger.error("Dataset validation failed: empty dataset")
            raise ValueError("Dataset cannot be empty")

        if len(dataset.shape) < 3:

```

```

        self.logger.error(f"Dataset validation failed: invalid shape {dataset.shape}")
        raise ValueError(f"Dataset must have at least 3 dimensions, got {dataset.shape}")

    self.logger.debug(f"Dataset validation passed: shape {dataset.shape}")

    def _create_subplot_grid(
        self,
        dataset_size: int,
        rows: int,
        cols: int,
        figsize: Tuple[int, int]
    ) -> Tuple:
        total_plots = rows * cols

        if dataset_size < total_plots:
            actual_rows = (dataset_size + cols - 1) // cols
            fig, axes = plt.subplots(actual_rows, cols, figsize=figsize)
            if actual_rows == 1:
                axes = axes.reshape(1, -1)
        else:
            fig, axes = plt.subplots(rows, cols, figsize=figsize)

        return fig, axes

    def _plot_images_on_axes(
        self,
        dataset: np.ndarray,
        axes: np.ndarray,
        images_to_plot: int,
        labels: Optional[np.ndarray] = None,
        class_names: Optional[Dict[int, str]] = None
    ) -> None:
        for i, ax in enumerate(axes.flat):
            if i >= images_to_plot:
                ax.axis('off')
                continue

            img = dataset[i]
            img_rescaled = np.clip((img + 1) / 2, 0, 1)

            ax.imshow(img_rescaled)
            ax.axis('off')

            if labels is not None and i < len(labels):
                label_text = self._get_label_text(labels[i], class_names)
                ax.set_title(label_text, fontsize=10, pad=5)

    def _get_label_text(self, label: Union[int, str], class_names: Optional[Dict[int, str]] = None) -> str:
        if class_names and isinstance(label, (int, np.integer)) and label in class_names:
            return class_names[label]
        return str(label)

    def _finalize_plot(self, fig, title: str) -> None:
        fig.suptitle(title, fontsize=25)
        plt.tight_layout()
        plt.show()

```

1.6 Строим модель

```
In [8]: class GANType(Enum):  
        VANILLA = "vanilla"  
        CONDITIONAL = "conditional"
```

```
In [9]: @dataclass  
        class ModelConfig:  
            image_shape: Tuple[int, int, int] = (256, 256, 3)  
            latent_dim: int = 100  
            learning_rate: float = 0.0002  
            beta_1: float = 0.5  
            num_classes: Optional[int] = None
```

```
In [10]: @dataclass  
        class ArchitectureConfig:  
            base_filters: int = 64  
            dropout_rate: float = 0.1  
            leaky_relu_alpha: float = 0.2  
            embedding_dim: int = 50  
            dense_units: int = 256
```

```
In [30]: class BaseModelPartBuilder(ABC):  
  
        def __init__(self, model_config: ModelConfig, arch_config: ArchitectureC  
            self.model_config = model_config  
            self.arch_config = arch_config  
  
        @abstractmethod  
        def build(self) -> Model:  
            pass  
  
        def _create_optimizer(self, lr_multiplier: float = 0.99999) -> Adam:  
            return Adam(  
                learning_rate=self.model_config.learning_rate * lr_multiplier,  
                beta_1=self.model_config.beta_1  
            )
```

```
In [12]: class GeneratorBuilder(BaseModelPartBuilder):  
  
        def build(self) -> Model:  
            if self.model_config.num_classes is not None:  
                return self._build_conditional_generator()  
            return self._build_vanilla_generator()  
  
        def _calculate_initial_size(self) -> int:  
            target_size = self.model_config.image_shape[0]  
            upsampling_layers = 5 # Number of times spatial dimensions are doub  
            initial_size = target_size // (2 ** upsampling_layers)  
            if initial_size < 1:  
                raise ValueError(  
                    f"Calculated initial size is {initial_size}, which is too sm
```

```

        f"Target image_shape {self.model_config.image_shape[0]} may
        f"for {upsampling_layers} upsampling layers, or base_filters
    )
    return initial_size

def _create_transpose_conv_block(self, x: tf.Tensor, filters: int) -> tf.Tensor:
    x = Conv2DTranspose(
        filters,
        kernel_size=(4, 4),
        strides=(2, 2),
        padding='same',
        use_bias=False
    )(x)
    x = BatchNormalization()(x)
    x = LeakyReLU(self.arch_config.leaky_relu_alpha)(x)
    return x

def _build_vanilla_generator(self) -> Model:
    initial_size = self._calculate_initial_size()
    initial_filters = self.arch_config.base_filters * (2 ** (5-1)) # e.g. 16

    model = Sequential(name="vanilla_generator")
    model.add(Dense(
        initial_size * initial_size * initial_filters,
        input_dim=self.model_config.latent_dim,
        use_bias=False
    ))
    model.add(BatchNormalization())
    model.add(LeakyReLU(self.arch_config.leaky_relu_alpha))
    model.add(Reshape((initial_size, initial_size, initial_filters)))

    # Example: For 256x256 output from initial_size=16 (16 -> 32 -> 64 -> 128 -> 256)
    current_filters = initial_filters
    for _ in range(5):
        current_filters //= 2
        model.add(Conv2DTranspose(
            current_filters, (4,4), strides=(2,2), padding='same', use_bias=False
        ))
        model.add(BatchNormalization())
        model.add(LeakyReLU(self.arch_config.leaky_relu_alpha))

    model.add(Conv2DTranspose(
        self.model_config.image_shape[2], # Number of channels
        kernel_size=(4, 4),
        strides=(1,1), # Final layer might use stride 1 if size is already 256
        padding='same',
        activation='tanh'
    ))

    assert model.output_shape == (None, *self.model_config.image_shape), \
        f"Generator output shape {model.output_shape} does not match target {self.model_config.image_shape}"

    return model

def _build_conditional_generator(self) -> Model:
    if self.model_config.num_classes is None:

```



```

        raise ValueError("num_classes must be specified for conditional

noise_input = Input(shape=(self.model_config.latent_dim,), name="noi
label_input = Input(shape=(1,), name="label_input", dtype='int32')

label_embedding = Embedding(
    self.model_config.num_classes,
    self.arch_config.embedding_dim,
    name="label_embedding"
)(label_input)
label_embedding = Flatten(name="flatten_embedding")(label_embedding)

concatenated_input = Concatenate(name="concatenate_inputs")([noise_i

initial_size = self._calculate_initial_size()
initial_filters = self.arch_config.base_filters * (2 ** (5-1))

x = Dense(
    initial_size * initial_size * initial_filters,
    use_bias=False,
    name="dense_projection"
)(concatenated_input)
x = BatchNormalization(name="bn_projection")(x)
x = LeakyReLU(self.arch_config.leaky_relu_alpha, name="leaky_relu_pr
x = Reshape((initial_size, initial_size, initial_filters), name="res

current_filters = initial_filters
for i in range(5): # 4 upsampling blocks
    current_filters //= 2
    x = self._create_transpose_conv_block(x, current_filters)

generated_image = Conv2DTranspose(
    self.model_config.image_shape[2], # Channels
    kernel_size=(4, 4),
    strides=(1,1),
    padding='same',
    activation='tanh',
    name="output_image"
)(x)

model = Model([noise_input, label_input], generated_image, name="cor
return model

```

In [31]: `class DiscriminatorBuilder(BaseModelPartBuilder):`

```

def build(self) -> Model:
    if self.model_config.num_classes is not None:
        model = self._build_conditional_discriminator()
    else:
        model = self._build_vanilla_discriminator()

    model.compile(
        loss='binary_crossentropy',
        optimizer=self._create_optimizer(lr_multiplier=0.99999),
        metrics=['accuracy']
    )

```

```

        return model

def _create_conv_block(self, x: tf.Tensor, filters: int, apply_batchnorm: bool):
    x = Conv2D(
        filters,
        kernel_size=(4, 4),
        strides=(strides, strides),
        padding='same',
        use_bias=not apply_batchnorm
    )(x)
    if apply_batchnorm:
        x = BatchNormalization()(x)
    x = LeakyReLU(self.arch_config.leaky_relu_alpha)(x)
    x = Dropout(self.arch_config.dropout_rate)(x)
    return x

def _build_vanilla_discriminator(self) -> Model:
    model = Sequential(name="vanilla_discriminator")
    model.add(Conv2D(
        self.arch_config.base_filters,
        (4, 4),
        strides=(2, 2),
        padding='same',
        input_shape=self.model_config.image_shape
    ))
    model.add(LeakyReLU(self.arch_config.leaky_relu_alpha))
    model.add(Dropout(self.arch_config.dropout_rate))

    num_conv_blocks = 5 # e.g. for 256x256 -> 128 -> 64 -> 32 -> 16 (4 conv blocks)
    current_filters = self.arch_config.base_filters
    for i in range(num_conv_blocks):
        current_filters *= 2
        model.add(Conv2D(current_filters, (4,4), strides=(2,2), padding='same'))
        model.add(BatchNormalization())
        model.add(LeakyReLU(self.arch_config.leaky_relu_alpha))
        model.add(Dropout(self.arch_config.dropout_rate))

    model.add(Flatten())
    model.add(Dense(self.arch_config.dense_units))
    model.add(LeakyReLU(self.arch_config.leaky_relu_alpha))
    model.add(Dropout(self.arch_config.dropout_rate))
    model.add(Dense(1, activation='sigmoid'))

    return model

def _build_conditional_discriminator(self) -> Model:
    """Builds a conditional discriminator model."""
    if self.model_config.num_classes is None:
        raise ValueError("num_classes must be specified for conditional discriminator")

    image_input = Input(shape=self.model_config.image_shape, name="image_input")
    label_input = Input(shape=(1,), name="label_input", dtype='int32')

    embedding_output_dim = self.model_config.image_shape[0] * self.model_config.image_shape[1]
    label_embedding = Embedding(
        self.model_config.num_classes,
        embedding_output_dim
    )(label_input)

    image_embedding = Conv2D(
        self.arch_config.base_filters,
        (4, 4),
        strides=(2, 2),
        padding='same',
        input_shape=self.model_config.image_shape
    )(image_input)

    image_embedding = BatchNormalization()(image_embedding)
    image_embedding = LeakyReLU(self.arch_config.leaky_relu_alpha)(image_embedding)
    image_embedding = Dropout(self.arch_config.dropout_rate)(image_embedding)

    combined_input = Concatenate()([image_embedding, label_embedding])
    combined_input = Conv2D(
        self.arch_config.base_filters,
        (4, 4),
        strides=(2, 2),
        padding='same'
    )(combined_input)

    combined_input = BatchNormalization()(combined_input)
    combined_input = LeakyReLU(self.arch_config.leaky_relu_alpha)(combined_input)
    combined_input = Dropout(self.arch_config.dropout_rate)(combined_input)

    combined_input = Flatten()(combined_input)
    combined_input = Dense(self.arch_config.dense_units)(combined_input)
    combined_input = LeakyReLU(self.arch_config.leaky_relu_alpha)(combined_input)
    combined_input = Dropout(self.arch_config.dropout_rate)(combined_input)
    combined_input = Dense(1, activation='sigmoid')(combined_input)

    return combined_input

```

```

        embedding_output_dim,
        name="label_embedding"
    )(label_input)

    label_embedding_resaped = Reshape(
        (self.model_config.image_shape[0], self.model_config.image_shape
        name="reshape_embedding_to_channel"
    )(label_embedding)

    concatenated_input = Concatenate(name="concatenate_image_label")([im

    x = self._create_conv_block(concatenated_input, self.arch_config.bas

    num_conv_blocks = 5
    current_filters = self.arch_config.base_filters
    for i in range(num_conv_blocks):
        current_filters *= 2
        x = self._create_conv_block(x, current_filters, apply_batchnorm=

    x = Flatten(name="flatten_features")(x)
    x = Dense(self.arch_config.dense_units, name="dense_intermediate")(x)
    x = LeakyReLU(self.arch_config.leaky_relu_alpha, name="leaky_relu_ir
    x = Dropout(self.arch_config.dropout_rate, name="dropout_intermediat
    validity = Dense(1, activation='sigmoid', name="output_validity")(x)

    model = Model([image_input, label_input], validity, name="conditiona
    return model

```

In [14]: **class** GANBuilder:

```

    def __init__(self, model_config: ModelConfig, arch_config: Optional[Arch
        self.model_config = model_config
        self.arch_config = arch_config or ArchitectureConfig()
        self.generator_builder = GeneratorBuilder(self.model_config, self.ar
        self.discriminator_builder = DiscriminatorBuilder(self.model_config,

    def build_generator(self) -> Model:
        return self.generator_builder.build()

    def build_discriminator(self) -> Model:
        return self.discriminator_builder.build()

    def build_gan_model(self, generator: Model, discriminator: Model) -> Moc
        discriminator.trainable = True

    if self.model_config.num_classes is not None:
        noise_input = Input(shape=(self.model_config.latent_dim,), name=
        label_input = Input(shape=(1,), name="gan_label_input", dtype='i

        fake_image = generator([noise_input, label_input])
        validity = discriminator([fake_image, label_input])

        gan_model = Model([noise_input, label_input], validity, name="cc
    else:
        # Vanilla GAN
        noise_input = Input(shape=(self.model_config.latent_dim,), name=

```

```

        fake_image = generator(noise_input)
        validity = discriminator(fake_image)

        gan_model = Model(noise_input, validity, name="combined_vanilla_

gan_model.compile(
    loss='binary_crossentropy',
    optimizer=Adam(
        learning_rate=self.model_config.learning_rate,
        beta_1=self.model_config.beta_1
    ),
    metrics=['accuracy']
)
return gan_model

def build_all(self) -> Tuple[Model, Model, Model]:
    generator = self.build_generator()
    discriminator = self.build_discriminator()
    gan_model = self.build_gan_model(generator, discriminator)
    return generator, discriminator, gan_model

```

```

In [15]: class GANModelParts(Protocol):
    def train_on_batch(self, x, y) -> Tuple[float, float]: ...
    def predict(self, x, verbose: int = 0) -> np.ndarray: ...
    def save_weights(self, filepath: str) -> None: ...
    def load_weights(self, filepath: str) -> None: ...

```

1.7 Доп. функции

```

In [64]: class ModelWeightsManager:
    def __init__(self, base_output_dir: str):
        self.base_output_dir = Path(base_output_dir)
        self.logger = logging.getLogger(__name__)

    def save_model_weights(
        self,
        generator_model: Model,
        discriminator_model: Model,
        suffix: Optional[str] = None
    ) -> Path:
        timestamp = datetime.now().strftime("%d_%m_%Y_%H_%M_%S")
        dir_name = f"model_weights_{timestamp}"
        if suffix:
            dir_name += f"_{suffix}"

        output_dir = self.base_output_dir / dir_name
        output_dir.mkdir(parents=True, exist_ok=True)

        # Corrected filenames
        generator_path = output_dir / "generator.weights.h5"
        discriminator_path = output_dir / "discriminator.weights.h5"

        generator_model.save_weights(str(generator_path))

```

```

        discriminator_model.save_weights(str(discriminator_path))

        self.logger.info(f"Model weights saved to {output_dir}")
        return output_dir

    def load_model_weights(
        self,
        generator_model: Model,
        discriminator_model: Model,
        weights_dir: str
    ) -> None:
        weights_path = Path(weights_dir)

        if not weights_path.exists():
            raise FileNotFoundError(f"Weights directory '{weights_path}' does not exist")

        # Corrected filenames to match the saving convention
        generator_file = weights_path / "generator.weights.h5"
        discriminator_file = weights_path / "discriminator.weights.h5"

        for file_path, model_name in [
            (generator_file, "generator"),
            (discriminator_file, "discriminator")
        ]:
            if not file_path.exists():
                raise FileNotFoundError(f"{model_name} weights file '{file_path}' does not exist")

        generator_model.load_weights(str(generator_file), skip_mismatch=True)
        discriminator_model.load_weights(str(discriminator_file), skip_mismatch=True)

        self.logger.info(f"Model weights loaded successfully from {weights_dir}")

```

```

In [17]: class SampleGeneratorService:
    def __init__(self, is_conditional: bool = False, num_classes: Optional[int] = None):
        self.is_conditional = is_conditional
        self.num_classes = num_classes

        if is_conditional and num_classes is None:
            raise ValueError("num_classes must be specified for conditional GAN")

    def generate_noise_samples(self, num_samples: int, noise_dim: int) -> np.ndarray:
        return np.random.randn(num_samples, noise_dim).astype(np.float32)

    def generate_random_labels(self, num_samples: int) -> np.ndarray:
        if not self.is_conditional:
            raise ValueError("Cannot generate labels for non-conditional GAN")
        return np.random.randint(0, self.num_classes, (num_samples, 1)).astype(np.int32)

    def generate_real_samples(
        self,
        dataset: np.ndarray,
        num_samples: int,
        labels: Optional[np.ndarray] = None
    ) -> Tuple[np.ndarray, np.ndarray, Optional[np.ndarray]]:
        indices = np.random.randint(0, dataset.shape[0], num_samples)
        real_images = dataset[indices].astype(np.float32)

```

```

real_y_labels = np.ones((num_samples, 1), dtype=np.float32)

if self.is_conditional and labels is not None:
    real_class_labels = labels[indices].astype(np.int32)
    if real_class_labels.ndim == 1:
        real_class_labels = real_class_labels.reshape(-1, 1)
    return real_images, real_y_labels, real_class_labels

return real_images, real_y_labels, None

def generate_fake_samples(
    self,
    generator: Model,
    noise_dim: int,
    num_samples: int
) -> Tuple[np.ndarray, np.ndarray, Optional[np.ndarray]]:
    noise = self.generate_noise_samples(num_samples, noise_dim)
    fake_y_labels = np.zeros((num_samples, 1), dtype=np.float32)

    if self.is_conditional:
        fake_class_labels = self.generate_random_labels(num_samples)
        fake_images = generator.predict([noise, fake_class_labels], verbose=0)
        fake_images = fake_images.astype(np.float32)
        return fake_images, fake_y_labels, fake_class_labels

    fake_images = generator.predict(noise, verbose=0)
    fake_images = fake_images.astype(np.float32)
    return fake_images, fake_y_labels, None

```

```

In [18]: class GANVisualizationService:
    def __init__(
        self,
        is_conditional: bool = False,
        num_classes: Optional[int] = None,
        logger: Optional[logging.Logger] = None
    ):
        self.is_conditional = is_conditional
        self.num_classes = num_classes
        self.logger = logger or logging.getLogger(__name__)

        if is_conditional and num_classes is None:
            raise ValueError("num_classes must be specified for conditional")

    def generate_images(
        self,
        generator: Model,
        epoch: int,
        num_samples: int = 6,
        noise_dim: int = 100,
        class_labels: Optional[np.ndarray] = None
    ) -> np.ndarray:
        noise = np.random.randn(num_samples, noise_dim)

        if self.is_conditional:
            if class_labels is None:
                class_labels = np.random.randint(0, self.num_classes, (num_s

```

```

        images = generator.predict([noise, class_labels], verbose=0)
    else:
        images = generator.predict(noise, verbose=0)

    return (images + 1) / 2

def plot_generated_images(
    self,
    generator: Model,
    epoch: int,
    num_samples: int = 6,
    noise_dim: int = 100,
    class_labels: Optional[np.ndarray] = None
) -> None:
    import matplotlib.pyplot as plt

    images = self.generate_images(generator, epoch, num_samples, noise_dim, class_labels)

    fig, axes = plt.subplots(1, num_samples, figsize=(15, 3))
    for i in range(num_samples):
        axes[i].imshow(images[i])
        axes[i].axis('off')

    fig.suptitle(f"Generated Images at Epoch {epoch + 1}", fontsize=22)
    plt.tight_layout()
    plt.show()

```

1.8 Трейнер

```

In [19]: @dataclass
class TrainingConfig:
    num_epochs: int = 100
    batch_size: int = 32
    noise_dimension: int = 100
    is_conditional: bool = False
    num_classes: Optional[int] = None
    display_frequency: int = 10
    save_frequency: int = 20
    log_frequency: int = 50
    verbose: bool = True
    real_time_visualization: bool = False
    weights_save_path: Optional[str] = "gan_weights"
    weights_checkpoint_dir: Optional[str] = None

@dataclass
class TrainingMetrics:
    discriminator_loss: float
    discriminator_accuracy: float
    generator_loss: float
    epoch: int
    batch: Optional[int] = None
    epoch_total_batches: Optional[int] = None
    epoch_total_epochs: Optional[int] = None

```

```
timestamp: float = field(default_factory=time.time)
strategy_metrics: Dict[str, Any] = field(default_factory=dict)
```

```
In [20]: class RealTimeVisualizer:
    def __init__(
        self,
        is_conditional: bool = False,
        num_classes: Optional[int] = None,
        grid_size: Tuple[int, int] = (2, 3),
        figsize: Tuple[int, int] = (12, 8)
    ):
        self.is_conditional = is_conditional
        self.num_classes = num_classes
        self.grid_size = grid_size
        self.figsize = figsize
        self.sample_count = grid_size[0] * grid_size[1]

        plt.style.use('default')
        plt.ion()

    def generate_sample_images(
        self,
        generator: Model,
        epoch: int,
        noise_dim: int = 100
    ) -> np.ndarray:
        noise = np.random.randn(self.sample_count, noise_dim).astype(np.float32)

        if self.is_conditional and self.num_classes:
            class_labels = np.random.randint(
                0, self.num_classes, (self.sample_count, 1)
            ).astype(np.int32)
            images = generator.predict([noise, class_labels], verbose=0)
        else:
            images = generator.predict(noise, verbose=0)

        return (images + 1) / 2

    def update_visualization(
        self,
        generator: Model,
        epoch: int,
        batch: int,
        metrics: 'TrainingMetrics',
        total_batches: int
    ) -> None:
        clear_output(wait=True)

        fig = plt.figure(figsize=self.figsize)

        gs = fig.add_gridspec(2, 2, height_ratios=[3, 1], width_ratios=[2, 1])

        images_ax = fig.add_subplot(gs[0, :])
        loss_ax = fig.add_subplot(gs[1, 0])
        acc_ax = fig.add_subplot(gs[1, 1])
```



```

images = self.generate_sample_images(generator, epoch)

for i in range(self.sample_count):
    row = i // self.grid_size[1]
    col = i % self.grid_size[1]

    x_pos = col / self.grid_size[1]
    y_pos = 1 - (row + 1) / self.grid_size[0]
    width = 1 / self.grid_size[1]
    height = 1 / self.grid_size[0]

    ax = fig.add_axes([x_pos, y_pos, width, height])
    ax.imshow(images[i])
    ax.axis('off')

images_ax.set_title(
    f'Generated Images - Epoch {epoch + 1}, Batch {batch + 1}/{total_batches}'
    fontsize=14,
    fontweight='bold'
)
images_ax.axis('off')

progress = (batch + 1) / total_batches
loss_ax.barh(['D_Loss', 'G_Loss'],
             [metrics.discriminator_loss, metrics.generator_loss],
             color=['red', 'blue'], alpha=0.7)
loss_ax.set_xlim(0, 2)
loss_ax.set_title('Losses')
loss_ax.grid(True, alpha=0.3)

acc_ax.barh(['D_Acc', 'Progress'],
            [metrics.discriminator_accuracy * 100, progress * 100],
            color=['green', 'orange'], alpha=0.7)
acc_ax.set_xlim(0, 100)
acc_ax.set_title('Accuracy & Progress (%)')
acc_ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

```

In [21]: class ProgressTracker:
    def __init__(self, log_frequency: int = 50):
        self.log_frequency = log_frequency
        self.start_time = time.time()
        self.epoch_start_time = time.time()

    def should_log(self, batch: int) -> bool:
        return batch % self.log_frequency == 0 or batch == 0

    def log_progress(
        self,
        metrics: TrainingMetrics,
        total_batches: int,
        total_epochs: int
    ) -> None:
        if not self.should_log(metrics.batch):

```

```

        return

    current_time = time.time()
    elapsed_total = current_time - self.start_time
    elapsed_epoch = current_time - self.epoch_start_time

    if metrics.batch == 0:
        self.epoch_start_time = current_time

    epoch_progress = (metrics.batch + 1) / total_batches
    total_progress = (metrics.epoch + epoch_progress) / total_epochs

    eta_epoch = (elapsed_epoch / epoch_progress) * (1 - epoch_progress)
    eta_total = (elapsed_total / total_progress) * (1 - total_progress)

    print(f"\r Epoch {metrics.epoch + 1}/{total_epochs} | "
          f"Batch {metrics.batch + 1}/{total_batches} | "
          f"D_loss: {metrics.discriminator_loss:.4f} | "
          f"D_acc: {metrics.discriminator_accuracy * 100:.1f}% | "
          f"G_loss: {metrics.generator_loss:.4f} | "
          f"ETA: {eta_epoch:.0f}s", end='', flush=True)

```

```

In [22]: class EnhancedGANTrainer:
    def __init__(
        self,
        generator: Model,
        discriminator: Model,
        gan: Model,
        config: TrainingConfig,
        weights_manager: Optional['ModelWeightsManager'] = None
    ):
        self.generator = generator
        self.discriminator = discriminator
        self.gan = gan
        self.config = config
        self.weights_manager = weights_manager

        if config.weights_checkpoint_dir and self.weights_manager:
            self.weights_manager.load_model_weights(
                generator_model=generator,
                discriminator_model=discriminator,
                weights_dir=config.weights_checkpoint_dir
            )

        self.sample_generator = SampleGeneratorService(
            is_conditional=config.is_conditional,
            num_classes=config.num_classes
        )

        if config.real_time_visualization:
            self.visualizer = RealTimeVisualizer(
                is_conditional=config.is_conditional,
                num_classes=config.num_classes
            )
        else:
            self.visualizer = None

```

```

self.progress_tracker = ProgressTracker(config.log_frequency)
self.logger = self._setup_logger()
self.training_history: List[TrainingMetrics] = []
self.saved_images: List[np.ndarray] = []

def _setup_logger(self) -> logging.Logger:
    logger = logging.getLogger(f"GANTrainer_{id(self)}")
    logger.setLevel(logging.INFO if self.config.verbose else logging.WARN)

    if not logger.handlers:
        handler = logging.StreamHandler()
        formatter = logging.Formatter(
            '%(asctime)s - %(name)s - %(levelname)s - %(message)s'
        )
        handler.setFormatter(formatter)
        logger.addHandler(handler)

    return logger

def _prepare_conditional_data(
    self,
    images: np.ndarray,
    labels: np.ndarray,
    y_labels: np.ndarray
) -> Tuple[List[tf.Tensor], tf.Tensor]:
    if labels.ndim == 1:
        labels = labels.reshape(-1, 1)

    images_tensor = tf.convert_to_tensor(images, dtype=tf.float32)
    labels_tensor = tf.convert_to_tensor(labels, dtype=tf.int32)
    y_labels_tensor = tf.convert_to_tensor(y_labels, dtype=tf.float32)

    return [images_tensor, labels_tensor], y_labels_tensor

def _prepare_vanilla_data(
    self,
    images: np.ndarray,
    y_labels: np.ndarray
) -> Tuple[tf.Tensor, tf.Tensor]:
    images_tensor = tf.convert_to_tensor(images, dtype=tf.float32)
    y_labels_tensor = tf.convert_to_tensor(y_labels, dtype=tf.float32)

    return images_tensor, y_labels_tensor

def _train_discriminator_batch(
    self,
    dataset: np.ndarray,
    half_batch_size: int,
    labels: Optional[np.ndarray] = None
) -> Tuple[float, float]:
    if self.config.is_conditional and labels is not None:
        real_data = self.sample_generator.generate_real_samples(
            dataset, half_batch_size, labels
        )
        real_images, real_y_labels, real_class_labels = real_data

```

```

        fake_data = self.sample_generator.generate_fake_samples(
            self.generator, self.config.noise_dimension, half_batch_size
        )
        fake_images, fake_y_labels, fake_class_labels = fake_data

        real_inputs, real_targets = self._prepare_conditional_data(
            real_images, real_class_labels, real_y_labels
        )
        fake_inputs, fake_targets = self._prepare_conditional_data(
            fake_images, fake_class_labels, fake_y_labels
        )

        real_loss, real_acc = self.discriminator.train_on_batch(real_inputs, real_targets)
        fake_loss, fake_acc = self.discriminator.train_on_batch(fake_inputs, fake_targets)
    else:
        real_data = self.sample_generator.generate_real_samples(dataset,
            real_images, real_y_labels, _ = real_data

        fake_data = self.sample_generator.generate_fake_samples(
            self.generator, self.config.noise_dimension, half_batch_size
        )
        fake_images, fake_y_labels, _ = fake_data

        real_inputs, real_targets = self._prepare_vanilla_data(real_images, real_y_labels)
        fake_inputs, fake_targets = self._prepare_vanilla_data(fake_images, fake_y_labels)

        real_loss, real_acc = self.discriminator.train_on_batch(real_inputs, real_targets)
        fake_loss, fake_acc = self.discriminator.train_on_batch(fake_inputs, fake_targets)

    avg_loss = 0.5 * (real_loss + fake_loss)
    avg_acc = 0.5 * (real_acc + fake_acc)

    return float(avg_loss), float(avg_acc)

def _train_generator_batch(self, batch_size: int) -> float:
    noise = self.sample_generator.generate_noise_samples(batch_size, self.config.noise_dimension)
    gan_labels = np.ones((batch_size, 1), dtype=np.float32)

    if self.config.is_conditional:
        class_labels = self.sample_generator.generate_random_labels(batch_size, self.config.class_labels)
        noise_tensor = tf.convert_to_tensor(noise, dtype=tf.float32)
        labels_tensor = tf.convert_to_tensor(class_labels, dtype=tf.int32)
        targets_tensor = tf.convert_to_tensor(gan_labels, dtype=tf.float32)

        loss, _ = self.gan.train_on_batch([noise_tensor, labels_tensor], targets_tensor)
    else:
        noise_tensor = tf.convert_to_tensor(noise, dtype=tf.float32)
        targets_tensor = tf.convert_to_tensor(gan_labels, dtype=tf.float32)

        loss, _ = self.gan.train_on_batch(noise_tensor, targets_tensor)

    return float(loss)

def _handle_epoch_checkpoint(self, epoch: int) -> None:
    if (epoch % self.config.save_frequency == 0 and

```

```

        epoch != 0 and
        self.weights_manager is not None):
    self.weights_manager.save_model_weights(
        self.generator,
        self.discriminator,
        suffix=f"epoch_{epoch}"
    )
    print(f"\n Model weights saved at epoch {epoch + 1}")

def train(
    self,
    dataset: np.ndarray,
    labels: Optional[np.ndarray] = None
) -> List[np.ndarray]:
    dataset = dataset.astype(np.float32)
    if labels is not None:
        labels = labels.astype(np.int32)

    batches_per_epoch = dataset.shape[0] // self.config.batch_size
    half_batch_size = self.config.batch_size // 2

    self.logger.info(f" Starting GAN training")
    self.logger.info(f" Dataset: {dataset.shape[0]} samples, {batches_per_epoch} batches")
    self.logger.info(f" Config: {self.config.num_epochs} epochs, {batches_per_epoch} batches")

    print(f" Training started: {self.config.num_epochs} epochs, {batches_per_epoch} batches")

    for epoch in range(self.config.num_epochs):
        epoch_metrics = []

        for batch_num in range(batches_per_epoch):
            disc_loss, disc_acc = self._train_discriminator_batch(
                dataset, half_batch_size, labels
            )
            gen_loss = self._train_generator_batch(self.config.batch_size, dataset, half_batch_size, labels)

            metrics = TrainingMetrics(
                discriminator_loss=disc_loss,
                discriminator_accuracy=disc_acc,
                generator_loss=gen_loss,
                epoch=epoch,
                batch=batch_num,
                timestamp=time.time()
            )

            epoch_metrics.append(metrics)

        if self.config.real_time_visualization and self.visualizer:
            if batch_num % self.config.display_frequency == 0:
                self.visualizer.update_visualization(
                    self.generator, epoch, batch_num, metrics, batch_num
                )
            else:
                self.progress_tracker.log_progress(
                    metrics, batches_per_epoch, self.config.num_epochs
                )

```

```

        self.training_history.extend(epoch_metrics)
        self._handle_epoch_checkpoint(epoch)

        if not self.config.real_time_visualization:
            print()

        self.logger.info(" Training completed successfully")
        print(f"\n Training completed! Total time: {time.time() - self.start_time}")

        return self.saved_images

def get_training_summary(self) -> Dict[str, Any]:
    if not self.training_history:
        return {}

    recent_metrics = self.training_history[-10:]

    return {
        "total_epochs": len(set(m.epoch for m in self.training_history)),
        "total_batches": len(self.training_history),
        "avg_discriminator_loss": float(np.mean([m.discriminator_loss for m in recent_metrics])),
        "avg_discriminator_accuracy": float(np.mean([m.discriminator_accuracy for m in recent_metrics])),
        "avg_generator_loss": float(np.mean([m.generator_loss for m in recent_metrics])),
        "final_epoch": self.training_history[-1].epoch + 1,
        "training_duration": self.training_history[-1].timestamp - self.start_time
    }

```

```

In [23]: def setup_enhanced_training_pipeline(
    generator: Model,
    discriminator: Model,
    gan: Model,
    weights_save_path: Optional[str] = None,
    config: Optional[TrainingConfig] = None
) -> EnhancedGANTrainer:
    if config is None:
        config = TrainingConfig()

    weights_manager = None
    if weights_save_path:
        from pathlib import Path
        weights_manager = ModelWeightsManager(weights_save_path)

    trainer = EnhancedGANTrainer(
        generator=generator,
        discriminator=discriminator,
        gan=gan,
        config=config,
        weights_manager=weights_manager
    )

    return trainer

```

1.9 Обучение

1.9.1 Подготовка данных

```
In [46]: dataset_info, label_mapping, distribution = load_image_dataset_with_info(
    data_path=os.path.join(path, 'cat-breeds/cat-breeds'),
    target_size=(128, 128),
    pattern="*.jpg",
    limit=20000,
    batch_size=1000,
    strategy=DatasetStrategy.HIERARCHICAL,
    return_labels=True
)

logger.info(f"Classes: {label_mapping.class_to_idx}")
logger.info(f"Distribution: {distribution}")
```

2025-06-01 14:02:56,616 - __main__ - INFO - Discovered 66 classes: ['havana_brown', 'american_bobtail', 'nebelung', 'turkish_van', 'abyssinian', 'kurilian_bobtail', 'peterbald', 'khao_manee', 'somali', 'american_curl', 'safari', 'toyger', 'singapura', 'devon_rex', 'munchkin', 'exotic_shorthair', 'bombay', 'savannah', 'thai', 'manx', 'burmese', 'oriental_shorthair', 'siberian', 'american_shorthair', 'norwegian_forest_cat', 'maine_coon', 'cymric', 'chausie', 'donskoy', 'egyptian_mau', 'scottish_fold', 'karelian_bobtail', 'cyprus', 'sphinx', 'vankedis', 'korean_bobtail', 'ragdoll', 'ocicat', 'ukrainian_levkoy', 'chartreux', 'balinese', 'serengeti', 'japanese_bobtail', 'american_wirehair', 'persian', 'german_rex', 'himalayan', 'bengal', 'tonkinese', 'mekong_bobtail', 'korat', 'cornish_rex', 'european_shorthair', 'turkish_angora', 'oregon_rex', 'russian_blue', 'laperm', 'birman', 'british_shorthair', 'ragamuffin', 'ural_rex', 'sokoke', 'selkirk_rex', 'lykoi', 'pixie_bob', 'siamese']

INFO: __main__:Discovered 66 classes: ['havana_brown', 'american_bobtail', 'nebelung', 'turkish_van', 'abyssinian', 'kurilian_bobtail', 'peterbald', 'khao_manee', 'somali', 'american_curl', 'safari', 'toyger', 'singapura', 'devon_rex', 'munchkin', 'exotic_shorthair', 'bombay', 'savannah', 'thai', 'manx', 'burmese', 'oriental_shorthair', 'siberian', 'american_shorthair', 'norwegian_forest_cat', 'maine_coon', 'cymric', 'chausie', 'donskoy', 'egyptian_mau', 'scottish_fold', 'karelian_bobtail', 'cyprus', 'sphinx', 'vankedis', 'korean_bobtail', 'ragdoll', 'ocicat', 'ukrainian_levkoy', 'chartreux', 'balinese', 'serengeti', 'japanese_bobtail', 'american_wirehair', 'persian', 'german_rex', 'himalayan', 'bengal', 'tonkinese', 'mekong_bobtail', 'korat', 'cornish_rex', 'european_shorthair', 'turkish_angora', 'oregon_rex', 'russian_blue', 'laperm', 'birman', 'british_shorthair', 'ragamuffin', 'ural_rex', 'sokoke', 'selkirk_rex', 'lykoi', 'pixie_bob', 'siamese']

2025-06-01 14:02:57,606 - __main__ - INFO - Found 11276 images across 66 classes

INFO: __main__:Found 11276 images across 66 classes

2025-06-01 14:02:57,607 - __main__ - INFO - Loading dataset with 11276 images

INFO: __main__:Loading dataset with 11276 images

2025-06-01 14:02:58,437 - __main__ - INFO - Processed batch 1/12

INFO: __main__:Processed batch 1/12

2025-06-01 14:02:59,245 - __main__ - INFO - Processed batch 2/12

INFO: __main__:Processed batch 2/12

2025-06-01 14:03:00,054 - __main__ - INFO - Processed batch 3/12

INFO: __main__:Processed batch 3/12

2025-06-01 14:03:00,868 - __main__ - INFO - Processed batch 4/12

INFO: __main__:Processed batch 4/12

2025-06-01 14:03:01,688 - __main__ - INFO - Processed batch 5/12

INFO: __main__:Processed batch 5/12

2025-06-01 14:03:02,500 - __main__ - INFO - Processed batch 6/12

INFO: __main__:Processed batch 6/12

2025-06-01 14:03:03,313 - __main__ - INFO - Processed batch 7/12

INFO: __main__:Processed batch 7/12

2025-06-01 14:03:04,128 - __main__ - INFO - Processed batch 8/12

INFO: __main__:Processed batch 8/12

2025-06-01 14:03:04,937 - __main__ - INFO - Processed batch 9/12

INFO: __main__:Processed batch 9/12

2025-06-01 14:03:05,763 - __main__ - INFO - Processed batch 10/12

INFO: __main__:Processed batch 10/12

2025-06-01 14:03:06,584 - __main__ - INFO - Processed batch 11/12

INFO: __main__:Processed batch 11/12

2025-06-01 14:03:06,808 - __main__ - INFO - Processed batch 12/12


```
INFO: __main__:Processed batch 12/12
2025-06-01 14:03:07,387 - __main__ - INFO - Dataset loaded successfully. Images shape: (11276, 128, 128, 3)
INFO: __main__:Dataset loaded successfully. Images shape: (11276, 128, 128, 3)
2025-06-01 14:03:07,388 - __main__ - INFO - Labels shape: (11276,)
INFO: __main__:Labels shape: (11276,)
2025-06-01 14:03:07,402 - __main__ - INFO - Classes: {'abyssinian': 0, 'american_bobtail': 1, 'american_curl': 2, 'american_shorthair': 3, 'american_wirehair': 4, 'balinese': 5, 'bengal': 6, 'birman': 7, 'bombay': 8, 'british_shorthair': 9, 'burmese': 10, 'chartreux': 11, 'chausie': 12, 'cornish_rex': 13, 'cymric': 14, 'cyprus': 15, 'devon_rex': 16, 'donskoy': 17, 'egyptian_mau': 18, 'european_shorthair': 19, 'exotic_shorthair': 20, 'german_rex': 21, 'havana_brown': 22, 'himalayan': 23, 'japanese_bobtail': 24, 'karelian_bobtail': 25, 'khao_manee': 26, 'korat': 27, 'korean_bobtail': 28, 'kurilian_bobtail': 29, 'laperm': 30, 'lykoi': 31, 'maine_coon': 32, 'manx': 33, 'mekong_bobtail': 34, 'munchkin': 35, 'nebelung': 36, 'norwegian_forest_cat': 37, 'ocicat': 38, 'oregon_rex': 39, 'oriental_shorthair': 40, 'persian': 41, 'peterbald': 42, 'pixie_bob': 43, 'ragamuffin': 44, 'ragdoll': 45, 'russian_blue': 46, 'safari': 47, 'savannah': 48, 'scottish_fold': 49, 'selkirk_rex': 50, 'serengeti': 51, 'siamese': 52, 'siberian': 53, 'singapura': 54, 'sokoke': 55, 'somali': 56, 'sphynx': 57, 'thai': 58, 'tonkinese': 59, 'toyger': 60, 'turkish_angora': 61, 'turkish_van': 62, 'ukrainian_levkoy': 63, 'ural_rex': 64, 'vankedisii': 65}
INFO: __main__:Classes: {'abyssinian': 0, 'american_bobtail': 1, 'american_curl': 2, 'american_shorthair': 3, 'american_wirehair': 4, 'balinese': 5, 'bengal': 6, 'birman': 7, 'bombay': 8, 'british_shorthair': 9, 'burmese': 10, 'chartreux': 11, 'chausie': 12, 'cornish_rex': 13, 'cymric': 14, 'cyprus': 15, 'devon_rex': 16, 'donskoy': 17, 'egyptian_mau': 18, 'european_shorthair': 19, 'exotic_shorthair': 20, 'german_rex': 21, 'havana_brown': 22, 'himalayan': 23, 'japanese_bobtail': 24, 'karelian_bobtail': 25, 'khao_manee': 26, 'korat': 27, 'korean_bobtail': 28, 'kurilian_bobtail': 29, 'laperm': 30, 'lykoi': 31, 'maine_coon': 32, 'manx': 33, 'mekong_bobtail': 34, 'munchkin': 35, 'nebelung': 36, 'norwegian_forest_cat': 37, 'ocicat': 38, 'oregon_rex': 39, 'oriental_shorthair': 40, 'persian': 41, 'peterbald': 42, 'pixie_bob': 43, 'ragamuffin': 44, 'ragdoll': 45, 'russian_blue': 46, 'safari': 47, 'savannah': 48, 'scottish_fold': 49, 'selkirk_rex': 50, 'serengeti': 51, 'siamese': 52, 'siberian': 53, 'singapura': 54, 'sokoke': 55, 'somali': 56, 'sphynx': 57, 'thai': 58, 'tonkinese': 59, 'toyger': 60, 'turkish_angora': 61, 'turkish_van': 62, 'ukrainian_levkoy': 63, 'ural_rex': 64, 'vankedisii': 65}
2025-06-01 14:03:07,403 - __main__ - INFO - Distribution: {'havana_brown': 158, 'american_bobtail': 140, 'nebelung': 194, 'turkish_van': 200, 'abyssinian': 200, 'kurilian_bobtail': 120, 'peterbald': 196, 'khao_manee': 198, 'somali': 139, 'american_curl': 137, 'safari': 149, 'toyger': 160, 'singapura': 135, 'devon_rex': 138, 'munchkin': 191, 'exotic_shorthair': 156, 'bombay': 154, 'savannah': 197, 'thai': 198, 'manx': 192, 'burmese': 197, 'oriental_shorthair': 197, 'siberian': 159, 'american_shorthair': 199, 'norwegian_forest_cat': 97, 'maine_coon': 133, 'cymric': 198, 'chausie': 200, 'donskoy': 200, 'egyptian_mau': 184, 'scottish_fold': 184, 'karelian_bobtail': 196, 'cyprus': 200, 'sphynx': 120, 'vankedisii': 187, 'korean_bobtail': 139, 'ragdoll': 199, 'ocicat': 182, 'ukrainian_levkoy': 134, 'chartreux': 114, 'balinese': 195, 'serengeti': 174, 'japanese_bobtail': 194, 'american_wirehair': 196, 'persian': 192, 'german_rex': 199, 'himalayan': 192, 'bengal': 189, 'tonkinese': 195, 'mekong_bobtail': 140, 'korat': 152, 'cornish_rex': 197, 'european_shorthair': 200, 'turkish_angora': 188, 'oregon_rex': 120, 'russian_blue': 109, 'laperm': 120, 'birman': 131, 'british_shorthair': 174, 'ragamuffin': 18
```

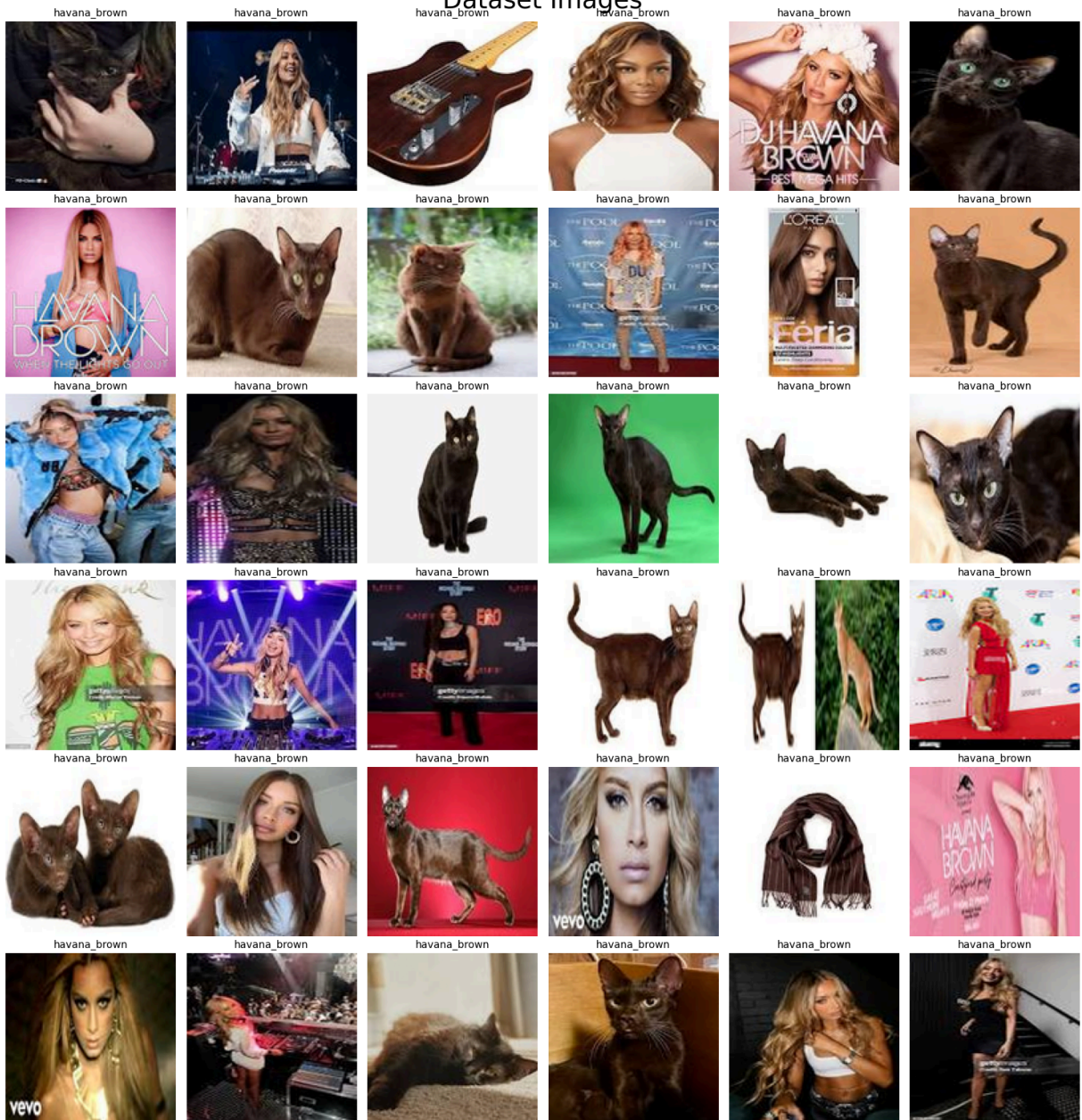
```
2, 'ural_rex': 137, 'sokoke': 199, 'selkirk_rex': 192, 'lykoi': 200, 'pixie_bob': 200, 'siamese': 138}
INFO: __main__:Distribution: {'havana_brown': 158, 'american_bobtail': 140, 'nebelung': 194, 'turkish_van': 200, 'abyssinian': 200, 'kurilian_bobtail': 120, 'peterbald': 196, 'khao_manee': 198, 'somali': 139, 'american_curl': 137, 'safari': 149, 'toyger': 160, 'singapura': 135, 'devon_rex': 138, 'munchkin': 191, 'exotic_shorthair': 156, 'bombay': 154, 'savannah': 197, 'thai': 198, 'manx': 192, 'burmese': 197, 'oriental_shorthair': 197, 'siberian': 159, 'american_shorthair': 199, 'norwegian_forest_cat': 97, 'maine_coon': 133, 'cymric': 198, 'chausie': 200, 'donskoy': 200, 'egyptian_mau': 184, 'scottish_fold': 184, 'karelian_bobtail': 196, 'cyprus': 200, 'sphinx': 120, 'vankedis': 187, 'korean_bobtail': 139, 'ragdoll': 199, 'ocicat': 182, 'ukrainian_levkoy': 134, 'chartreux': 114, 'balinese': 195, 'serengeti': 174, 'japanese_bobtail': 194, 'american_wirehair': 196, 'persian': 192, 'german_rex': 199, 'himalayan': 192, 'bengal': 189, 'tonkinese': 195, 'mekong_bobtail': 140, 'korat': 152, 'cornish_rex': 197, 'european_shorthair': 200, 'turkish_angora': 188, 'oregon_rex': 120, 'russian_blue': 109, 'laperm': 120, 'birman': 131, 'british_shorthair': 174, 'ragamuffin': 182, 'ural_rex': 137, 'sokoke': 199, 'selkirk_rex': 192, 'lykoi': 200, 'pixie_bob': 200, 'siamese': 138}
```

```
In [47]: images, labels = dataset_info

visualizer = DatasetVisualizer()
visualizer.plot_images(
    images,
    labels,
    label_mapping.idx_to_class if label_mapping else None
)
```

```
2025-06-01 14:03:07,408 - __main__ - INFO - Dataset shape: (11276, 128, 128, 3), Plotting 36/11276 images
INFO: __main__:Dataset shape: (11276, 128, 128, 3), Plotting 36/11276 images
```

Dataset Images



2025-06-01 14:03:09,245 - __main__ - INFO - Visualization completed successfully
 INFO: __main__:Visualization completed successfully

1.9.2 Ванильный генератор

```
In [53]: vanilla_model_config = ModelConfig(
    image_shape=(64, 64, 3),
    latent_dim=100,
    learning_rate=0.00025,
    beta_1=0.5
)

vanilla_arch_config = ArchitectureConfig(
    base_filters=64,
    dropout_rate=0.3,
    dense_units=128
```

```
)

vanilla_gan_builder = GANBuilder(vanilla_model_config, vanilla_arch_config)
g_vanilla, d_vanilla, gan_vanilla = vanilla_gan_builder.build_all()

print("\nVanilla Generator Summary:")
g_vanilla.summary()
print("\nVanilla Discriminator Summary:")
d_vanilla.summary()
print("\nCombined Vanilla GAN Summary:")
gan_vanilla.summary()
```

Vanilla Generator Summary:
Model: "vanilla_generator"

Layer (type)	Output Shape	Par
dense_6 (Dense)	(None, 4096)	409
batch_normalization_62 (BatchNormalization)	(None, 4096)	16
leaky_re_lu_70 (LeakyReLU)	(None, 4096)	
reshape_2 (Reshape)	(None, 2, 2, 1024)	
conv2d_transpose_32 (Conv2DTranspose)	(None, 4, 4, 512)	8,388
batch_normalization_63 (BatchNormalization)	(None, 4, 4, 512)	2
leaky_re_lu_71 (LeakyReLU)	(None, 4, 4, 512)	
conv2d_transpose_33 (Conv2DTranspose)	(None, 8, 8, 256)	2,097
batch_normalization_64 (BatchNormalization)	(None, 8, 8, 256)	1
leaky_re_lu_72 (LeakyReLU)	(None, 8, 8, 256)	
conv2d_transpose_34 (Conv2DTranspose)	(None, 16, 16, 128)	524
batch_normalization_65 (BatchNormalization)	(None, 16, 16, 128)	
leaky_re_lu_73 (LeakyReLU)	(None, 16, 16, 128)	
conv2d_transpose_35 (Conv2DTranspose)	(None, 32, 32, 64)	131
batch_normalization_66 (BatchNormalization)	(None, 32, 32, 64)	
leaky_re_lu_74 (LeakyReLU)	(None, 32, 32, 64)	
conv2d_transpose_36 (Conv2DTranspose)	(None, 64, 64, 32)	32
batch_normalization_67 (BatchNormalization)	(None, 64, 64, 32)	
leaky_re_lu_75 (LeakyReLU)	(None, 64, 64, 32)	
conv2d_transpose_37 (Conv2DTranspose)	(None, 64, 64, 3)	1

Total params: 11,605,379 (44.27 MB)

Trainable params: 11,595,203 (44.23 MB)

Non-trainable params: 10,176 (39.75 KB)

Vanilla Discriminator Summary:
Model: "vanilla_discriminator"

Layer (type)	Output Shape	Par
conv2d_36 (Conv2D)	(None, 32, 32, 64)	3
leaky_re_lu_76 (LeakyReLU)	(None, 32, 32, 64)	
dropout_38 (Dropout)	(None, 32, 32, 64)	
conv2d_37 (Conv2D)	(None, 16, 16, 128)	131
batch_normalization_68 (BatchNormalization)	(None, 16, 16, 128)	
leaky_re_lu_77 (LeakyReLU)	(None, 16, 16, 128)	
dropout_39 (Dropout)	(None, 16, 16, 128)	
conv2d_38 (Conv2D)	(None, 8, 8, 256)	524
batch_normalization_69 (BatchNormalization)	(None, 8, 8, 256)	1
leaky_re_lu_78 (LeakyReLU)	(None, 8, 8, 256)	
dropout_40 (Dropout)	(None, 8, 8, 256)	
conv2d_39 (Conv2D)	(None, 4, 4, 512)	2,097
batch_normalization_70 (BatchNormalization)	(None, 4, 4, 512)	2
leaky_re_lu_79 (LeakyReLU)	(None, 4, 4, 512)	
dropout_41 (Dropout)	(None, 4, 4, 512)	
conv2d_40 (Conv2D)	(None, 2, 2, 1024)	8,388
batch_normalization_71 (BatchNormalization)	(None, 2, 2, 1024)	4
leaky_re_lu_80 (LeakyReLU)	(None, 2, 2, 1024)	
dropout_42 (Dropout)	(None, 2, 2, 1024)	
conv2d_41 (Conv2D)	(None, 1, 1, 2048)	33,554
batch_normalization_72 (BatchNormalization)	(None, 1, 1, 2048)	8
leaky_re_lu_81 (LeakyReLU)	(None, 1, 1, 2048)	
dropout_43 (Dropout)	(None, 1, 1, 2048)	
flatten_2 (Flatten)	(None, 2048)	
dense_7 (Dense)	(None, 128)	262
leaky_re_lu_82 (LeakyReLU)	(None, 128)	

dropout_44 (Dropout)	(None , 128)	
dense_8 (Dense)	(None , 1)	

Total params: 44,976,961 (171.57 MB)

Trainable params: 44,969,025 (171.54 MB)

Non-trainable params: 7,936 (31.00 KB)

Combined Vanilla GAN Summary:

Model: "combined_vanilla_gan"

Layer (type)	Output Shape	Par
gan_noise_input (InputLayer)	(None , 100)	
vanilla_generator (Sequential)	(None , 64, 64, 3)	11,605
vanilla_discriminator (Sequential)	(None , 1)	44,976

Total params: 56,582,340 (215.84 MB)

Trainable params: 56,564,228 (215.78 MB)

Non-trainable params: 18,112 (70.75 KB)

```
In [54]: sample_generator = SampleGeneratorService(is_conditional=False)
random_labels = sample_generator.generate_fake_samples(generator=g_vanilla,
gan_visualizator = GANVisualizationService(is_conditional=False)
gan_visualizator.plot_generated_images(g_vanilla, epoch=0)
```

Generated Images at Epoch 1



```
In [55]: config = TrainingConfig(
    num_epochs=10000,
    batch_size=1024,
    display_frequency=10,
    verbose=True,
    real_time_visualization=False,
    log_frequency=1,
    save_frequency=999,
    is_conditional=False,
    # weights_checkpoint_dir="/home/student/projects/CVTools/HomeWork/HW4/mc
)

trainer = setup_enhanced_training_pipeline(
    generator=g_vanilla,
    discriminator=d_vanilla,
    gan=gan_vanilla,
    weights_save_path="./model_weights",
    config=config
```



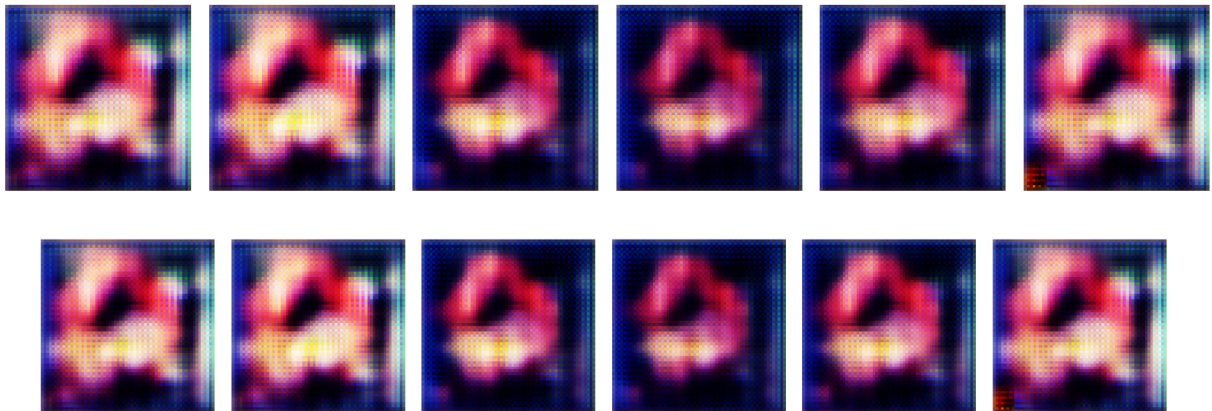
```
)

saved_images = trainer.train(images, labels)
summary = trainer.get_training_summary()
print(f"\n Training Summary: {summary}")
```

KeyboardInterrupt

```
In [35]: sample_generator = SampleGeneratorService(is_conditional=False)
random_labels = sample_generator.generate_fake_samples(generator=g_vanilla,
gan_visualizator = GANVisualizationService(is_conditional=False)
gan_visualizator.plot_generated_images(g_vanilla, epoch=0)
```

Generated Images at Epoch 1



1.9.3 Условный генератор

```
In [60]: condition_model_config = ModelConfig(
    image_shape=(128, 128, 3),
    latent_dim=100,
    learning_rate=0.0001,
    beta_1=0.5,
    num_classes=66
)

condition_arch_config = ArchitectureConfig(
    base_filters=128,
    dropout_rate=0.01,
    dense_units=64
)

condition_gan_builder = GANBuilder(condition_model_config, condition_arch_co
g_condition, d_condition, gan_condition = condition_gan_builder.build_all()

print("\nVanilla Generator Summary:")
g_condition.summary()
print("\nVanilla Discriminator Summary:")
d_condition.summary()
print("\nCombined Vanilla GAN Summary:")
gan_condition.summary()
```

Vanilla Generator Summary:
Model: "conditional_generator"

Layer (type)	Output Shape	Param #	Connected to
label_input (InputLayer)	(None, 1)	0	-
label_embedding (Embedding)	(None, 1, 50)	3,300	label_input[0]
noise_input (InputLayer)	(None, 100)	0	-
flatten_embedding (Flatten)	(None, 50)	0	label_embeddi
concatenate_inputs (Concatenate)	(None, 150)	0	noise_input[0] flatten_embed
dense_projection (Dense)	(None, 32768)	4,915,200	concatenate_i
bn_projection (BatchNormalizatio...	(None, 32768)	131,072	dense_project
leaky_relu_project... (LeakyReLU)	(None, 32768)	0	bn_projection
reshape_to_image_d... (Reshape)	(None, 4, 4, 2048)	0	leaky_relu_pr
conv2d_transpose_43 (Conv2DTranspose)	(None, 8, 8, 1024)	33,554,432	reshape_to_im
batch_normalizatio... (BatchNormalizatio...	(None, 8, 8, 1024)	4,096	conv2d_transp
leaky_re_lu_94 (LeakyReLU)	(None, 8, 8, 1024)	0	batch_normali
conv2d_transpose_44 (Conv2DTranspose)	(None, 16, 16, 512)	8,388,608	leaky_re_lu_9
batch_normalizatio... (BatchNormalizatio...	(None, 16, 16, 512)	2,048	conv2d_transp
leaky_re_lu_95 (LeakyReLU)	(None, 16, 16, 512)	0	batch_normali
conv2d_transpose_45 (Conv2DTranspose)	(None, 32, 32, 256)	2,097,152	leaky_re_lu_9
batch_normalizatio... (BatchNormalizatio...	(None, 32, 32, 256)	1,024	conv2d_transp
leaky_re_lu_96 (LeakyReLU)	(None, 32, 32, 256)	0	batch_normali
conv2d_transpose_46 (Conv2DTranspose)	(None, 64, 64, 128)	524,288	leaky_re_lu_9

batch_normalizatio... (BatchNormalizatio...	(None, 64, 64, 128)	512	conv2d_transp
leaky_re_lu_97 (LeakyReLU)	(None, 64, 64, 128)	0	batch_normali
conv2d_transpose_47 (Conv2DTranspose)	(None, 128, 128, 64)	131,072	leaky_re_lu_9
batch_normalizatio... (BatchNormalizatio...	(None, 128, 128, 64)	256	conv2d_transp
leaky_re_lu_98 (LeakyReLU)	(None, 128, 128, 64)	0	batch_normali
output_image (Conv2DTranspose)	(None, 128, 128, 3)	3,075	leaky_re_lu_9

Total params: 49,756,135 (189.80 MB)

Trainable params: 49,686,631 (189.54 MB)

Non-trainable params: 69,504 (271.50 KB)

Vanilla Discriminator Summary:

Model: "conditional_discriminator"

Layer (type)	Output Shape	Param #	Connected to
label_input (InputLayer)	(None, 1)	0	-
label_embedding (Embedding)	(None, 1, 16384)	1,081,344	label_input[0]
image_input (InputLayer)	(None, 128, 128, 3)	0	-
reshape_embedding_... (Reshape)	(None, 128, 128, 1)	0	label_embeddi
concatenate_image_... (Concatenate)	(None, 128, 128, 4)	0	image_input[0] reshape_embed
conv2d_48 (Conv2D)	(None, 64, 64, 128)	8,320	concatenate_i
leaky_re_lu_99 (LeakyReLU)	(None, 64, 64, 128)	0	conv2d_48[0][
dropout_51 (Dropout)	(None, 64, 64, 128)	0	leaky_re_lu_9
conv2d_49 (Conv2D)	(None, 32, 32, 256)	524,288	dropout_51[0]
batch_normalizatio... (BatchNormalizatio...	(None, 32, 32, 256)	1,024	conv2d_49[0][
leaky_re_lu_100 (LeakyReLU)	(None, 32, 32, 256)	0	batch_normali
dropout_52 (Dropout)	(None, 32, 32, 256)	0	leaky_re_lu_1
conv2d_50 (Conv2D)	(None, 16, 16, 512)	2,097,152	dropout_52[0]
batch_normalizatio... (BatchNormalizatio...	(None, 16, 16, 512)	2,048	conv2d_50[0][
leaky_re_lu_101 (LeakyReLU)	(None, 16, 16, 512)	0	batch_normali
dropout_53 (Dropout)	(None, 16, 16, 512)	0	leaky_re_lu_1
conv2d_51 (Conv2D)	(None, 8, 8, 1024)	8,388,608	dropout_53[0]
batch_normalizatio... (BatchNormalizatio...	(None, 8, 8, 1024)	4,096	conv2d_51[0][
leaky_re_lu_102 (LeakyReLU)	(None, 8, 8, 1024)	0	batch_normali

dropout_54 (Dropout)	(None, 8, 8, 1024)	0	leaky_re_lu_1
conv2d_52 (Conv2D)	(None, 4, 4, 2048)	33,554,432	dropout_54[0]
batch_normalizatio... (BatchNormalizatio...)	(None, 4, 4, 2048)	8,192	conv2d_52[0][
leaky_re_lu_103 (LeakyReLU)	(None, 4, 4, 2048)	0	batch_normali
dropout_55 (Dropout)	(None, 4, 4, 2048)	0	leaky_re_lu_1
conv2d_53 (Conv2D)	(None, 2, 2, 4096)	134,217,7...	dropout_55[0]
batch_normalizatio... (BatchNormalizatio...)	(None, 2, 2, 4096)	16,384	conv2d_53[0][
leaky_re_lu_104 (LeakyReLU)	(None, 2, 2, 4096)	0	batch_normali
dropout_56 (Dropout)	(None, 2, 2, 4096)	0	leaky_re_lu_1
flatten_features (Flatten)	(None, 16384)	0	dropout_56[0]
dense_intermediate (Dense)	(None, 64)	1,048,640	flatten_featu
leaky_relu_interme... (LeakyReLU)	(None, 64)	0	dense_interme
dropout_intermedia... (Dropout)	(None, 64)	0	leaky_relu_in
output_validity (Dense)	(None, 1)	65	dropout_inter

Total params: 180,952,321 (690.28 MB)

Trainable params: 180,936,449 (690.22 MB)

Non-trainable params: 15,872 (62.00 KB)

Combined Vanilla GAN Summary:

Model: "combined_conditional_gan"

Layer (type)	Output Shape	Param #	Connected to
gan_noise_input (InputLayer)	(None, 100)	0	-
gan_label_input (InputLayer)	(None, 1)	0	-
conditional_genera... (Functional)	(None, 128, 128, 3)	49,756,135	gan_noise_inp gan_label_inp
conditional_discrim... (Functional)	(None, 1)	180,952,3...	conditional_g gan_label_inp

Total params: 230,708,456 (880.08 MB)

Trainable params: 230,623,080 (879.76 MB)

Non-trainable params: 85,376 (333.50 KB)

```
In [61]: sample_generator = SampleGeneratorService(is_conditional=True, num_classes=66,
random_labels = sample_generator.generate_random_labels(6)
gan_visualizator = GANVisualizationService(is_conditional=True, num_classes=66)
gan_visualizator.plot_generated_images(g_condition, epoch=0, class_labels=random_labels)
```

Generated Images at Epoch 1

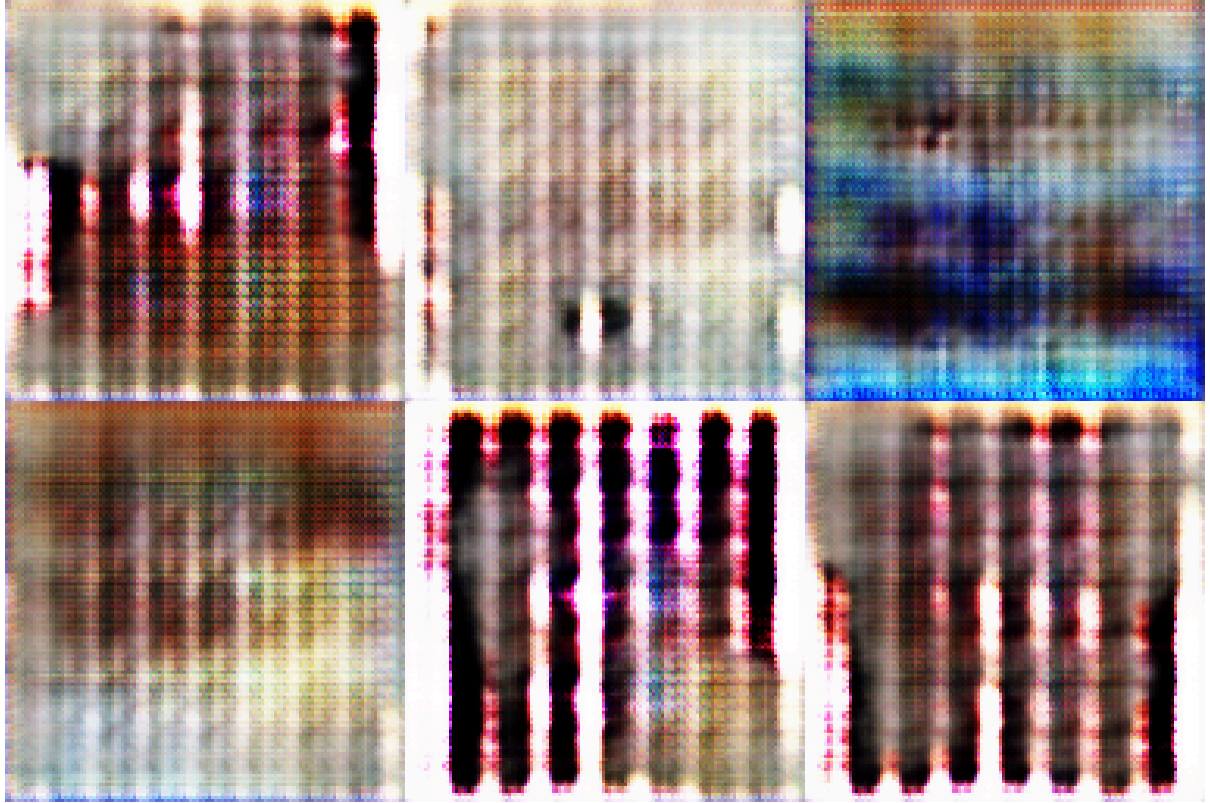


```
In [67]: config = TrainingConfig(
    num_epochs=10000,
    batch_size=16,
    display_frequency=100,
    verbose=False,
    real_time_visualization=True,
    is_conditional=True,
    num_classes=66,
    log_frequency=100,
    save_frequency=999,
    # weights_checkpoint_dir="/home/student/projects/CVTools/HomeWork/HW4/model_weights"
)

trainer = setup_enhanced_training_pipeline(
    generator=g_condition,
    discriminator=d_condition,
    gan=gan_condition,
    weights_save_path="./model_weights",
    config=config
)

saved_images = trainer.train(images, labels)
```

```
summary = trainer.get_training_summary()  
print(f"\n Training Summary: {summary}")
```



KeyboardInterrupt

Traceback (most recent call last)

Cell In[67], line 22

```
1 config = TrainingConfig(
2     num_epochs=10000,
3     batch_size=16,
4     (...) 11 weights_checkpoint_dir="/home/student/projects/CVTools/H
omeWork/HW4/model_weights/model_weights_01_06_2025_13_30_57_epoch_999"
5 )
6
7
8
9
10
11
12 )
13
14 trainer = setup_enhanced_training_pipeline(
15     generator=g_condition,
16     discriminator=d_condition,
17     (...) 19 config=config
18 )
19
20
21
22 ---> 22 saved_images = trainer.train(images, labels)
23 summary = trainer.get_training_summary()
24 print(f"\n Training Summary: {summary}")
```

Cell In[22], line 178, in EnhancedGANTrainer.train(self, dataset, labels)

```
175 epoch_metrics = []
176
177 for batch_num in range(batches_per_epoch):
178 ---> 178     disc_loss, disc_acc = self._train_discriminator_batch(
179         dataset, half_batch_size, labels
180     )
181     gen_loss = self._train_generator_batch(self.config.batch_size)
182     metrics = TrainingMetrics(
183         discriminator_loss=disc_loss,
184         discriminator_accuracy=disc_acc,
185         (...) 189 timestamp=time.time()
186     )
187
188
189
190 )
```

Cell In[22], line 92, in EnhancedGANTrainer._train_discriminator_batch(self, dataset, half_batch_size, labels)

```
87 real_data = self.sample_generator.generate_real_samples(
88     dataset, half_batch_size, labels
89 )
90 real_images, real_y_labels, real_class_labels = real_data
91
92 ---> 92 fake_data = self.sample_generator.generate_fake_samples(
93     self.generator, self.config.noise_dimension, half_batch_size
94 )
95 fake_images, fake_y_labels, fake_class_labels = fake_data
96
97 real_inputs, real_targets = self._prepare_conditional_data(
98     real_images, real_class_labels, real_y_labels
99 )
```

Cell In[17], line 46, in SampleGeneratorService.generate_fake_samples(self, generator, noise_dim, num_samples)

```
44 if self.is_conditional:
45     fake_class_labels = self.generate_random_labels(num_samples)
46 ---> 46     fake_images = generator.predict([noise, fake_class_labels], verb
ose=0)
47     fake_images = fake_images.astype(np.float32)
48     return fake_images, fake_y_labels, fake_class_labels
```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/utils/t
raceback_utils.py:117, in filter_traceback.<locals>.error_handler(*args, **k

```

wargs)
    115 filtered_tb = None
    116 try:
--> 117     return fn(*args, **kwargs)
    118 except Exception as e:
    119     filtered_tb = _process_traceback_frames(e.__traceback__)

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/backends/tensorflow/trainer.py:505, in TensorFlowTrainer.predict(self, x, batch_size, verbose, steps, callbacks)
    500 @traceback_utils.filter_traceback
    501 def predict(
    502     self, x, batch_size=None, verbose="auto", steps=None, callbacks=
None
    503 ):
    504     # Create an iterator that yields batches of input data.
--> 505     epoch_iterator = TFEPOCHIterator(
    506         x=x,
    507         batch_size=batch_size,
    508         steps_per_epoch=steps,
    509         shuffle=False,
    510         distribute_strategy=self.distribute_strategy,
    511         steps_per_execution=self.steps_per_execution,
    512     )
    514     # Container that configures and calls callbacks.
    515     if not isinstance(callbacks, callbacks_module.CallbackList):

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/backends/tensorflow/trainer.py:726, in TFEPOCHIterator.__init__(self, distribute_strategy, *args, **kwargs)
    724 super().__init__(*args, **kwargs)
    725 self.distribute_strategy = distribute_strategy
--> 726 dataset = self.data_adapter.get_tf_dataset()
    727 if not isinstance(dataset, tf.distribute.DistributedDataset):
    728     dataset = self._distribute_strategy.experimental_distribute_data
set(
    729         dataset
    730     )

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/trainers/data_adapters/array_data_adapter.py:140, in ArrayDataAdapter.get_tf_dataset(self)
    133     return indices
    135 # We prefetch a single element. Computing large permutations can take
e
    136 # quite a while so we don't want to wait for prefetching over an epoch
ch
    137 # boundary to trigger the next permutation. On the other hand, too many
any
    138 # simultaneous shuffles can contend on a hardware level and degrade all
all
    139 # performance.
--> 140 indices_dataset = indices_dataset.map(permutation).prefetch(1)
    142 def slice_batch_indices(indices):
    143     """Convert a Tensor of indices into a dataset of batched indices.
S.

```

```

144
145     This step can be accomplished in several ways. The most natural
is
(...) 157         A Dataset of batched indices.
158     """

```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/tensorflow/python/data/ops/dataset_ops.py:2341, in DatasetV2.map(self, map_func, num_parallel_calls, deterministic, synchronous, use_unbounded_threadpool, name)

```

2336 # Loaded lazily due to a circular dependency (dataset_ops -> map_op
->
2337 # dataset_ops).
2338 # pylint: disable=g-import-not-at-top,protected-access
2339 from tensorflow.python.data.ops import map_op
-> 2341 return map_op._map_v2(
2342     self,
2343     map_func,
2344     num_parallel_calls=num_parallel_calls,
2345     deterministic=deterministic,
2346     synchronous=synchronous,
2347     use_unbounded_threadpool=use_unbounded_threadpool,
2348     name=name,
2349 )

```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/tensorflow/python/data/ops/map_op.py:43, in _map_v2(input_dataset, map_func, num_parallel_calls, deterministic, synchronous, use_unbounded_threadpool, name)

```

38 if deterministic is not None and not debug_mode.DEBUG_MODE:
39     warnings.warn(
40         "The `deterministic` argument has no effect unless the "
41         "`num_parallel_calls` argument is specified."
42     )
--> 43 return MapDataset(
44     input_dataset,
45     map_func,
46     preserve_cardinality=True,
47     force_synchronous=False if synchronous is None else synchronous
s,
48     name=name,
49 )
50 else:
51     if synchronous:

```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/tensorflow/python/data/ops/map_op.py:164, in MapDataset.__init__(self, input_dataset, map_func, force_synchronous, use_inter_op_parallelism, preserve_cardinality, use_legacy_function, name)

```

162 self._force_synchronous = force_synchronous
163 self.name = name
--> 164 variant_tensor = gen_dataset_ops.map_dataset(
165     input_dataset._variant_tensor, # pylint: disable=protected-acce
ss
166     self._map_func.function.captured_inputs,
167     f=self._map_func.function,
168     use_inter_op_parallelism=self._use_inter_op_parallelism,
169     preserve_cardinality=self._preserve_cardinality,

```

```

170     force_synchronous=self._force_synchronous,
171     **self._common_args
172 )
173 super().__init__(input_dataset, variant_tensor)

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/tensorflow/python/ops/gen_dataset_ops.py:3530, in map_dataset(input_dataset, other_arguments, f, output_types, output_shapes, use_inter_op_parallelism, preserve_cardinality, force_synchronous, metadata, name)
    3528 if tld.is_eager:
    3529     try:
-> 3530         _result = pywrap_tfe.TFE_Py_FastPathExecute(
    3531             _ctx,                , name, input_dataset, other_arguments,    ,
    f,
    3532             , output_types,                , output_shapes,
    3533             , use_inter_op_parallelism,    ,
    3534             , preserve_cardinality,
    ,
    3535             force_synchronous,                , metadata)
    3536     return _result
    3537 except _core._NotOkStatusException as e:
KeyboardInterrupt:

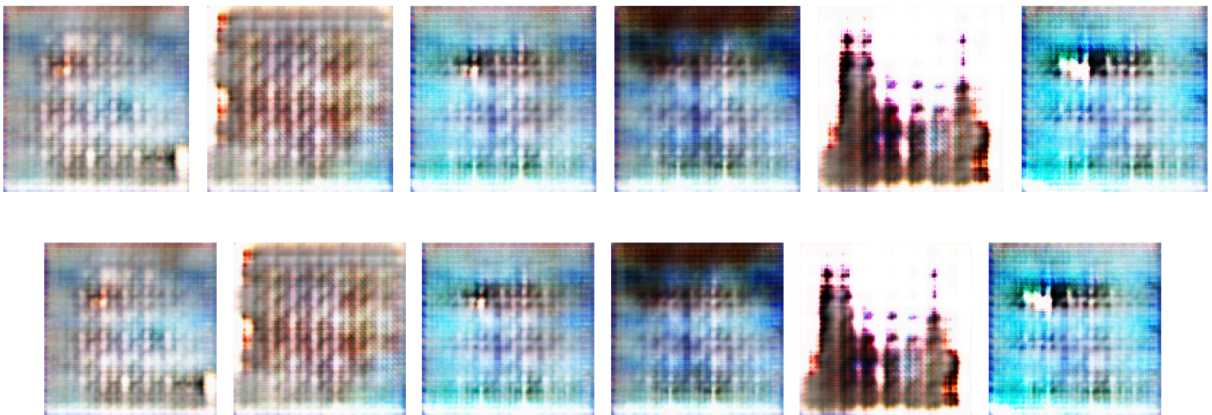
```

```

In [89]: sample_generator = SampleGeneratorService(is_conditional=True, num_classes=6)
random_labels = sample_generator.generate_random_labels(6)
gan_visualizator = GANVisualizationService(is_conditional=True, num_classes=6)
gan_visualizator.plot_generated_images(g_condition, epoch=0, class_labels=random_labels)

```

Generated Images at Epoch 1



1.10 VAE + cGAN

1.11 Строим модель

```

In [6]: @dataclass
class VAECGANConfig:
    image_shape: Tuple[int, int, int] = (256, 256, 3)
    latent_dim: int = 512
    noise_dim: int = 100

```

```

num_classes: int = 10
learning_rate_vae: float = 0.0001
learning_rate_gan: float = 0.0002
beta_1: float = 0.5
beta_2: float = 0.999
vae_beta: float = 1.0
reconstruction_weight: float = 10.0
adversarial_weight: float = 1.0
pretrained_vae_path: Optional[str] = None
freeze_vae_encoder: bool = False
freeze_vae_decoder: bool = False

```

```

In [7]: @dataclass
class ArchitectureConfig:
    base_filters: int = 64
    dropout_rate: float = 0.3
    leaky_relu_alpha: float = 0.2
    embedding_dim: int = 128
    dense_units: int = 512
    use_spectral_norm: bool = True
    use_self_attention: bool = True

```

```

In [8]: class SelfAttentionLayer(Layer):
def __init__(self, **kwargs):
    super().__init__(**kwargs)

def build(self, input_shape):
    channels = input_shape[-1]
    self.channels = channels
    self.query_conv = Conv2D(channels // 8, 1, name='query')
    self.key_conv = Conv2D(channels // 8, 1, name='key')
    self.value_conv = Conv2D(channels, 1, name='value')
    self.gamma = self.add_weight(
        name='gamma',
        shape=(),
        initializer='zeros',
        trainable=True
    )
    super().build(input_shape)

def call(self, inputs):
    batch_size = tf.shape(inputs)[0]
    height = tf.shape(inputs)[1]
    width = tf.shape(inputs)[2]

    query = self.query_conv(inputs)
    key = self.key_conv(inputs)
    value = self.value_conv(inputs)

    query = tf.reshape(query, [batch_size, height * width, self.channels])
    key = tf.reshape(key, [batch_size, height * width, self.channels // 8])
    value = tf.reshape(value, [batch_size, height * width, self.channels])

    attention = tf.nn.softmax(tf.matmul(query, key, transpose_b=True))
    attended = tf.matmul(attention, value)

```

```

        attended = tf.reshape(attended, [batch_size, height, width, self.channels])

        output = inputs + self.gamma * attended
        return output

```

```

In [9]: class SamplingLayer(Layer):
        def call(self, inputs):
            z_mean, z_log_var = inputs
            batch = tf.shape(z_mean)[0]
            dim = tf.shape(z_mean)[1]
            epsilon = tf.random.normal(shape=(batch, dim))
            return z_mean + tf.exp(0.5 * z_log_var) * epsilon

```

```

In [10]: class BaseModelBuilder(ABC):
        def __init__(self, config: VAEGGANConfig, arch_config: ArchitectureConfig):
            self.config = config
            self.arch_config = arch_config

        @abstractmethod
        def build(self) -> Model:
            pass

        def _create_conv_layer(self, filters: int, kernel_size: int = 4,
                               strides: int = 2, **kwargs) -> Conv2D:
            return Conv2D(filters, kernel_size, strides=strides, **kwargs)

        def _add_self_attention(self, x: tf.Tensor) -> tf.Tensor:
            if self.arch_config.use_self_attention:
                return SelfAttentionLayer()(x)
            return x

```

```

In [11]: class VAEEncoder(BaseModelBuilder):
        def build(self) -> Model:
            inputs = Input(shape=self.config.image_shape, name="encoder_input")

            x = inputs
            current_filters = self.arch_config.base_filters

            for i in range(5):
                x = self._create_conv_layer(
                    current_filters, padding='same', use_bias=False
                )(x)
                x = BatchNormalization()(x)
                x = LeakyReLU(self.arch_config.leaky_relu_alpha)(x)

                if i == 2:
                    x = self._add_self_attention(x)

                current_filters = min(current_filters * 2, 512)

            x = GlobalAveragePooling2D()(x)
            x = Dense(self.arch_config.dense_units)(x)
            x = LeakyReLU(self.arch_config.leaky_relu_alpha)(x)
            x = Dropout(self.arch_config.dropout_rate)(x)

```

```

z_mean = Dense(self.config.latent_dim, name="z_mean")(x)
z_log_var = Dense(self.config.latent_dim, name="z_log_var")(x)

model = Model(inputs, [z_mean, z_log_var], name="vae_encoder")
return model

```

```

In [12]: class VAEDecoder(BaseModelBuilder):
def build(self) -> Model:
    latent_input = Input(shape=(self.config.latent_dim,), name="latent_i

    initial_size = self.config.image_shape[0] // (2 ** 5)
    initial_filters = 512

    x = Dense(initial_size * initial_size * initial_filters)(latent_input)
    x = BatchNormalization()(x)
    x = LeakyReLU(self.arch_config.leaky_relu_alpha)(x)
    x = Reshape((initial_size, initial_size, initial_filters))(x)

    current_filters = initial_filters
    for i in range(5):
        current_filters //= 2

        x = Conv2DTranspose(
            current_filters, 4, strides=2, padding='same', use_bias=False
        )(x)
        x = BatchNormalization()(x)
        x = LeakyReLU(self.arch_config.leaky_relu_alpha)(x)

        if i == 2:
            x = self._add_self_attention(x)

    outputs = Conv2DTranspose(
        self.config.image_shape[2], 4, strides=1,
        padding='same', activation='tanh', name="decoded_image"
    )(x)

    model = Model(latent_input, outputs, name="vae_decoder")
    return model

```

```

In [13]: class ConditionalGenerator(BaseModelBuilder):
def build(self) -> Model:
    noise_input = Input(shape=(self.config.noise_dim,), name="noise_input")
    label_input = Input(shape=(1,), name="label_input", dtype='int32')

    label_embedding = Embedding(
        self.config.num_classes,
        self.arch_config.embedding_dim
    )(label_input)
    label_embedding = Flatten()(label_embedding)

    combined = Concatenate()([noise_input, label_embedding])

    x = Dense(self.arch_config.dense_units)(combined)
    x = BatchNormalization()(x)
    x = LeakyReLU(self.arch_config.leaky_relu_alpha)(x)

```



```

x = Dropout(self.arch_config.dropout_rate)(x)

x = Dense(self.arch_config.dense_units)(x)
x = BatchNormalization()(x)
x = LeakyReLU(self.arch_config.leaky_relu_alpha)(x)
x = Dropout(self.arch_config.dropout_rate)(x)

latent_output = Dense(self.config.latent_dim, activation='linear',
                      name="generated_latent")(x)

model = Model([noise_input, label_input], latent_output,
              name="conditional_generator")
return model

```

```

In [14]: class ConditionalDiscriminator(BaseModelBuilder):
def build(self) -> Model:
    image_input = Input(shape=self.config.image_shape, name="image_input")
    label_input = Input(shape=(1,), name="label_input", dtype='int32')

    label_embedding = Embedding(
        self.config.num_classes,
        self.config.image_shape[0] * self.config.image_shape[1]
    )(label_input)
    label_embedding = Reshape(
        (self.config.image_shape[0], self.config.image_shape[1], 1)
    )(label_embedding)

    x = Concatenate()([image_input, label_embedding])

    current_filters = self.arch_config.base_filters

    for i in range(5):
        x = self._create_conv_layer(
            current_filters, padding='same', use_bias=False
        )(x)

        if i > 0:
            x = BatchNormalization()(x)

        x = LeakyReLU(self.arch_config.leaky_relu_alpha)(x)
        x = Dropout(self.arch_config.dropout_rate)(x)

        if i == 2:
            x = self._add_self_attention(x)

        current_filters = min(current_filters * 2, 512)

    x = GlobalAveragePooling2D()(x)
    x = Dense(self.arch_config.dense_units)(x)
    x = LeakyReLU(self.arch_config.leaky_relu_alpha)(x)
    x = Dropout(self.arch_config.dropout_rate)(x)

    validity_output = Dense(1, activation='sigmoid', name="validity")(x)
    class_output = Dense(self.config.num_classes, activation='softmax',
                        name="class_prediction")(x)

```



```

model = Model([image_input, label_input], [validity_output, class_output],
              name="conditional_discriminator")
return model

```

```

In [15]: class VAEcGANArchitecture:
    def __init__(self, config: VAEcGANConfig, arch_config: ArchitectureConfig):
        self.config = config
        self.arch_config = arch_config

        self.vae_encoder = None
        self.vae_decoder = None
        self.generator = None
        self.discriminator = None
        self.vae_model = None
        self.gan_model = None

    def build_models(self) -> Dict[str, Model]:
        logger.info("Building VAE+cGAN architecture...")

        encoder_builder = VAEEncoder(self.config, self.arch_config)
        decoder_builder = VAEDecoder(self.config, self.arch_config)

        self.vae_encoder = encoder_builder.build()
        self.vae_decoder = decoder_builder.build()

        generator_builder = ConditionalGenerator(self.config, self.arch_config)
        discriminator_builder = ConditionalDiscriminator(self.config, self.arch_config)

        self.generator = generator_builder.build()
        self.discriminator = discriminator_builder.build()

        self._build_vae_model()
        self._build_gan_model()

        if self.config.pretrained_vae_path:
            self._load_pretrained_vae()

        return {
            "vae_encoder": self.vae_encoder,
            "vae_decoder": self.vae_decoder,
            "generator": self.generator,
            "discriminator": self.discriminator,
            "vae_model": self.vae_model,
            "gan_model": self.gan_model
        }

    def _build_vae_model(self):
        inputs = Input(shape=self.config.image_shape, name="vae_input")

        z_mean, z_log_var = self.vae_encoder(inputs)
        z = SamplingLayer()(z_mean, z_log_var)
        reconstructed = self.vae_decoder(z)

        self.vae_model = Model(inputs, [reconstructed, z_mean, z_log_var],
                               name="vae_model")

```

```

def _build_gan_model(self):
    noise_input = Input(shape=(self.config.noise_dim,), name="gan_noise")
    label_input = Input(shape=(1,), name="gan_label", dtype='int32')

    generated_latent = self.generator([noise_input, label_input])
    generated_image = self.vae_decoder(generated_latent)

    self.discriminator.trainable = True
    validity, class_pred = self.discriminator([generated_image, label_input])

    self.gan_model = Model([noise_input, label_input], [validity, class_pred],
                           name="gan_model")

def _load_pretrained_vae(self):
    try:
        logger.info(f"Loading pre-trained VAE from {self.config.pretrained_vae_path}")

        encoder_path = Path(self.config.pretrained_vae_path) / "encoder.pth"
        decoder_path = Path(self.config.pretrained_vae_path) / "decoder.pth"

        if encoder_path.exists():
            self.vae_encoder.load_weights(str(encoder_path))
            if self.config.freeze_vae_encoder:
                self.vae_encoder.trainable = False
                logger.info("VAE encoder frozen")

        if decoder_path.exists():
            self.vae_decoder.load_weights(str(decoder_path))
            if self.config.freeze_vae_decoder:
                self.vae_decoder.trainable = False
                logger.info("VAE decoder frozen")

    except Exception as e:
        logger.warning(f"Could not load pre-trained VAE: {e}")

def get_model_summary(self) -> Dict[str, str]:
    summaries = {}

    for name, model in [
        ("VAE Encoder", self.vae_encoder),
        ("VAE Decoder", self.vae_decoder),
        ("Generator", self.generator),
        ("Discriminator", self.discriminator)
    ]:
        if model:
            summary_list = []
            model.summary(print_fn=lambda x: summary_list.append(x))
            summaries[name] = '\n'.join(summary_list)

    return summaries

```

```

In [16]: @dataclass
class VAECGANTrainingConfig:
    num_epochs: int = 10000
    batch_size: int = 16
    vae_train_frequency: int = 1

```

```

gan_train_frequency: int = 1
discriminator_steps: int = 1
vae_lr: float = 0.0001
generator_lr: float = 0.0002
discriminator_lr: float = 0.0002
reconstruction_weight: float = 10.0
kl_weight: float = 1.0
adversarial_weight: float = 1.0
classification_weight: float = 1.0
visualization_frequency: int = 50
save_frequency: int = 100
log_frequency: int = 100
weights_save_path: Optional[str] = "vae_cgan_weights"
samples_save_path: Optional[str] = "generated_samples"
real_time_visualization: bool = True
grid_size: Tuple[int, int] = (2, 4)

```

```

In [17]: @dataclass
class TrainingMetrics:
    epoch: int
    batch: int
    vae_loss: float
    reconstruction_loss: float
    kl_loss: float
    generator_loss: float
    discriminator_loss: float
    discriminator_accuracy: float
    classification_accuracy: float
    timestamp: float = field(default_factory=time.time)

```

1.12 Доп. функции

```

In [18]: class VAECGANLossManager:
    def __init__(self, config: VAECGANTrainingConfig):
        self.config = config
        self.binary_crossentropy = BinaryCrossentropy(from_logits=False)
        self.categorical_crossentropy = SparseCategoricalCrossentropy(from_logits=False)

    def vae_loss(self, y_true: tf.Tensor, y_pred: List[tf.Tensor]) -> Tuple[float, tf.Tensor, tf.Tensor]:
        reconstructed, z_mean, z_log_var = y_pred

        reconstruction_loss = tf.reduce_mean(
            tf.reduce_sum(tf.square(y_true - reconstructed), axis=[1, 2, 3])
        )

        kl_loss = -0.5 * tf.reduce_mean(
            tf.reduce_sum(1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var), axis=[1, 2, 3])
        )

        total_vae_loss = (self.config.reconstruction_weight * reconstruction_loss +
                          self.config.kl_weight * kl_loss)

        return total_vae_loss, reconstruction_loss, kl_loss

```

```

def discriminator_loss(self, real_validity: tf.Tensor, fake_validity: tf.Tensor,
                      real_labels: tf.Tensor, real_class_pred: tf.Tensor,
                      fake_labels: tf.Tensor, fake_class_pred: tf.Tensor) -> tf.Tensor:
    real_loss = self.binary_crossentropy(tf.ones_like(real_validity), real_labels)
    fake_loss = self.binary_crossentropy(tf.zeros_like(fake_validity), fake_labels)
    adversarial_loss = (real_loss + fake_loss) / 2

    class_loss = (self.categorical_crossentropy(real_labels, real_class_pred) +
                  self.categorical_crossentropy(fake_labels, fake_class_pred))

    total_d_loss = (self.config.adversarial_weight * adversarial_loss +
                    self.config.classification_weight * class_loss)

    return total_d_loss, adversarial_loss

def generator_loss(self, fake_validity: tf.Tensor, fake_labels: tf.Tensor,
                  fake_class_pred: tf.Tensor) -> tf.Tensor:
    adversarial_loss = self.binary_crossentropy(tf.ones_like(fake_validity), fake_labels)
    class_loss = self.categorical_crossentropy(fake_labels, fake_class_pred)

    total_g_loss = (self.config.adversarial_weight * adversarial_loss +
                    self.config.classification_weight * class_loss)

    return total_g_loss

```

```

In [19]: class VAECGANSampleGenerator:
def __init__(self, noise_dim: int, num_classes: int):
    self.noise_dim = noise_dim
    self.num_classes = num_classes

def generate_noise(self, batch_size: int) -> np.ndarray:
    return np.random.randn(batch_size, self.noise_dim).astype(np.float32)

def generate_labels(self, batch_size: int, specific_class: Optional[int]) -> np.ndarray:
    if specific_class is not None:
        return np.full((batch_size, 1), specific_class, dtype=np.int32)
    return np.random.randint(0, self.num_classes, (batch_size, 1)).astype(np.int32)

def sample_real_data(self, dataset: np.ndarray, labels: np.ndarray,
                    batch_size: int) -> Tuple[np.ndarray, np.ndarray]:
    indices = np.random.randint(0, len(dataset), batch_size)
    return dataset[indices], labels[indices]

```

```

In [20]: class SimpleProgressTracker:
def __init__(self, log_frequency: int = 100):
    self.log_frequency = log_frequency
    self.start_time = time.time()

def should_log(self, batch: int) -> bool:
    return batch % self.log_frequency == 0 or batch == 0

def log_progress(self, epoch: int, batch: int, total_batches: int,
                vae_loss: float, gen_loss: float, disc_loss: float, disc_gen_loss: float):
    if self.should_log(batch):
        elapsed = time.time() - self.start_time

```

```

print(f"\rEpoch {epoch + 1:4d} | Batch {batch + 1:4d}/{total_batches}
      f"VAE: {vae_loss:.4f} | Gen: {gen_loss:.4f} | "
      f"Disc: {disc_loss:.4f} | D_Acc: {disc_acc:.3f} | "
      f"Time: {elapsed:.1f}s", end='', flush=True)

```

```

In [21]: class VAEGANVisualizer:
    def __init__(self, config: VAEGANTrainingConfig, num_classes: int):
        self.config = config
        self.num_classes = num_classes
        self.sample_generator = VAEGANSampleGenerator(100, num_classes)

        plt.style.use('default')
        plt.ion()

        self.fixed_noise = self.sample_generator.generate_noise(
            self.config.grid_size[0] * self.config.grid_size[1]
        )

        samples_needed = self.config.grid_size[0] * self.config.grid_size[1]
        if self.num_classes <= samples_needed:
            repeats = samples_needed // self.num_classes + 1
            self.fixed_labels = np.tile(np.arange(self.num_classes), repeats)
        else:
            self.fixed_labels = np.random.randint(0, self.num_classes, samples_needed)

        self.fixed_labels = self.fixed_labels.reshape(-1, 1).astype(np.int32)

    def update_visualization(self, models: Dict[str, Any], metrics: TrainingMetrics,
                           total_batches: int) -> None:
        clear_output(wait=True)

        try:
            fig = plt.figure(figsize=(20, 14))

            gs = fig.add_gridspec(
                3, 2,
                height_ratios=[3, 1, 1],
                width_ratios=[2, 1],
                hspace=0.3,
                wspace=0.2
            )

            self._plot_generated_images(fig, gs, models)
            self._plot_loss_curves(fig, gs, metrics)
            self._plot_training_info(fig, gs, metrics, total_batches)

            plt.show()

        except Exception as e:
            print(f"Visualization error: {e}")

    def _plot_generated_images(self, fig, gs, models):
        try:
            generator = models['generator']
            vae_decoder = models['vae_decoder']

```

```

        generated_latents = generator.predict([self.fixed_noise, self.fixed_noise])
        generated_images = vae_decoder.predict(generated_latents, verbose=False)

        generated_images = (generated_images + 1) / 2
        generated_images = np.clip(generated_images, 0, 1)

        ax_main = fig.add_subplot(gs[0, :])
        ax_main.set_title('Generated Images', fontsize=12, fontweight='bold')
        ax_main.axis('off')

        rows, cols = self.config.grid_size

        for i in range(min(rows * cols, len(generated_images))):
            row = i // cols
            col = i % cols

            left = col * 0.8 / cols
            bottom = row * 0.8 / rows
            width = 0.8 / cols
            height = 0.8 / rows

            img_ax = fig.add_axes([left, bottom, width, height])
            img_ax.imshow(generated_images[i])
            img_ax.set_title(f'Class {self.fixed_labels[i][0]}', fontsize=10)
            img_ax.axis('off')

            for spine in img_ax.spines.values():
                spine.set_visible(True)
                spine.set_linewidth(2)
                spine.set_edgecolor('black')

    except Exception as e:
        print(f"Image generation error: {e}")

def _plot_loss_curves(self, fig, gs, metrics):
    ax_losses = fig.add_subplot(gs[1, 0])

    losses = ['VAE', 'Recon', 'KL', 'Gen', 'Disc']
    values = [
        metrics.vae_loss,
        metrics.reconstruction_loss,
        metrics.kl_loss,
        metrics.generator_loss,
        metrics.discriminator_loss
    ]

    colors = ['#2E86AB', '#A23B72', '#F18F01', '#C73E1D', '#6A994E']
    bars = ax_losses.bar(losses, values, color=colors, alpha=0.8, edgecolor='black')

    ax_losses.set_title('Current Losses', fontsize=14, fontweight='bold')
    ax_losses.set_ylabel('Loss Value', fontsize=12)
    ax_losses.tick_params(axis='x', rotation=45, labelsiz=10)
    ax_losses.tick_params(axis='y', labelsiz=10)
    ax_losses.grid(True, alpha=0.3, axis='y')

    for bar, value in zip(bars, values):

```

```

        height = bar.get_height()
        ax_losses.text(
            bar.get_x() + bar.get_width()/2.,
            height + height * 0.02,
            f'{value:.3f}',
            ha='center', va='bottom',
            fontsize=9, fontweight='bold'
        )

def _plot_training_info(self, fig, gs, metrics, total_batches):
    ax_info = fig.add_subplot(gs[1, 1])

    progress = (metrics.batch + 1) / total_batches
    categories = ['D_Acc', 'C_Acc', 'Progress']
    values = [
        metrics.discriminator_accuracy * 100,
        metrics.classification_accuracy * 100,
        progress * 100
    ]

    colors = ['#E74C3C', '#27AE60', '#3498DB']
    bars = ax_info.barh(categories, values, color=colors, alpha=0.8, edge

    ax_info.set_xlim(0, 100)
    ax_info.set_title('Accuracy & Progress (%)', fontsize=14, fontweight
    ax_info.set_xlabel('Percentage', fontsize=12)
    ax_info.tick_params(axis='x', labels=10)
    ax_info.tick_params(axis='y', labels=10)
    ax_info.grid(True, alpha=0.3, axis='x')

    for bar, value in zip(bars, values):
        width = bar.get_width()
        ax_info.text(
            width + 1,
            bar.get_y() + bar.get_height()/2.,
            f'{value:.1f}%',
            ha='left', va='center',
            fontsize=10, fontweight='bold'
        )

    ax_stats = fig.add_subplot(gs[2, :])

    stats_text = (
        f"Epoch: {metrics.epoch + 1:4d} | "
        f"Batch: {metrics.batch + 1:4d}/{total_batches} | "
        f"VAE Loss: {metrics.vae_loss:.4f} | "
        f"Generator Loss: {metrics.generator_loss:.4f} | "
        f"Discriminator Loss: {metrics.discriminator_loss:.4f} | "
        f"D_Accuracy: {metrics.discriminator_accuracy:.3f} | "
        f"Class_Accuracy: {metrics.classification_accuracy:.3f}"
    )

    ax_stats.text(
        0.5, 0.5, stats_text,
        transform=ax_stats.transAxes,
        ha='center', va='center',

```

```

        fontsize=14, fontweight='bold',
        bbox=dict(
            boxstyle="round,pad=0.5",
            facecolor="#ECF0F1",
            edgecolor="#34495E",
            linewidth=2
        )
    )
    ax_stats.axis('off')

```

1.13 Трейнер

```

In [22]: class VAEGANTrainer:
    def __init__(self, architecture, config: VAEGANTrainingConfig):
        self.architecture = architecture
        self.config = config
        self.models = architecture.build_models()

        self.loss_manager = VAEGANLossManager(config)
        self.sample_generator = VAEGANSampleGenerator(
            architecture.config.noise_dim, architecture.config.num_classes
        )

        if config.real_time_visualization:
            self.visualizer = VAEGANVisualizer(config, architecture.config)
        else:
            self.visualizer = None

        self.progress_tracker = SimpleProgressTracker(config.log_frequency)
        self.training_history: List[TrainingMetrics] = []
        self._setup_optimizers()

    def _setup_optimizers(self):
        self.vae_optimizer = Adam(
            learning_rate=self.config.vae_lr,
            beta_1=0.5, beta_2=0.999
        )
        self.generator_optimizer = Adam(
            learning_rate=self.config.generator_lr,
            beta_1=0.5, beta_2=0.999
        )
        self.discriminator_optimizer = Adam(
            learning_rate=self.config.discriminator_lr,
            beta_1=0.5, beta_2=0.999
        )

    def _train_vae_step(self, real_images: tf.Tensor) -> Tuple[float, float,
        with tf.GradientTape() as tape:
            vae_outputs = self.models['vae_model'](real_images, training=True)
            vae_loss, recon_loss, kl_loss = self.loss_manager.vae_loss(real_

            vae_vars = (self.models['vae_encoder'].trainable_variables +
                self.models['vae_decoder'].trainable_variables)
            gradients = tape.gradient(vae_loss, vae_vars)
            self.vae_optimizer.apply_gradients(zip(gradients, vae_vars))

```



```

        return float(vae_loss), float(recon_loss), float(kl_loss)

def _train_discriminator_step(self, real_images: tf.Tensor, real_labels:
                             noise: tf.Tensor, fake_labels: tf.Tensor) -
    with tf.GradientTape() as tape:
        real_validity, real_class_pred = self.models['discriminator']([
            real_images, real_labels], training=True
        )

        generated_latents = self.models['generator']([noise, fake_labels])
        fake_images = self.models['vae_decoder'](generated_latents, traini
        fake_validity, fake_class_pred = self.models['discriminator']([
            fake_images, fake_labels], training=True
        )

        d_loss, adv_loss = self.loss_manager.discriminator_loss(
            real_validity, fake_validity, real_labels, real_class_pred,
            fake_labels, fake_class_pred
        )

        d_vars = self.models['discriminator'].trainable_variables
        gradients = tape.gradient(d_loss, d_vars)
        self.discriminator_optimizer.apply_gradients(zip(gradients, d_vars))

    return float(d_loss), float(adv_loss)

def _train_generator_step(self, noise: tf.Tensor, fake_labels: tf.Tensor)
    with tf.GradientTape() as tape:
        generated_latents = self.models['generator']([noise, fake_labels])
        fake_images = self.models['vae_decoder'](generated_latents, traini

        fake_validity, fake_class_pred = self.models['discriminator']([
            fake_images, fake_labels], training=False
        )

        g_loss = self.loss_manager.generator_loss(fake_validity, fake_la

        g_vars = self.models['generator'].trainable_variables
        gradients = tape.gradient(g_loss, g_vars)
        self.generator_optimizer.apply_gradients(zip(gradients, g_vars))

    return float(g_loss)

def train(self, dataset: np.ndarray, labels: np.ndarray) -> List[Trainin
    logger.info("Starting VAE+cGAN training...")

    dataset = dataset.astype(np.float32)
    if labels.ndim == 1:
        labels = labels.reshape(-1, 1)
    labels = labels.astype(np.int32)

    batches_per_epoch = len(dataset) // self.config.batch_size

    print(f" Training started: {self.config.num_epochs} epochs, "
          f"{batches_per_epoch} batches per epoch")

```

```

print(f" Dataset shape: {dataset.shape}, Labels shape: {labels.shape}")

for epoch in range(self.config.num_epochs):
    epoch_start_time = time.time()

    for batch in range(batches_per_epoch):
        real_images, real_labels = self.sample_generator.sample_real(
            dataset, labels, self.config.batch_size
        )

        noise = self.sample_generator.generate_noise(self.config.batch_size)
        fake_labels = self.sample_generator.generate_labels(self.config.batch_size)

        real_images = tf.convert_to_tensor(real_images, dtype=tf.float32)
        real_labels = tf.convert_to_tensor(real_labels, dtype=tf.int32)
        noise = tf.convert_to_tensor(noise, dtype=tf.float32)
        fake_labels = tf.convert_to_tensor(fake_labels, dtype=tf.int32)

        if batch % self.config.vae_train_frequency == 0:
            vae_loss, recon_loss, kl_loss = self._train_vae_step(real_images, real_labels, noise, fake_labels)
        else:
            vae_loss = recon_loss = kl_loss = 0.0

        d_loss = adv_loss = 0.0
        for _ in range(self.config.discriminator_steps):
            d_loss, adv_loss = self._train_discriminator_step(
                real_images, real_labels, noise, fake_labels
            )

        if batch % self.config.gan_train_frequency == 0:
            g_loss = self._train_generator_step(noise, fake_labels)
        else:
            g_loss = 0.0

        metrics = self._calculate_metrics(
            epoch, batch, vae_loss, recon_loss, kl_loss, g_loss, d_loss,
            real_images, real_labels
        )

        self.training_history.append(metrics)

        if (self.config.real_time_visualization and self.visualizer is not None and
            batch % self.config.visualization_frequency == 0):
            try:
                self.visualizer.update_visualization(
                    self.models, metrics, batches_per_epoch
                )
            except Exception as e:
                print(f"Visualization error: {e}")
        else:
            self.progress_tracker.log_progress(
                epoch, batch, batches_per_epoch,
                vae_loss, g_loss, d_loss, metrics.discriminator_accuracy
            )

    epoch_time = time.time() - epoch_start_time

```

```

        print(f"\nEpoch {epoch + 1} completed in {epoch_time:.2f}s")

        if epoch % self.config.save_frequency == 0 and epoch > 0:
            self._save_models(epoch)

    logger.info("Training completed!")
    return self.training_history

def _calculate_metrics(self, epoch: int, batch: int, vae_loss: float,
                       recon_loss: float, kl_loss: float, g_loss: float,
                       d_loss: float, real_images: tf.Tensor,
                       real_labels: tf.Tensor) -> TrainingMetrics:

    try:
        real_validity, real_class_pred = self.models['discriminator'](
            [real_images, real_labels], training=False
        )

        d_acc_real = tf.reduce_mean(tf.cast(real_validity > 0.5, tf.float32))

        real_labels_int64 = tf.cast(tf.squeeze(real_labels), tf.int64)
        predicted_classes = tf.cast(tf.argmax(real_class_pred, axis=1), tf.int64)

        class_acc = tf.reduce_mean(
            tf.cast(tf.equal(predicted_classes, real_labels_int64), tf.float32)
        )

    except Exception as e:
        print(f"Metrics calculation error: {e}")
        d_acc_real = tf.constant(0.5, dtype=tf.float32)
        class_acc = tf.constant(0.1, dtype=tf.float32)

    return TrainingMetrics(
        epoch=epoch,
        batch=batch,
        vae_loss=vae_loss,
        reconstruction_loss=recon_loss,
        kl_loss=kl_loss,
        generator_loss=g_loss,
        discriminator_loss=d_loss,
        discriminator_accuracy=float(d_acc_real),
        classification_accuracy=float(class_acc)
    )

def _save_models(self, epoch: int):
    if self.config.weights_save_path:
        save_dir = Path(self.config.weights_save_path) / f"epoch_{epoch}"
        save_dir.mkdir(parents=True, exist_ok=True)

        try:
            self.models['vae_encoder'].save_weights(save_dir / "vae_encoder.weights")
            self.models['vae_decoder'].save_weights(save_dir / "vae_decoder.weights")
            self.models['generator'].save_weights(save_dir / "generator.weights")
            self.models['discriminator'].save_weights(save_dir / "discriminator.weights")

        except Exception as e:
            print(f"Error saving models: {e}")

        logger.info(f"Models saved to {save_dir}")

```

```

        except Exception as e:
            logger.error(f"Failed to save models: {e}")

    def generate_samples(self, num_samples: int = 8,
                        specific_classes: Optional[List[int]] = None) -> np.ndarray:
        noise = self.sample_generator.generate_noise(num_samples)

        if specific_classes:
            labels = np.array(specific_classes).reshape(-1, 1)
        else:
            labels = self.sample_generator.generate_labels(num_samples)

        try:
            generated_latents = self.models['generator'].predict([noise, labels])
            generated_images = self.models['vae_decoder'].predict(generated_latents)

            return (generated_images + 1) / 2
        except Exception as e:
            print(f"Sample generation error: {e}")
            return np.random.rand(num_samples, *self.architecture.config.image_shape)

    def get_training_summary(self) -> Dict[str, Any]:
        if not self.training_history:
            return {}

        recent = self.training_history[-10:]

        return {
            "total_epochs": max(m.epoch for m in self.training_history) + 1,
            "total_batches": len(self.training_history),
            "final_vae_loss": self.training_history[-1].vae_loss,
            "final_generator_loss": self.training_history[-1].generator_loss,
            "final_discriminator_loss": self.training_history[-1].discriminator_loss,
            "avg_discriminator_accuracy": np.mean([m.discriminator_accuracy for m in recent]),
            "avg_classification_accuracy": np.mean([m.classification_accuracy for m in recent])
        }

```

1.14 Обучение

1.15 Подготовка данных

```

In [6]: path = kagglehub.dataset_download("andrewmvd/animal-faces")
        print("Path to dataset files:", path)

```

Path to dataset files: /home/student/.cache/kagglehub/datasets/andrewmvd/animal-faces/versions/1

```

In [24]: dataset_info, label_mapping, distribution = load_image_dataset_with_info(
        data_path=os.path.join(path, 'afhq/train'),
        target_size=(64, 64),
        pattern="*.jpg",
        limit=25000,
        batch_size=3000,
        strategy=DatasetStrategy.HIERARCHICAL,
    )

```

```

    return_labels=True
)

logger.info(f"Classes: {label_mapping.class_to_idx}")
logger.info(f"Distribution: {distribution}")

```

```

2025-06-01 21:35:46,486 - __main__ - INFO - Discovered 3 classes: ['wild',
'dog', 'cat']
INFO: __main__:Discovered 3 classes: ['wild', 'dog', 'cat']
2025-06-01 21:35:46,531 - __main__ - INFO - Found 14630 images across 3 clas
ses
INFO: __main__:Found 14630 images across 3 classes
2025-06-01 21:35:46,532 - __main__ - INFO - Loading dataset with 14630 image
s
INFO: __main__:Loading dataset with 14630 images
2025-06-01 21:35:57,515 - __main__ - INFO - Processed batch 1/5
INFO: __main__:Processed batch 1/5
2025-06-01 21:36:08,274 - __main__ - INFO - Processed batch 2/5
INFO: __main__:Processed batch 2/5
2025-06-01 21:36:18,982 - __main__ - INFO - Processed batch 3/5
INFO: __main__:Processed batch 3/5
2025-06-01 21:36:29,621 - __main__ - INFO - Processed batch 4/5
INFO: __main__:Processed batch 4/5
2025-06-01 21:36:39,130 - __main__ - INFO - Processed batch 5/5
INFO: __main__:Processed batch 5/5
2025-06-01 21:36:39,323 - __main__ - INFO - Dataset loaded successfully. Ima
ges shape: (14630, 64, 64, 3)
INFO: __main__:Dataset loaded successfully. Images shape: (14630, 64, 64, 3)
2025-06-01 21:36:39,324 - __main__ - INFO - Labels shape: (14630,)
INFO: __main__:Labels shape: (14630,)
2025-06-01 21:36:39,334 - __main__ - INFO - Classes: {'cat': 0, 'dog': 1, 'w
ild': 2}
INFO: __main__:Classes: {'cat': 0, 'dog': 1, 'wild': 2}
2025-06-01 21:36:39,335 - __main__ - INFO - Distribution: {'wild': 4738, 'do
g': 4739, 'cat': 5153}
INFO: __main__:Distribution: {'wild': 4738, 'dog': 4739, 'cat': 5153}

```

1.16 Условный генератор с VAE

```

In [25]: model_config = VAEGANConfig(
    image_shape=(64, 64, 3),
    num_classes=3,
    latent_dim=128,
    pretrained_vae_path=None
)

arch_config = ArchitectureConfig(
    use_spectral_norm=False,
    use_self_attention=True,
    base_filters=64,
    dropout_rate=0.25
)

architecture = VAEGANArchitecture(model_config, arch_config)

```

```
In [26]: training_config = VAEGANTrainingConfig(
    real_time_visualization=True,
    num_epochs=10000,
    batch_size=512,
    visualization_frequency=10,
    log_frequency=100,
    save_frequency=100
)
trainer = VAEGANTrainer(architecture, training_config)
```

```
2025-06-01 21:36:39,344 - __main__ - INFO - Building VAE+cGAN architecture...
```

```
INFO: __main__:Building VAE+cGAN architecture...
```

```
I0000 00:00:1748813799.577808 4154575 gpu_device.cc:2019] Created device /job:localhost/replica:0/task:0/device:GPU:0 with 65147 MB memory: -> device:0, name: NVIDIA A100 80GB PCIe, pci bus id: 0000:0b:00.0, compute capability: 8.0
```

```
In [27]: summaries = architecture.get_model_summary()
    for name, summary in summaries.items():
        print(f"\n=== {name} ===")
        print(summary)
```

=== VAE Encoder ===
Model: "vae_encoder"

Layer (type)	Output Shape	Param #	Connected to
encoder_input (InputLayer)	(None, 64, 64, 3)	0	-
conv2d (Conv2D)	(None, 32, 32, 64)	3,072	encoder_input[0]...
batch_normalization (BatchNormalizatio...	(None, 32, 32, 64)	256	conv2d[0][0]
leaky_re_lu (LeakyReLU)	(None, 32, 32, 64)	0	batch_normalizat...
conv2d_1 (Conv2D)	(None, 16, 16, 128)	131,072	leaky_re_lu[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 16, 16, 128)	512	conv2d_1[0][0]
leaky_re_lu_1 (LeakyReLU)	(None, 16, 16, 128)	0	batch_normalizat...
conv2d_2 (Conv2D)	(None, 8, 8, 256)	524,288	leaky_re_lu_1[0]...
batch_normalizatio... (BatchNormalizatio...	(None, 8, 8, 256)	1,024	conv2d_2[0][0]
leaky_re_lu_2 (LeakyReLU)	(None, 8, 8, 256)	0	batch_normalizat...
self_attention_lay... (SelfAttentionLaye...	(None, 8, 8, 256)	82,241	leaky_re_lu_2[0]...
conv2d_3 (Conv2D)	(None, 4, 4, 512)	2,097,152	self_attention_l...
batch_normalizatio... (BatchNormalizatio...	(None, 4, 4, 512)	2,048	conv2d_3[0][0]
leaky_re_lu_3 (LeakyReLU)	(None, 4, 4, 512)	0	batch_normalizat...
conv2d_4 (Conv2D)	(None, 2, 2, 512)	4,194,304	leaky_re_lu_3[0]...
batch_normalizatio... (BatchNormalizatio...	(None, 2, 2, 512)	2,048	conv2d_4[0][0]
leaky_re_lu_4 (LeakyReLU)	(None, 2, 2, 512)	0	batch_normalizat...
global_average_poo... (GlobalAveragePool...	(None, 512)	0	leaky_re_lu_4[0]...

dense (Dense)	(None, 512)	262,656	global_average_p...
leaky_re_lu_5 (LeakyReLU)	(None, 512)	0	dense[0][0]
dropout (Dropout)	(None, 512)	0	leaky_re_lu_5[0]...
z_mean (Dense)	(None, 128)	65,664	dropout[0][0]
z_log_var (Dense)	(None, 128)	65,664	dropout[0][0]

Total params: 7,432,001 (28.35 MB)
Trainable params: 7,429,057 (28.34 MB)
Non-trainable params: 2,944 (11.50 KB)

=== VAE Decoder ===
Model: "vae_decoder"

Layer (type)	Output Shape	Param #
latent_input (InputLayer)	(None, 128)	0
dense_1 (Dense)	(None, 2048)	264,192
batch_normalization_5 (BatchNormalization)	(None, 2048)	8,192
leaky_re_lu_6 (LeakyReLU)	(None, 2048)	0
reshape (Reshape)	(None, 2, 2, 512)	0
conv2d_transpose (Conv2DTranspose)	(None, 4, 4, 256)	2,097,152
batch_normalization_6 (BatchNormalization)	(None, 4, 4, 256)	1,024
leaky_re_lu_7 (LeakyReLU)	(None, 4, 4, 256)	0
conv2d_transpose_1 (Conv2DTranspose)	(None, 8, 8, 128)	524,288
batch_normalization_7 (BatchNormalization)	(None, 8, 8, 128)	512
leaky_re_lu_8 (LeakyReLU)	(None, 8, 8, 128)	0
conv2d_transpose_2 (Conv2DTranspose)	(None, 16, 16, 64)	131,072
batch_normalization_8 (BatchNormalization)	(None, 16, 16, 64)	256
leaky_re_lu_9 (LeakyReLU)	(None, 16, 16, 64)	0

self_attention_layer_1 (SelfAttentionLayer)	(None, 16, 16, 64)	5,201
conv2d_transpose_3 (Conv2DTranspose)	(None, 32, 32, 32)	32,768
batch_normalization_9 (BatchNormalization)	(None, 32, 32, 32)	128
leaky_re_lu_10 (LeakyReLU)	(None, 32, 32, 32)	0
conv2d_transpose_4 (Conv2DTranspose)	(None, 64, 64, 16)	8,192
batch_normalization_10 (BatchNormalization)	(None, 64, 64, 16)	64
leaky_re_lu_11 (LeakyReLU)	(None, 64, 64, 16)	0
decoded_image (Conv2DTranspose)	(None, 64, 64, 3)	771

Total params: 3,073,812 (11.73 MB)

Trainable params: 3,068,724 (11.71 MB)

Non-trainable params: 5,088 (19.88 KB)

=== Generator ===

Model: "conditional_generator"

Layer (type)	Output Shape	Param #	Connected to
label_input (InputLayer)	(None, 1)	0	-
embedding (Embedding)	(None, 1, 128)	384	label_input[0][0]
noise_input (InputLayer)	(None, 100)	0	-
flatten (Flatten)	(None, 128)	0	embedding[0][0]
concatenate (Concatenate)	(None, 228)	0	noise_input[0][0]... flatten[0][0]
dense_2 (Dense)	(None, 512)	117,248	concatenate[0][0]
batch_normalizatio... (BatchNormalizatio...)	(None, 512)	2,048	dense_2[0][0]
leaky_re_lu_12 (LeakyReLU)	(None, 512)	0	batch_normalizatio...
dropout_1 (Dropout)	(None, 512)	0	leaky_re_lu_12[0]...
dense_3 (Dense)	(None, 512)	262,656	dropout_1[0][0]

batch_normalizatio... (BatchNormalizatio...	(None, 512)	2,048	dense_3[0][0]
leaky_re_lu_13 (LeakyReLU)	(None, 512)	0	batch_normalizat...
dropout_2 (Dropout)	(None, 512)	0	leaky_re_lu_13[0...
generated_latent (Dense)	(None, 128)	65,664	dropout_2[0][0]

Total params: 450,048 (1.72 MB)
Trainable params: 448,000 (1.71 MB)
Non-trainable params: 2,048 (8.00 KB)

=== Discriminator ===

Model: "conditional_discriminator"

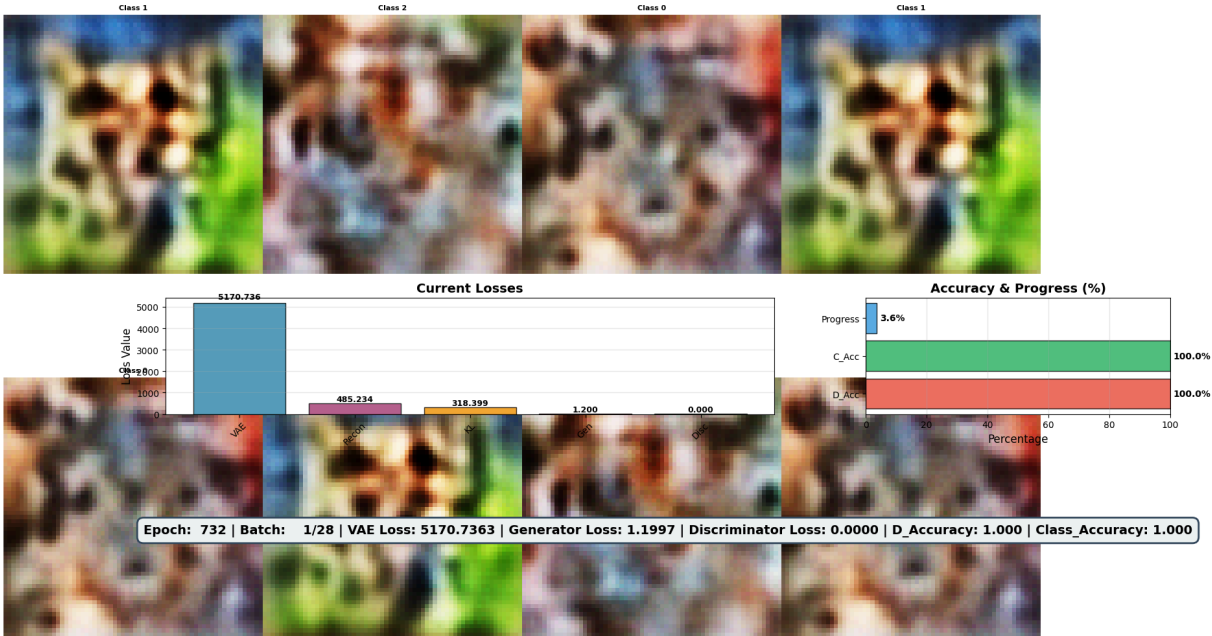
Layer (type)	Output Shape	Param #	Connected to
label_input (InputLayer)	(None, 1)	0	-
embedding_1 (Embedding)	(None, 1, 4096)	12,288	label_input[0][0]
image_input (InputLayer)	(None, 64, 64, 3)	0	-
reshape_1 (Reshape)	(None, 64, 64, 1)	0	embedding_1[0][0]
concatenate_1 (Concatenate)	(None, 64, 64, 4)	0	image_input[0][0... reshape_1[0][0]
conv2d_5 (Conv2D)	(None, 32, 32, 64)	4,096	concatenate_1[0]...
leaky_re_lu_14 (LeakyReLU)	(None, 32, 32, 64)	0	conv2d_5[0][0]
dropout_3 (Dropout)	(None, 32, 32, 64)	0	leaky_re_lu_14[0...
conv2d_6 (Conv2D)	(None, 16, 16, 128)	131,072	dropout_3[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 16, 16, 128)	512	conv2d_6[0][0]
leaky_re_lu_15 (LeakyReLU)	(None, 16, 16, 128)	0	batch_normalizat...
dropout_4 (Dropout)	(None, 16, 16, 128)	0	leaky_re_lu_15[0...

conv2d_7 (Conv2D)	(None, 8, 8, 256)	524,288	dropout_4[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 8, 8, 256)	1,024	conv2d_7[0][0]
leaky_re_lu_16 (LeakyReLU)	(None, 8, 8, 256)	0	batch_normalizat...
dropout_5 (Dropout)	(None, 8, 8, 256)	0	leaky_re_lu_16[0...
self_attention_lay... (SelfAttentionLaye...	(None, 8, 8, 256)	82,241	dropout_5[0][0]
conv2d_8 (Conv2D)	(None, 4, 4, 512)	2,097,152	self_attention_l...
batch_normalizatio... (BatchNormalizatio...	(None, 4, 4, 512)	2,048	conv2d_8[0][0]
leaky_re_lu_17 (LeakyReLU)	(None, 4, 4, 512)	0	batch_normalizat...
dropout_6 (Dropout)	(None, 4, 4, 512)	0	leaky_re_lu_17[0...
conv2d_9 (Conv2D)	(None, 2, 2, 512)	4,194,304	dropout_6[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 2, 2, 512)	2,048	conv2d_9[0][0]
leaky_re_lu_18 (LeakyReLU)	(None, 2, 2, 512)	0	batch_normalizat...
dropout_7 (Dropout)	(None, 2, 2, 512)	0	leaky_re_lu_18[0...
global_average_poo... (GlobalAveragePool...	(None, 512)	0	dropout_7[0][0]
dense_4 (Dense)	(None, 512)	262,656	global_average_p...
leaky_re_lu_19 (LeakyReLU)	(None, 512)	0	dense_4[0][0]
dropout_8 (Dropout)	(None, 512)	0	leaky_re_lu_19[0...
validity (Dense)	(None, 1)	513	dropout_8[0][0]
class_prediction (Dense)	(None, 3)	1,539	dropout_8[0][0]

Total params: 7,315,781 (27.91 MB)
Trainable params: 7,312,965 (27.90 MB)
Non-trainable params: 2,816 (11.00 KB)

```
In [28]: print("\n Starting training...")
training_history = trainer.train(dataset_info[0], dataset_info[1])
```

Generated Images



```

-----
KeyboardInterrupt                                Traceback (most recent call last)
Cell In[28], line 2
      1 print("\n Starting training...")
----> 2 training_history = trainer.train(dataset_info[0], dataset_info[1])

Cell In[22], line 119, in VAECGANTrainer.train(self, dataset, labels)
    116 fake_labels = tf.convert_to_tensor(fake_labels, dtype=tf.int32)
    118 if batch % self.config.vae_train_frequency == 0:
--> 119     vae_loss, recon_loss, kl_loss = self._train_vae_step(real_image
s)
    120 else:
    121     vae_loss = recon_loss = kl_loss = 0.0

Cell In[22], line 37, in VAECGANTrainer._train_vae_step(self, real_images)
    35 def _train_vae_step(self, real_images: tf.Tensor) -> Tuple[float, fl
oat, float]:
    36     with tf.GradientTape() as tape:
--> 37         vae_outputs = self.models[                ](real_images, training
=True)
    38         vae_loss, recon_loss, kl_loss = self.loss_manager.vae_loss(r
eal_images, vae_outputs)
    40         vae_vars = (self.models['vae_encoder'].trainable_variables +
    41                     self.models['vae_decoder'].trainable_variables)

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/utils/t
raceback_utils.py:117, in filter_traceback.<locals>.error_handler(*args, **k
wargs)
    115 filtered_tb = None
    116 try:
--> 117     return fn(*args, **kwargs)
    118 except Exception as e:
    119     filtered_tb = _process_traceback_frames(e.__traceback__)

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/layers/
layer.py:936, in Layer.__call__(self, *args, **kwargs)
    934     outputs = super().__call__(*args, **kwargs)
    935 else:
--> 936     outputs = super().__call__(*args, **kwargs)
    937 # Change the layout for the layer output if needed.
    938 # This is useful for relayout intermediate tensor in the model
    939 # to achieve the optimal performance.
    940 distribution = distribution_lib.distribution()

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/utils/t
raceback_utils.py:117, in filter_traceback.<locals>.error_handler(*args, **k
wargs)
    115 filtered_tb = None
    116 try:
--> 117     return fn(*args, **kwargs)
    118 except Exception as e:
    119     filtered_tb = _process_traceback_frames(e.__traceback__)

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/ops/ope
ration.py:58, in Operation.__call__(self, *args, **kwargs)
     53         call_fn = self.call

```

```

54     call_fn = traceback_utils.inject_argument_info_in_traceback(
55         call_fn,
56         object_name=(f"{self.__class__.__name__}.call()"),
57     )
--> 58     return call_fn(*args, **kwargs)
60 # Plain flow.
61 if any_symbolic_tensors(args, kwargs):

```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/traceback_utils.py:156, in inject_argument_info_in_traceback.<locals>.error_handler(*args, **kwargs)

```

154 bound_signature = None
155 try:
--> 156     return fn(*args, **kwargs)
157 except Exception as e:
158     if hasattr(e, "_keras_call_info_injected"):
159         # Only inject info for the innermost failing call

```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/models/functional.py:183, in Functional.call(self, inputs, training, mask, **kwargs)

```

181         if mask is not None:
182             backend.set_keras_mask(x, mask)
--> 183 outputs = self._run_through_graph(
184     inputs,
185     operation_fn=lambda op: operation_fn(
186         op, training=training, **kwargs
187     ),
188 )
189 return unpack_singleton(outputs)

```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/ops/functional.py:177, in Function._run_through_graph(self, inputs, operation_fn, call_fn)

```

175     outputs = call_fn(op, *args, **kwargs)
176 else:
--> 177     outputs = op(*args, **kwargs)
179 # Update tensor_dict.
180 for x, y in zip(node.outputs, tree.flatten(outputs)):

```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/models/functional.py:648, in operation_fn.<locals>.call(*args, **kwargs)

```

642     if (
643         name in getattr(operation, "_call_context_args", {})
644         and value is not None
645     ):
646         kwargs[name] = value
--> 648 return operation(*args, **kwargs)

```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/traceback_utils.py:117, in filter_traceback.<locals>.error_handler(*args, **kwargs)

```

115 filtered_tb = None
116 try:
--> 117     return fn(*args, **kwargs)
118 except Exception as e:

```

```

119         filtered_tb = _process_traceback_frames(e.__traceback__)

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/layers/
layer.py:936, in Layer.__call__(self, *args, **kwargs)
934         outputs = super().__call__(*args, **kwargs)
935     else:
--> 936         outputs = super().__call__(*args, **kwargs)
937     # Change the layout for the layer output if needed.
938     # This is useful for relayout intermediate tensor in the model
939     # to achieve the optimal performance.
940     distribution = distribution_lib.distribution()

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/utils/t
raceback_utils.py:117, in filter_traceback.<locals>.error_handler(*args, **k
wargs)
115     filtered_tb = None
116     try:
--> 117         return fn(*args, **kwargs)
118     except Exception as e:
119         filtered_tb = _process_traceback_frames(e.__traceback__)

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/ops/ope
ration.py:58, in Operation.__call__(self, *args, **kwargs)
53         call_fn = self.call
54         call_fn = traceback_utils.inject_argument_info_in_traceback(
55             call_fn,
56             object_name=f"{self.__class__.__name__}.call()",
57         )
--> 58         return call_fn(*args, **kwargs)
60     # Plain flow.
61     if any_symbolic_tensors(args, kwargs):

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/utils/t
raceback_utils.py:156, in inject_argument_info_in_traceback.<locals>.error_h
andler(*args, **kwargs)
154     bound_signature = None
155     try:
--> 156         return fn(*args, **kwargs)
157     except Exception as e:
158         if hasattr(e, "_keras_call_info_injected"):
159             # Only inject info for the innermost failing call

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/models/f
unctional.py:183, in Functional.call(self, inputs, training, mask, **kwargs)
181         if mask is not None:
182             backend.set_keras_mask(x, mask)
--> 183     outputs = self._run_through_graph(
184         inputs,
185         operation_fn=lambda op: operation_fn(
186             op, training=training, **kwargs
187         ),
188     )
189     return unpack_singleton(outputs)

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/ops/fun

```

```

tion.py:177, in Function._run_through_graph(self, inputs, operation_fn, call_fn)
    175     outputs = call_fn(op, *args, **kwargs)
    176 else:
--> 177     outputs = op(*args, **kwargs)
    179 # Update tensor_dict.
    180 for x, y in zip(node.outputs, tree.flatten(outputs)):

```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/models/functional.py:648, in operation_fn.<locals>.call(*args, **kwargs)

```

    642     if (
    643         name in getattr(operation, "_call_context_args", {})
    644         and value is not None
    645     ):
    646         kwargs[name] = value
--> 648     return operation(*args, **kwargs)

```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/utils/traceback_utils.py:117, in filter_traceback.<locals>.error_handler(*args, **kwargs)

```

    115 filtered_tb = None
    116 try:
--> 117     return fn(*args, **kwargs)
    118 except Exception as e:
    119     filtered_tb = _process_traceback_frames(e.__traceback__)

```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/layers/layer.py:936, in Layer.__call__(self, *args, **kwargs)

```

    934     outputs = super().__call__(*args, **kwargs)
    935 else:
--> 936     outputs = super().__call__(*args, **kwargs)
    937 # Change the layout for the layer output if needed.
    938 # This is useful for relayout intermediate tensor in the model
    939 # to achieve the optimal performance.
    940 distribution = distribution_lib.distribution()

```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/utils/traceback_utils.py:117, in filter_traceback.<locals>.error_handler(*args, **kwargs)

```

    115 filtered_tb = None
    116 try:
--> 117     return fn(*args, **kwargs)
    118 except Exception as e:
    119     filtered_tb = _process_traceback_frames(e.__traceback__)

```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/ops/operation.py:58, in Operation.__call__(self, *args, **kwargs)

```

    53     call_fn = self.call
    54     call_fn = traceback_utils.inject_argument_info_in_traceback(
    55         call_fn,
    56         object_name=f"{self.__class__.__name__}.call()",
    57     )
--> 58     return call_fn(*args, **kwargs)
    60 # Plain flow.
    61 if any_symbolic_tensors(args, kwargs):

```



```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/utils/traceback_utils.py:156, in inject_argument_info_in_traceback.<locals>.error_handler(*args, **kwargs)
    154 bound_signature = None
    155 try:
--> 156     return fn(*args, **kwargs)
    157 except Exception as e:
    158     if hasattr(e, "_keras_call_info_injected"):
    159         # Only inject info for the innermost failing call

Cell In[8], line 26, in SelfAttentionLayer.call(self, inputs)
    24 query = self.query_conv(inputs)
    25 key = self.key_conv(inputs)
---> 26 value = self.value_conv(inputs)
    28 query = tf.reshape(query, [batch_size, height * width, self.channels // 8])
    29 key = tf.reshape(key, [batch_size, height * width, self.channels // 8])

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/utils/traceback_utils.py:117, in filter_traceback.<locals>.error_handler(*args, **kwargs)
    115 filtered_tb = None
    116 try:
--> 117     return fn(*args, **kwargs)
    118 except Exception as e:
    119     filtered_tb = _process_traceback_frames(e.__traceback__)

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/layers/layer.py:936, in Layer.__call__(self, *args, **kwargs)
    934     outputs = super().__call__(*args, **kwargs)
    935 else:
--> 936     outputs = super().__call__(*args, **kwargs)
    937 # Change the layout for the layer output if needed.
    938 # This is useful for relay intermediate tensor in the model
    939 # to achieve the optimal performance.
    940 distribution = distribution_lib.distribution()

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/utils/traceback_utils.py:117, in filter_traceback.<locals>.error_handler(*args, **kwargs)
    115 filtered_tb = None
    116 try:
--> 117     return fn(*args, **kwargs)
    118 except Exception as e:
    119     filtered_tb = _process_traceback_frames(e.__traceback__)

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/ops/operation.py:58, in Operation.__call__(self, *args, **kwargs)
    53     call_fn = self.call
    54     call_fn = traceback_utils.inject_argument_info_in_traceback(
    55         call_fn,
    56         object_name=(f"{self.__class__.__name__}.call()"),
    57     )
--> 58     return call_fn(*args, **kwargs)
    60 # Plain flow.

```

```
61 if any_symbolic_tensors(args, kwargs):
```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/utils/traceback_utils.py:156, in inject_argument_info_in_traceback.<locals>.error_handler(*args, **kwargs)

```
154 bound_signature = None
155 try:
--> 156     return fn(*args, **kwargs)
157 except Exception as e:
158     if hasattr(e, "_keras_call_info_injected"):
159         # Only inject info for the innermost failing call
```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/layers/convolutional/base_conv.py:259, in BaseConv.call(self, inputs)

```
257     else:
258         bias_shape = (1, self.filters) + (1,) * self.rank
--> 259         bias = ops.reshape(self.bias, bias_shape)
260         outputs = ops.add(outputs, bias)
262 if self.activation is not None:
```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/ops/numeric.py:5074, in reshape(x, newshape)

```
5072 if any_symbolic_tensors((x,)):
5073     return Reshape(newshape).symbolic_call(x)
-> 5074 return backend.numpy.reshape(x, newshape)
```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/keras/src/backend/tensorflow/numpy.py:2068, in reshape(x, newshape)

```
2066     output.set_shape(output_shape)
2067     return output
-> 2068 return tf.reshape(x, newshape)
```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/tensorflow/python/ops/weak_tensor_ops.py:88, in weak_tensor_unary_op_wrapper.<locals>.wrapper(*args, **kwargs)

```
86 def wrapper(*args, **kwargs):
87     if not ops.is_auto_dtype_conversion_enabled():
---> 88         return op(*args, **kwargs)
89     bound_arguments = signature.bind(*args, **kwargs)
90     bound_arguments.apply_defaults()
```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/tensorflow/python/util/traceback_utils.py:150, in filter_traceback.<locals>.error_handler(*args, **kwargs)

```
148 filtered_tb = None
149 try:
--> 150     return fn(*args, **kwargs)
151 except Exception as e:
152     filtered_tb = _process_traceback_frames(e.__traceback__)
```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/tensorflow/python/util/dispatch.py:1260, in add_dispatch_support.<locals>.decorator.<locals>.op_dispatch_handler(*args, **kwargs)

```
1258 # fallback dispatch system (dispatch v1):
1259 try:
-> 1260     return dispatch_target(*args, **kwargs)
```

```

1261 except (TypeError, ValueError):
1262     # Note: convert_to_eager_tensor currently raises a ValueError, not
a
1263     # TypeError, when given unexpected types. So we need to catch bot
h.
1264     result = dispatch(op_dispatch_handler, args, kwargs)

```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/tensorflow/python/ops/array_ops.py:199, in reshape(tensor, shape, name)

```

63 @tf_export("reshape", v1=["reshape", "manip.reshape"])
64 @dispatch.add_dispatch_support
65 def reshape(tensor, shape, name=None): # pylint: disable=redefined-
outer-name
66     r"""Reshapes a tensor.
67
68     Given `tensor`, this operation returns a new `tf.Tensor` that has
the same
69     (...) 197     A `Tensor`. Has the same type as `tensor`.
198     """
--> 199     result = gen_array_ops.reshape(tensor, shape, name)
200     shape_util.maybe_set_static_shape(result, shape)
201     return result

```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/tensorflow/python/ops/gen_array_ops.py:8795, in reshape(tensor, shape, name)

```

8793 pass
8794 try:
-> 8795     return reshape_eager_fallback(
8796         tensor, shape, name=name, ctx=_ctx)
8797 except _core.SymbolicException:
8798     pass # Add nodes to the TensorFlow graph.

```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/tensorflow/python/ops/gen_array_ops.py:8817, in reshape_eager_fallback(tensor, shape, name, ctx)

```

8815 def reshape_eager_fallback(tensor: Annotated[Any, TV_Reshape_T], sha
pe: Annotated[Any, TV_Reshape_Tshape], name, ctx) -> Annotated[Any, TV_Resha
pe_T]:
8816     _attr_T, (tensor,) = _execute.args_to_matching_eager([tensor], ct
x, [])
-> 8817     _attr_Tshape, (shape,) = execute.args_to_matching_eager([shape],
ctx, [ dtypes.int32, dtypes.int64, ], dtypes.int32)
8818     _inputs_flat = [tensor, shape]
8819     _attrs = ("T", _attr_T, "Tshape", _attr_Tshape)

```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/tensorflow/python/eager/execute.py:251, in args_to_matching_eager(**failed_resolving_arguments)

```

247 # First see if we can get a valid dtype with the default conversion
248 # and see if it matches an allowed dtypes. Some ops like ConcatV2 ma
y
249 # not list allowed dtypes, in which case we should skip this.
250 if dtype is None and allowed_dtypes:
--> 251     tensor = tensor_conversion_registry.convert(t)
252     # If we did not match an allowed dtype, try again with the default
253     # dtype. This could be because we have an empty tensor and thus we

```

```

254 # picked the wrong type.
255 if tensor.dtype not in allowed_dtypes:

```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/tensorflow/python/framework/tensor_conversion_registry.py:234, in convert(value, dtype, name, as_ref, preferred_dtype, accepted_result_types)

```

225     raise RuntimeError(
226         _add_error_prefix(
227             f"Conversion function {conversion_func!r} for type "
228             f"actual = {ret.dtype.base_dtype.name}",
229             name=name))
230 if ret is None:
--> 234     ret = conversion_func(value, dtype=dtype, name=name, as_ref=as_ref)
235 if ret is NotImplemented:
236     continue

```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/tensorflow/python/framework/constant_tensor_conversion.py:29, in _constant_tensor_conversion_function(v, dtype, name, as_ref)

```

26 from tensorflow.python.framework import constant_op # pylint: disable=g-import-not-at-top
28 _ = as_ref
--> 29 return constant_op.constant(v, dtype=dtype, name=name)

```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/tensorflow/python/ops/weak_tensor_ops.py:142, in weak_tensor_binary_op_wrapper.<locals>.wrapper(*args, **kwargs)

```

140 def wrapper(*args, **kwargs):
141     if not ops.is_auto_dtype_conversion_enabled():
--> 142         return op(*args, **kwargs)
143     bound_arguments = signature.bind(*args, **kwargs)
144     bound_arguments.apply_defaults()

```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/tensorflow/python/framework/constant_op.py:276, in constant(value, dtype, shape, name)

```

177 @tf_export("constant", v1=[])
178 def constant(
179     value, dtype=None, shape=None, name="Const"
180 ) -> Union[ops.Operation, ops._EagerTensorBase]:
181     """Creates a constant tensor from a tensor-like object.
182
183     Note: All eager `tf.Tensor` values are immutable (in contrast to
184     ValueError: if called on a symbolic tensor.
185 """
--> 276     return constant_impl(value, dtype, shape, name, verify_shape=False,
277                          allow_broadcast=True)

```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/tensorflow/python/framework/constant_op.py:289, in _constant_impl(value, dtype, shape, name, verify_shape, allow_broadcast)

```

287     with trace.Trace("tf.constant"):
288         return _constant_eager_impl(ctx, value, dtype, shape, verify_shape)
--> 289     return _constant_eager_impl(ctx, value, dtype, shape, verify_shape)

```

```
e)
291 const_tensor = ops._create_graph_constant( # pylint: disable=protected-access
292     value, dtype, shape, name, verify_shape, allow_broadcast
293 )
294 return const_tensor
```

File ~/projects/CVTools/.venv/lib/python3.12/site-packages/tensorflow/python/framework/constant_op.py:301, in _constant_eager_impl(ctx, value, dtype, shape, verify_shape)

```
297 def _constant_eager_impl(
298     ctx, value, dtype, shape, verify_shape
299 ) -> ops._EagerTensorBase:
300     """Creates a constant on the current device."""
--> 301     t = convert_to_eager_tensor(value, ctx, dtype)
302     if shape is None:
303         return t
```

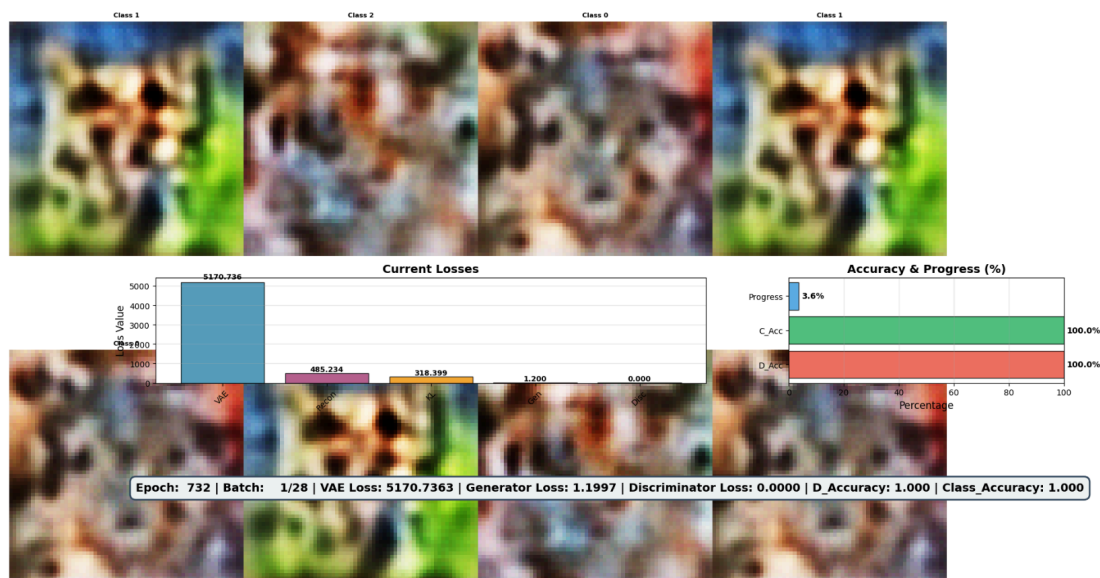
File ~/projects/CVTools/.venv/lib/python3.12/site-packages/tensorflow/python/framework/constant_op.py:108, in convert_to_eager_tensor(value, ctx, dtype)

```
106     dtype = dtypes.as_dtype(dtype).as_datatype_enum
107     ctx.ensure_initialized()
--> 108     return ops.EagerTensor(value, ctx.device_name, dtype)
```

KeyboardInterrupt:

```
In [29]: print("\n Generating samples...")
        samples = trainer.generate_samples(num_samples=8)
```

Generating samples...



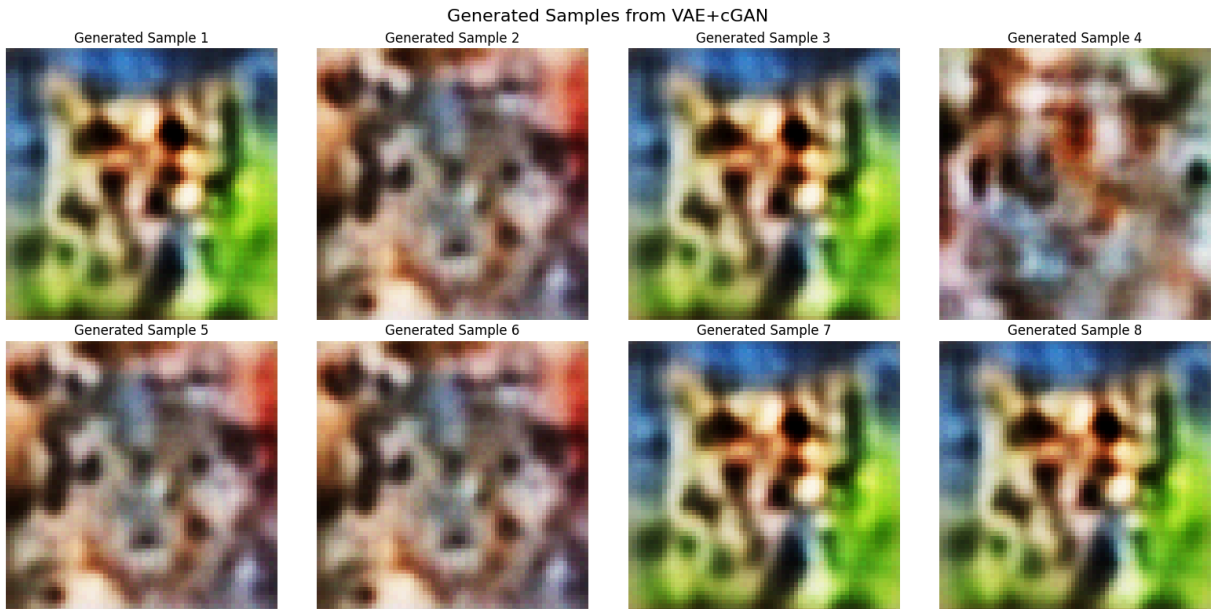
```
In [30]: fig, axes = plt.subplots(2, 4, figsize=(16, 8))
        for i, sample in enumerate(samples):
            row = i // 4
            col = i % 4
```

```

axes[row, col].imshow(sample)
axes[row, col].axis('off')
axes[row, col].set_title(f'Generated Sample {i+1}')

plt.suptitle('Generated Samples from VAE+cGAN', fontsize=16)
plt.tight_layout()
plt.show()

```



```

In [31]: summary = trainer.get_training_summary()
print("\n Training Summary:")
for key, value in summary.items():
    print(f"{key}: {value}")

```

```

 Training Summary:
total_epochs: 732
total_batches: 20469
final_vae_loss: 5170.736328125
final_generator_loss: 1.199704647064209
final_discriminator_loss: 8.346287216909332e-09
avg_discriminator_accuracy: 1.0
avg_classification_accuracy: 1.0

```

2. Применение Vision Transformers (ViT)

2.2 Подготовка данных

```

In [7]: class ImageDataset(Dataset):

    def __init__(
        self,
        dataset_path: Union[str, Path],
        image_processor: AutoImageProcessor,

```

```

split: str = 'train',
test_size: float = 0.2,
val_size: float = 0.2,
random_state: int = 42,
image_extensions: Tuple[str, ...] = ('.jpg', '.jpeg', '.png', '.bmp'
) -> None:
    self.dataset_path = Path(dataset_path)
    self.image_processor = image_processor
    self.split = split
    self.test_size = test_size
    self.val_size = val_size
    self.random_state = random_state
    self.image_extensions = image_extensions

    self._validate_inputs()
    self.class_to_idx, self.idx_to_class = self._build_class_mapping()
    self.data = self._prepare_split_data()

def _validate_inputs(self) -> None:
    if not self.dataset_path.exists():
        raise FileNotFoundError(f"Dataset path does not exist: {self.dat

    if not self.dataset_path.is_dir():
        raise ValueError(f"Dataset path is not a directory: {self.datase

    if self.split not in ['train', 'val', 'test']:
        raise ValueError(f"Split must be one of ['train', 'val', 'test']

    if not (0 < self.test_size < 1) or not (0 < self.val_size < 1):
        raise ValueError("test_size and val_size must be between 0 and 1

def _build_class_mapping(self) -> Tuple[Dict[str, int], Dict[int, str]]:
    class_dirs = [d for d in self.dataset_path.iterdir() if d.is_dir()]

    if not class_dirs:
        raise ValueError(f"No class directories found in {self.dataset_p

    classes = sorted([d.name for d in class_dirs])
    class_to_idx = {class_name: idx for idx, class_name in enumerate(class
    idx_to_class = {idx: class_name for class_name, idx in class_to_idx.

    return class_to_idx, idx_to_class

def _collect_image_paths(self) -> List[Dict[str, Union[str, int]]]:
    image_data = []

    for class_name, class_idx in self.class_to_idx.items():
        class_dir = self.dataset_path / class_name

        for image_path in class_dir.iterdir():
            if image_path.is_file() and image_path.suffix.lower() in self
                image_data.append({
                    'image_path': str(image_path),
                    'class_name': class_name,
                    'label': class_idx
                })

```



```

    if not image_data:
        raise ValueError(f"No images found in {self.dataset_path}")

    return image_data

def _prepare_split_data(self) -> pd.DataFrame:
    all_image_data = self._collect_image_paths()
    df = pd.DataFrame(all_image_data)

    if len(self.class_to_idx) == 1:
        stratify = None
    else:
        stratify = df['label']

    train_df, temp_df = train_test_split(
        df,
        test_size=(self.test_size + self.val_size),
        stratify=stratify,
        random_state=self.random_state
    )

    if self.val_size > 0 and len(temp_df) > 1:
        val_ratio = self.val_size / (self.test_size + self.val_size)

        if len(self.class_to_idx) == 1:
            temp_stratify = None
        else:
            temp_stratify = temp_df['label']

        val_df, test_df = train_test_split(
            temp_df,
            test_size=(1 - val_ratio),
            stratify=temp_stratify,
            random_state=self.random_state
        )
    else:
        val_df = temp_df.copy()
        test_df = temp_df.copy()

    split_mapping = {
        'train': train_df,
        'val': val_df,
        'test': test_df
    }

    selected_df = split_mapping[self.split].reset_index(drop=True)

    if selected_df.empty:
        raise ValueError(f"No data available for split '{self.split}'")

    return selected_df

def _load_image(self, image_path: str) -> Image.Image:
    try:
        return Image.open(image_path).convert("RGB")

```



```

        except Exception as e:
            raise IOError(f"Failed to load image {image_path}: {e}")

    def _create_dummy_tensor(self) -> torch.Tensor:
        dummy_size = (224, 224)

        if hasattr(self.image_processor, 'size'):
            size_info = self.image_processor.size
            if isinstance(size_info, dict):
                if 'shortest_edge' in size_info:
                    edge_size = size_info['shortest_edge']
                    dummy_size = (edge_size, edge_size)
                elif 'height' in size_info and 'width' in size_info:
                    dummy_size = (size_info['height'], size_info['width'])
            elif isinstance(size_info, (list, tuple)) and len(size_info) == 2:
                dummy_size = tuple(size_info)

        return torch.zeros((3, dummy_size[0], dummy_size[1]), dtype=torch.float32)

    def __len__(self) -> int:
        return len(self.data)

    def __getitem__(self, idx: int) -> Dict[str, torch.Tensor]:
        row = self.data.iloc[idx]
        image_path = row['image_path']
        label = row['label']

        try:
            image = self._load_image(image_path)
            processed_inputs = self.image_processor(images=image, return_tensors='pt')
            pixel_values = processed_inputs['pixel_values'].squeeze(0)
        except Exception as e:
            print(f"Warning: Failed to process image {image_path}: {e}. Using dummy tensor.")
            pixel_values = self._create_dummy_tensor()

        return {
            'pixel_values': pixel_values,
            'labels': torch.tensor(label, dtype=torch.long)
        }

    def get_class_distribution(self) -> Dict[str, int]:
        return self.data['class_name'].value_counts().to_dict()

    def get_num_classes(self) -> int:
        return len(self.class_to_idx)

    def get_class_names(self) -> List[str]:
        return list(self.class_to_idx.keys())

```

2.3 Модель

```

In [8]: class ImageClassifierMobileViT(nn.Module):
        def __init__(self, model_name_or_path: str, num_classes: int):
            super().__init__()

```

```

        self.model = MobileViTForImageClassification.from_pretrained(
            model_name_or_path,
            num_labels=num_classes,
            ignore_mismatched_sizes=True
        )

    def forward(self, pixel_values: torch.Tensor) -> Dict[str, torch.Tensor]:
        outputs = self.model(pixel_values=pixel_values)
        return {'logits': outputs.logits}

```

2.4 Трейнер

```

In [9]: def set_seed(seed: int = 42) -> None:
        random.seed(seed)
        np.random.seed(seed)
        torch.manual_seed(seed)
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
        torch.backends.cudnn.deterministic = True
        torch.backends.cudnn.benchmark = False
        os.environ['PYTHONHASHSEED'] = str(seed)

```

```

In [10]: def train_classifier(
        model: nn.Module, dataloader: DataLoader, optimizer: optim.Optimizer,
        scheduler: Optional[optim.lr_scheduler._LRScheduler], criterion: nn.Module,
        device: torch.device, epoch: int
    ) -> Tuple[float, float]:
    model.train()
    total_loss, correct_predictions, total_samples = 0.0, 0, 0
    total_tp, total_fp, total_fn, total_tn = 0, 0, 0, 0
    progress_bar = tqdm(dataloader, desc=f"Эпоха {epoch+1} Обучение", leave=

    for batch in progress_bar:
        pixel_values = batch['pixel_values'].to(device)
        labels = batch['labels'].to(device)

        optimizer.zero_grad()
        outputs = model(pixel_values=pixel_values)
        logits = outputs['logits']
        loss = criterion(logits, labels)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimizer.step()

        if scheduler is not None and not isinstance(scheduler, optim.lr_scheduler._LRScheduler):
            scheduler.step()

        total_loss += loss.item()
        _, predicted = torch.max(logits, 1)
        total_samples += labels.size(0)
        correct_predictions += (predicted == labels).sum().item()

        p_cpu, l_cpu = predicted.cpu(), labels.cpu()
        total_tp += ((p_cpu == 1) & (l_cpu == 1)).sum().item()

```

```

total_fp += ((p_cpu == 1) & (l_cpu == 0)).sum().item()
total_fn += ((p_cpu == 0) & (l_cpu == 1)).sum().item()
total_tn += ((p_cpu == 0) & (l_cpu == 0)).sum().item()

acc = correct_predictions / total_samples if total_samples > 0 else 0
rec = total_tp / (total_tp + total_fn) if (total_tp + total_fn) > 0 else 0
prec = total_tp / (total_tp + total_fp) if (total_tp + total_fp) > 0 else 0
f1 = 2 * (prec * rec) / (prec + rec) if (prec + rec) > 0 else 0.0
progress_bar.set_postfix(loss=loss.item(), acc=acc, rec=rec, pr=prec)

avg_loss = total_loss / len(dataloader) if len(dataloader) > 0 else 0.0
accuracy = correct_predictions / total_samples if total_samples > 0 else 0
recall_1 = total_tp / (total_tp + total_fn) if (total_tp + total_fn) > 0 else 0
precision_1 = total_tp / (total_tp + total_fp) if (total_tp + total_fp) > 0 else 0
f1_1 = 2 * (precision_1 * recall_1) / (precision_1 + recall_1) if (precision_1 + recall_1) > 0 else 0

if scheduler is not None and isinstance(scheduler, optim.lr_scheduler.ReduceLROnPlateau):
    scheduler.step(avg_loss)

print(f"Эпоха {epoch+1} Средняя потеря: {avg_loss:.4f}, Точность: {accuracy:.4f}")
return avg_loss, accuracy

```

```

In [11]: def evaluate_classifier(
    model: nn.Module, dataloader: DataLoader, criterion: nn.Module, device: torch.device) -> Tuple[float, float, float, float, float]:
    model.eval()
    total_loss, correct_predictions, total_samples = 0.0, 0, 0
    total_tp, total_fp, total_fn, total_tn = 0, 0, 0, 0
    progress_bar = tqdm(dataloader, desc="Валидация", leave=True)

    with torch.no_grad():
        for batch in progress_bar:
            pixel_values = batch['pixel_values'].to(device)
            labels = batch['labels'].to(device)
            outputs = model(pixel_values=pixel_values)
            logits = outputs['logits']
            loss = criterion(logits, labels)
            total_loss += loss.item()

            _, predicted = torch.max(logits, 1)
            total_samples += labels.size(0)
            correct_predictions += (predicted == labels).sum().item()

            p_cpu, l_cpu = predicted.cpu(), labels.cpu()
            total_tp += ((p_cpu == 1) & (l_cpu == 1)).sum().item()
            total_fp += ((p_cpu == 1) & (l_cpu == 0)).sum().item()
            total_fn += ((p_cpu == 0) & (l_cpu == 1)).sum().item()
            total_tn += ((p_cpu == 0) & (l_cpu == 0)).sum().item()

        acc = correct_predictions / total_samples if total_samples > 0 else 0
        rec = total_tp / (total_tp + total_fn) if (total_tp + total_fn) > 0 else 0
        prec = total_tp / (total_tp + total_fp) if (total_tp + total_fp) > 0 else 0
        f1 = 2 * (prec * rec) / (prec + rec) if (prec + rec) > 0 else 0.0
        progress_bar.set_postfix(loss=loss.item(), acc=acc, rec=rec, pr=prec)

    avg_loss = total_loss / len(dataloader) if len(dataloader) > 0 else 0.0

```

```

accuracy = correct_predictions / total_samples if total_samples > 0 else 0
recall_1 = total_tp / (total_tp + total_fn) if (total_tp + total_fn) > 0 else 0
precision_1 = total_tp / (total_tp + total_fp) if (total_tp + total_fp) > 0 else 0
f1_1 = 2 * (precision_1 * recall_1) / (precision_1 + recall_1) if (precision_1 + recall_1) > 0 else 0

print(f"Валидация: Потеря: {avg_loss:.4f}, Точность: {accuracy:.4f}, Полнота: {recall_1:.4f}, F1-мера: {f1_1:.4f}")
return avg_loss, accuracy, recall_1, precision_1, f1_1

```

2.5 Доп. функции

```

In [12]: def save_checkpoint(
    model: ImageClassifierMobileViT, optimizer: optim.Optimizer,
    scheduler: Optional[optim.lr_scheduler._LRScheduler], epoch: int,
    train_loss: float, eval_loss: float, eval_rec: float, output_dir: Path
) -> None:
    output_dir.mkdir(parents=True, exist_ok=True)
    model_save_path = output_dir / "hf_model"
    model.model.save_pretrained(model_save_path) # Save the underlying HuggingFace model

    state = {
        'epoch': epoch,
        'optimizer_state_dict': optimizer.state_dict(),
        'scheduler_state_dict': scheduler.state_dict() if scheduler else None,
        'train_loss': train_loss, 'eval_loss': eval_loss, 'eval_rec': eval_rec
    }
    torch.save(state, output_dir / "training_state.pt")
    print(f"Чекпоинт сохранён: {output_dir} (Эпоха {epoch}, Потеря обуч. {train_loss:.4f}, Точность валидации {eval_rec:.4f})")

def load_checkpoint_state(
    output_dir: Path, device: torch.device,
    optimizer: Optional[optim.Optimizer] = None,
    scheduler: Optional[optim.lr_scheduler._LRScheduler] = None
) -> Tuple[int, float, float, float]:
    path = output_dir / "training_state.pt"
    epoch, tr_loss, ev_loss, ev_rec = 0, float('inf'), float('inf'), float('inf')
    if path.exists():
        print(f"Загрузка состояния обучения из: {path}")
        state = torch.load(path, map_location=device)
        epoch = state.get('epoch', 0)
        tr_loss = state.get('train_loss', float('inf'))
        ev_loss = state.get('eval_loss', float('inf'))
        ev_rec = state.get('eval_rec', float('-inf'))
        if optimizer and 'optimizer_state_dict' in state:
            try: optimizer.load_state_dict(state['optimizer_state_dict']);
            except Exception as e: print(f"Ошибка загрузки состояния оптимизатора: {e}")
        if scheduler and 'scheduler_state_dict' in state and state['scheduler_state_dict']:
            try: scheduler.load_state_dict(state['scheduler_state_dict']);
            except Exception as e: print(f"Ошибка загрузки состояния планировщика: {e}")
        print(f"Состояние загружено: Эпоха {epoch}, Потеря обуч. {tr_loss:.4f}, Точность валидации {ev_rec:.4f}")
    else:
        print(f"Файл состояния обучения не найден ({path}). Начинаем с нуля.")
    return epoch, tr_loss, ev_loss, ev_rec

```

2.6 Основной execute

```
In [13]: def resume_or_start_training(
    train_dataloader: DataLoader, valid_dataloader: DataLoader, device: torch.device,
    epochs: int, output_dir: Path, model_name_or_path: str, num_classes: int,
    learning_rate: float = 5e-5, class_1_weight_factor: float = 1.0
) -> ImageClassifierMobileViT:
    output_dir.mkdir(exist_ok=True, parents=True)
    hf_model_path = output_dir / "hf_model"

    try:
        if hf_model_path.exists() and os.listdir(hf_model_path):
            print(f"Загрузка модели из чекпоинта: {hf_model_path}")
            model = ImageClassifierMobileViT(str(hf_model_path), num_classes)
        else:
            print(f"Инициализация новой модели из: {model_name_or_path}")
            model = ImageClassifierMobileViT(model_name_or_path, num_classes)
    except Exception as ex:
        print(ex)
        print(f"Инициализация предобученной модели")
        model = ImageClassifierMobileViT(HF_MODEL_NAME, num_classes).to(device)

    print(f"Модель: {sum(p.numel() for p in model.parameters() if p.requires_grad_)}")
    optimizer = optim.AdamW(model.parameters(), lr=learning_rate, weight_decay=weight_decay)

    total_steps = len(train_dataloader) * epochs
    scheduler = None
    if total_steps > 0:
        warmup_steps = int(total_steps * 0.1)
        if warmup_steps < total_steps:
            scheduler = optim.lr_scheduler.OneCycleLR(
                optimizer, max_lr=learning_rate, total_steps=total_steps,
                pct_start=warmup_steps / total_steps
            )
        else:
            print("Предупреждение: warmup_steps >= total_steps. OneCycleLR не используется")
    else:
        print("Предупреждение: total_steps <= 0. OneCycleLR не используется")

    start_epoch, _, _, best_eval_rec = load_checkpoint_state(output_dir, device)
    if best_eval_rec == float('-inf'): best_eval_rec = 0.0

    weights = torch.tensor([1.0, class_1_weight_factor], dtype=torch.float)
    criterion = nn.CrossEntropyLoss(weight=weights if num_classes == 2 else None)

    best_loss, best_acc, best_prec, best_f1 = float('inf'), 0.0, 0.0, 0.0
    print(f"Начало обучения с эпохи {start_epoch}. Лучшая полнота валидации: {best_f1}")

    for epoch in range(start_epoch, epochs):
        tr_loss, ev_loss, ev_rec = float('inf'), float('inf'), float('-inf')
        try:
            tr_loss, _ = train_classifier(model, train_dataloader, optimizer, criterion, device)
            ev_loss, ev_acc, ev_rec, ev_prec, ev_f1 = evaluate_classifier(model, valid_dataloader, device)

            if ev_rec > best_eval_rec:
                best_eval_rec = ev_rec
```

```

        best_loss, best_acc, best_prec, best_f1 = ev_loss, ev_acc, ev_prec, ev_f1
        print(f"□ Новая лучшая модель (Полнота {ev_rec:.4f}) - Потеря {ev_loss:.4f}")
        save_checkpoint(model, optimizer, scheduler, epoch + 1, tr_loss, tr_acc, tr_prec, tr_f1)
    except KeyboardInterrupt:
        print("\nОбучение прервано. Сохранение...");
        save_checkpoint(model, optimizer, scheduler, epoch + 1, tr_loss, tr_acc, tr_prec, tr_f1)
    except Exception as e:
        print(f"\nОшибка: {e}. Сохранение...");
        save_checkpoint(model, optimizer, scheduler, epoch + 1, tr_loss, tr_acc, tr_prec, tr_f1)

print("\nОбучение завершено.")
print(f"Лучшие метрики (по Полноте={best_eval_rec:.4f}): Потеря={best_loss:.4f}, Точность={best_acc:.4f}, Прецизия={best_prec:.4f}, F1={best_f1:.4f}")
return model

```

2.7 Параметры

```

In [17]: SEED = 42
BASE_OUTPUT_DIR = './vit_output'
PATH_TO_DATASET = os.path.join(path, 'afhq/train')
CACHE_NAME_DIR = 'cache'
TEST_SIZE = 0.2
TRAIN_BATCH_SIZE = 64
VALID_BATCH_SIZE = 64
VAL_SIZE=0.2
NUM_WORKERS = 0
EPOCHS = 100
NUM_CLASSES = 3
START_LEARNING_RATE = 5e-5
CLASS_1_WEIGHT_FACTOR = 1.0

```

2.9 ViT

```

In [18]: HF_MODEL_NAME = "apple/mobilevit-small"

```

2.9.1 Подготовка перед запуском

```

In [19]: set_seed(SEED)
base_output_dir = Path(BASE_OUTPUT_DIR)
base_output_dir.mkdir(parents=True, exist_ok=True)

image_processor = AutoImageProcessor.from_pretrained(HF_MODEL_NAME, use_fast_image=True)

train_dataset = ImageDataset(
    dataset_path=PATH_TO_DATASET,
    image_processor=image_processor,
    split='train',
    test_size=TEST_SIZE,
    val_size=VAL_SIZE,
    random_state=SEED
)

```

```

valid_dataset = ImageDataset(
    dataset_path=PATH_TO_DATASET,
    image_processor=image_processor,
    split='val',
    test_size=TEST_SIZE,
    val_size=VAL_SIZE,
    random_state=SEED
)

print("Number of classes in dataset:", train_dataset.get_num_classes())
print("Number of classes in model:", NUM_CLASSES)
assert train_dataset.get_num_classes() == NUM_CLASSES

print("Train labels:", train_dataset.data['label'].unique())
print("Valid labels:", valid_dataset.data['label'].unique())

assert train_dataset.data['label'].isin(range(NUM_CLASSES)).all(), "Invalid
assert valid_dataset.data['label'].isin(range(NUM_CLASSES)).all(), "Invalid

print(f"Train dataset size: {len(train_dataset)}")
print(f"Valid dataset size: {len(valid_dataset)}")
print(f"Class distribution train: {train_dataset.get_class_distribution()}")
print(f"Class distribution valid: {valid_dataset.get_class_distribution()}")

train_dataloader = DataLoader(
    train_dataset,
    batch_size=TRAIN_BATCH_SIZE,
    shuffle=True,
    num_workers=NUM_WORKERS
)
valid_dataloader = DataLoader(
    valid_dataset,
    batch_size=VALID_BATCH_SIZE,
    shuffle=False,
    num_workers=NUM_WORKERS
)

if len(train_dataloader) == 0 or len(valid_dataloader) == 0:
    print("Один из DataLoader'ов пуст. Проверьте размеры датасетов и batch_s
    exit(1)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Используется устройство: {device}")

```

```

Number of classes in dataset: 3
Number of classes in model: 3
Train labels: [2 0 1]
Valid labels: [2 1 0]
Train dataset size: 8778
Valid dataset size: 2926
Class distribution train: {'cat': 3092, 'wild': 2843, 'dog': 2843}
Class distribution valid: {'cat': 1030, 'wild': 948, 'dog': 948}
Используется устройство: cuda

```

2.9.2 Обучение

```
In [20]: trained_model_wrapper = resume_or_start_training(
    train_dataloader=train_dataloader,
    valid_dataloader=valid_dataloader,
    device=device,
    epochs=EPOCHS,
    output_dir=base_output_dir / "training_output",
    model_name_or_path=HF_MODEL_NAME,
    num_classes=NUM_CLASSES,
    learning_rate=START_LEARNING_RATE,
    class_1_weight_factor=CLASS_1_WEIGHT_FACTOR
)

print("\nTraining finished (from main).")
```

Загрузка модели из чекпоинта: vit_output/training_output/hf_model
 Error no file named pytorch_model.bin, model.safetensors, tf_model.h5, model.ckpt.index or flax_model.msgpack found in directory vit_output/training_output/hf_model.

Инициализация предобученной модели

Some weights of MobileViTForImageClassification were not initialized from the model checkpoint at apple/mobilevit-small and are newly initialized because the shapes did not match:

- classifier.weight: found shape torch.Size([1000, 640]) in the checkpoint and torch.Size([3, 640]) in the model instantiated
- classifier.bias: found shape torch.Size([1000]) in the checkpoint and torch.Size([3]) in the model instantiated

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

Модель: 4,939,555 обучаемых параметров.

Файл состояния обучения не найден (vit_output/training_output/training_state.pt). Начинаем с нуля.

Начало обучения с эпохи 0. Лучшая полнота валидации: 0.0000

Эпоха 1 Обучение: 100%|█| 138/138 [01:03<00:00, 2.16it/s, acc=0.334, f1=0.59, loss=1.09, pr=0.

Эпоха 1 Средняя потеря: 1.0997, Точность: 0.3340, Полнота(1): 0.6636, Точность(1): 0.5318, F1(1): 0.5904

Валидация: 100%|█| 46/46 [00:17<00:00, 2.66it/s, acc=0.526, f1=0.798, loss=1.05, pr=0.745, rec

Валидация: Потеря: 1.0603, Точность: 0.5263, Полнота(1): 0.8592, Точность(1): 0.7451, F1(1): 0.7981

□ Новая лучшая модель (Полнота 0.8592) - Потеря: 1.0603, Точность: 0.5263

Чекпоинт сохранён: vit_output/training_output (Эпоха 1, Потеря обуч. 1.0997, Потеря вал. 1.0603, Полнота вал. 0.8592)

Эпоха 2 Обучение: 100%|█| 138/138 [01:03<00:00, 2.18it/s, acc=0.687, f1=0.872, loss=0.912, pr=

Эпоха 2 Средняя потеря: 1.0141, Точность: 0.6867, Полнота(1): 0.9050, Точность(1): 0.8413, F1(1): 0.8720

Валидация: 100%|█| 46/46 [00:14<00:00, 3.07it/s, acc=0.867, f1=0.981, loss=0.899, pr=0.979, re

Валидация: Потеря: 0.9245, Точность: 0.8671, Полнота(1): 0.9831, Точность(1): 0.9790, F1(1): 0.9810

□ Новая лучшая модель (Полнота 0.9831) - Потеря: 0.9245, Точность: 0.8671

Чекпоинт сохранён: vit_output/training_output (Эпоха 2, Потеря обуч. 1.0141, Потеря вал. 0.9245, Полнота вал. 0.9831)

Эпоха 3 Обучение: 100%|█| 138/138 [01:02<00:00, 2.21it/s, acc=0.923, f1=0.986, loss=0.662, pr=
Эпоха 3 Средняя потеря: 0.8187, Точность: 0.9232, Полнота(1): 0.9878, Точность(1): 0.9842, F1(1): 0.9860

Валидация: 100%|█| 46/46 [00:15<00:00, 2.99it/s, acc=0.966, f1=0.998, loss=0.596, pr=0.999, re
Валидация: Потеря: 0.6356, Точность: 0.9658, Полнота(1): 0.9977, Точность(1): 0.9989, F1(1): 0.9983
□ Новая лучшая модель (Полнота 0.9977) - Потеря: 0.6356, Точность: 0.9658
Чекпоинт сохранён: vit_output/training_output (Эпоха 3, Потеря обуч. 0.8187, Потеря вал. 0.6356, Полнота вал. 0.9977)

Эпоха 4 Обучение: 100%|█| 138/138 [01:02<00:00, 2.20it/s, acc=0.981, f1=0.998, loss=0.397, pr=
Эпоха 4 Средняя потеря: 0.5004, Точность: 0.9813, Полнота(1): 0.9989, Точность(1): 0.9971, F1(1): 0.9980

Валидация: 100%|██████| 46/46 [00:15<00:00, 3.06it/s, acc=0.99, f1=1, loss=0.221, pr=1, rec=1]
Валидация: Потеря: 0.2689, Точность: 0.9901, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000
□ Новая лучшая модель (Полнота 1.0000) - Потеря: 0.2689, Точность: 0.9901
Чекпоинт сохранён: vit_output/training_output (Эпоха 4, Потеря обуч. 0.5004, Потеря вал. 0.2689, Полнота вал. 1.0000)

Эпоха 5 Обучение: 100%|█| 138/138 [01:03<00:00, 2.16it/s, acc=0.992, f1=0.999, loss=0.101, pr=
Эпоха 5 Средняя потеря: 0.2065, Точность: 0.9916, Полнота(1): 0.9996, Точность(1): 0.9989, F1(1): 0.9993

Валидация: 100%|█| 46/46 [00:15<00:00, 2.90it/s, acc=0.996, f1=0.999, loss=0.066, pr=0.999, re
Валидация: Потеря: 0.0933, Точность: 0.9959, Полнота(1): 1.0000, Точность(1): 0.9989, F1(1): 0.9995

Эпоха 6 Обучение: 100%|█| 138/138 [01:02<00:00, 2.21it/s, acc=0.995, f1=1, loss=0.0449, pr=1,
Эпоха 6 Средняя потеря: 0.0835, Точность: 0.9953, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:14<00:00, 3.07it/s, acc=0.997, f1=0.999, loss=0.0282, pr=0.999, r
Валидация: Потеря: 0.0420, Точность: 0.9969, Полнота(1): 1.0000, Точность(1): 0.9989, F1(1): 0.9995

Эпоха 7 Обучение: 100%|█| 138/138 [01:02<00:00, 2.21it/s, acc=0.997, f1=1, loss=0.206, pr=1, r
Эпоха 7 Средняя потеря: 0.0445, Точность: 0.9967, Полнота(1): 0.9996, Точность(1): 0.9996, F1(1): 0.9996

Валидация: 100%|█| 46/46 [00:15<00:00, 3.04it/s, acc=0.996, f1=0.999, loss=0.0129, pr=0.999, r
Валидация: Потеря: 0.0259, Точность: 0.9962, Полнота(1): 1.0000, Точность(1): 0.9989, F1(1): 0.9995

Эпоха 8 Обучение: 100%|█| 138/138 [01:02<00:00, 2.21it/s, acc=0.997, f1=1, loss=0.0265, pr=1,
Эпоха 8 Средняя потеря: 0.0279, Точность: 0.9972, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:15<00:00, 2.99it/s, acc=0.997, f1=0.999, loss=0.00819, pr=0.999,

Валидация: Потеря: 0.0191, Точность: 0.9969, Полнота(1): 1.0000, Точность (1): 0.9989, F1(1): 0.9995

Эпоха 9 Обучение: 100%|█| 138/138 [01:02<00:00, 2.20it/s, acc=0.998, f1=1, loss=0.0609, pr=1,

Эпоха 9 Средняя потеря: 0.0180, Точность: 0.9982, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:15<00:00, 2.92it/s, acc=0.997, f1=0.999, loss=0.00498, pr=0.999,

Валидация: Потеря: 0.0151, Точность: 0.9969, Полнота(1): 1.0000, Точность (1): 0.9989, F1(1): 0.9995

Эпоха 10 Обучение: 100%|█| 138/138 [01:02<00:00, 2.21it/s, acc=0.999, f1=1, loss=0.023, pr=1,

Эпоха 10 Средняя потеря: 0.0119, Точность: 0.9989, Полнота(1): 1.0000, Точность(1): 0.9996, F1(1): 0.9998

Валидация: 100%|█| 46/46 [00:15<00:00, 2.99it/s, acc=0.997, f1=0.999, loss=0.0034, pr=0.999, r

Валидация: Потеря: 0.0126, Точность: 0.9973, Полнота(1): 1.0000, Точность (1): 0.9989, F1(1): 0.9995

Эпоха 11 Обучение: 100%|█| 138/138 [01:02<00:00, 2.20it/s, acc=0.999, f1=1, loss=0.00609, pr=1

Эпоха 11 Средняя потеря: 0.0112, Точность: 0.9986, Полнота(1): 1.0000, Точность(1): 0.9996, F1(1): 0.9998

Валидация: 100%|█| 46/46 [00:15<00:00, 2.94it/s, acc=0.997, f1=0.999, loss=0.00248, pr=0.999,

Валидация: Потеря: 0.0130, Точность: 0.9973, Полнота(1): 1.0000, Точность (1): 0.9989, F1(1): 0.9995

Эпоха 12 Обучение: 100%|█| 138/138 [01:03<00:00, 2.18it/s, acc=0.999, f1=1, loss=0.00437, pr=1

Эпоха 12 Средняя потеря: 0.0088, Точность: 0.9989, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:16<00:00, 2.85it/s, acc=0.997, f1=1, loss=0.00189, pr=1, rec=1]

Валидация: Потеря: 0.0115, Точность: 0.9973, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 13 Обучение: 100%|█| 138/138 [01:02<00:00, 2.20it/s, acc=0.999, f1=1, loss=0.0154, pr=1,

Эпоха 13 Средняя потеря: 0.0064, Точность: 0.9991, Полнота(1): 0.9996, Точность(1): 1.0000, F1(1): 0.9998

Валидация: 100%|█| 46/46 [00:15<00:00, 2.93it/s, acc=0.997, f1=0.999, loss=0.00139, pr=0.999,

Валидация: Потеря: 0.0091, Точность: 0.9973, Полнота(1): 1.0000, Точность (1): 0.9989, F1(1): 0.9995

Эпоха 14 Обучение: 100%|█| 138/138 [01:02<00:00, 2.20it/s, acc=1, f1=1, loss=0.0441, pr=1, rec

Эпоха 14 Средняя потеря: 0.0045, Точность: 0.9998, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:15<00:00, 3.05it/s, acc=0.998, f1=1, loss=0.00123, pr=1, rec=1]

Валидация: Потеря: 0.0104, Точность: 0.9979, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 15 Обучение: 100%|█| 138/138 [01:02<00:00, 2.20it/s, acc=0.999, f1=1, loss=0.00794, pr=1

Эпоха 15 Средняя потеря: 0.0057, Точность: 0.9991, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:15<00:00, 2.92it/s, acc=0.998, f1=0.999, loss=0.000987, pr=0.999,

Валидация: Потеря: 0.0122, Точность: 0.9976, Полнота(1): 1.0000, Точность(1): 0.9989, F1(1): 0.9995

Эпоха 16 Обучение: 100%|█| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=0.35, pr=1, rec=1

Эпоха 16 Средняя потеря: 0.0058, Точность: 0.9998, Полнота(1): 0.9996, Точность(1): 1.0000, F1(1): 0.9998

Валидация: 100%|█| 46/46 [00:16<00:00, 2.87it/s, acc=0.997, f1=0.999, loss=0.000955, pr=0.999,

Валидация: Потеря: 0.0135, Точность: 0.9973, Полнота(1): 1.0000, Точность(1): 0.9989, F1(1): 0.9995

Эпоха 17 Обучение: 100%|█| 138/138 [01:02<00:00, 2.20it/s, acc=1, f1=1, loss=0.00823, pr=1, re

Эпоха 17 Средняя потеря: 0.0026, Точность: 0.9999, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:15<00:00, 2.88it/s, acc=0.997, f1=1, loss=0.000709, pr=1, rec=1]

Валидация: Потеря: 0.0146, Точность: 0.9973, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Эпоха 18 Обучение: 100%|█| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=0.0231, pr=1, rec

Эпоха 18 Средняя потеря: 0.0039, Точность: 0.9997, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:15<00:00, 3.04it/s, acc=0.997, f1=1, loss=0.000511, pr=1, rec=1]

Валидация: Потеря: 0.0155, Точность: 0.9973, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Эпоха 19 Обучение: 100%|█| 138/138 [01:05<00:00, 2.11it/s, acc=1, f1=1, loss=0.000796, pr=1, r

Эпоха 19 Средняя потеря: 0.0033, Точность: 0.9995, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:15<00:00, 3.06it/s, acc=0.998, f1=0.999, loss=0.000456, pr=0.999,

Валидация: Потеря: 0.0137, Точность: 0.9976, Полнота(1): 1.0000, Точность(1): 0.9989, F1(1): 0.9995

Эпоха 20 Обучение: 100%|█| 138/138 [01:02<00:00, 2.20it/s, acc=1, f1=1, loss=0.00102, pr=1, re

Эпоха 20 Средняя потеря: 0.0015, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:15<00:00, 2.92it/s, acc=0.998, f1=0.999, loss=0.000367, pr=0.999,

Валидация: Потеря: 0.0149, Точность: 0.9976, Полнота(1): 1.0000, Точность(1): 0.9989, F1(1): 0.9995

Эпоха 21 Обучение: 100%|█| 138/138 [01:02<00:00, 2.19it/s, acc=1, f1=1, loss=0.00223, pr=1, re

Эпоха 21 Средняя потеря: 0.0019, Точность: 0.9995, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:14<00:00, 3.08it/s, acc=0.997, f1=1, loss=0.000358, pr=1, rec=1]

Валидация: Потеря: 0.0147, Точность: 0.9973, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 22 Обучение: 100%|█| 138/138 [01:03<00:00, 2.17it/s, acc=1, f1=1, loss=0.00146, pr=1, re

Эпоха 22 Средняя потеря: 0.0019, Точность: 0.9997, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:16<00:00, 2.79it/s, acc=0.996, f1=0.999, loss=0.00031, pr=0.999,

Валидация: Потеря: 0.0191, Точность: 0.9962, Полнота(1): 1.0000, Точность (1): 0.9989, F1(1): 0.9995

Эпоха 23 Обучение: 100%|█| 138/138 [01:03<00:00, 2.19it/s, acc=1, f1=1, loss=0.00167, pr=1, re

Эпоха 23 Средняя потеря: 0.0024, Точность: 0.9995, Полнота(1): 1.0000, Точность(1): 0.9996, F1(1): 0.9998

Валидация: 100%|█| 46/46 [00:14<00:00, 3.09it/s, acc=0.998, f1=0.999, loss=0.000319, pr=0.999,

Валидация: Потеря: 0.0096, Точность: 0.9979, Полнота(1): 1.0000, Точность (1): 0.9989, F1(1): 0.9995

Эпоха 24 Обучение: 100%|█| 138/138 [01:02<00:00, 2.22it/s, acc=0.999, f1=1, loss=1.05, pr=1, r

Эпоха 24 Средняя потеря: 0.0096, Точность: 0.9994, Полнота(1): 1.0000, Точность(1): 0.9996, F1(1): 0.9998

Валидация: 100%|█| 46/46 [00:15<00:00, 3.06it/s, acc=0.997, f1=0.999, loss=0.00026, pr=0.999,

Валидация: Потеря: 0.0186, Точность: 0.9969, Полнота(1): 1.0000, Точность (1): 0.9989, F1(1): 0.9995

Эпоха 25 Обучение: 100%|█| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=0.0015, pr=1, res

Эпоха 25 Средняя потеря: 0.0008, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:14<00:00, 3.08it/s, acc=0.997, f1=0.999, loss=0.000247, pr=0.999,

Валидация: Потеря: 0.0171, Точность: 0.9966, Полнота(1): 1.0000, Точность (1): 0.9989, F1(1): 0.9995

Эпоха 26 Обучение: 100%|█| 138/138 [01:02<00:00, 2.20it/s, acc=1, f1=1, loss=0.656, pr=1, res=

Эпоха 26 Средняя потеря: 0.0073, Точность: 0.9995, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:15<00:00, 2.99it/s, acc=0.997, f1=0.999, loss=0.000234, pr=0.999,

Валидация: Потеря: 0.0144, Точность: 0.9969, Полнота(1): 1.0000, Точность (1): 0.9989, F1(1): 0.9995

Эпоха 27 Обучение: 100%|█| 138/138 [01:02<00:00, 2.22it/s, acc=1, f1=1, loss=0.00356, pr=1, re

Эпоха 27 Средняя потеря: 0.0014, Точность: 0.9998, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:14<00:00, 3.08it/s, acc=0.997, f1=0.999, loss=0.000191, pr=0.999,

Валидация: Потеря: 0.0174, Точность: 0.9973, Полнота(1): 1.0000, Точность (1): 0.9989, F1(1): 0.9995

Эпоха 28 Обучение: 100%|█| 138/138 [01:07<00:00, 2.05it/s, acc=1, f1=1, loss=0.000672, pr=1, r

Эпоха 28 Средняя потеря: 0.0007, Точность: 0.9999, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:15<00:00, 3.06it/s, acc=0.997, f1=0.999, loss=0.000135, pr=0.999,

Валидация: Потеря: 0.0185, Точность: 0.9973, Полнота(1): 1.0000, Точность(1): 0.9989, F1(1): 0.9995

Эпоха 29 Обучение: 100%|█| 138/138 [01:02<00:00, 2.20it/s, acc=1, f1=1, loss=0.00196, pr=1, re

Эпоха 29 Средняя потеря: 0.0014, Точность: 0.9998, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:14<00:00, 3.08it/s, acc=0.997, f1=0.999, loss=0.00013, pr=0.999,

Валидация: Потеря: 0.0245, Точность: 0.9969, Полнота(1): 1.0000, Точность(1): 0.9989, F1(1): 0.9995

Эпоха 30 Обучение: 100%|█| 138/138 [01:02<00:00, 2.20it/s, acc=1, f1=1, loss=0.607, pr=1, res=

Эпоха 30 Средняя потеря: 0.0050, Точность: 0.9999, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:15<00:00, 2.88it/s, acc=0.998, f1=0.999, loss=0.000137, pr=0.999,

Валидация: Потеря: 0.0138, Точность: 0.9979, Полнота(1): 1.0000, Точность(1): 0.9989, F1(1): 0.9995

Эпоха 31 Обучение: 100%|█| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=0.12, pr=1, res=1

Эпоха 31 Средняя потеря: 0.0022, Точность: 0.9997, Полнота(1): 1.0000, Точность(1): 0.9996, F1(1): 0.9998

Валидация: 100%|█| 46/46 [00:15<00:00, 3.06it/s, acc=0.997, f1=0.999, loss=0.000131, pr=0.999,

Валидация: Потеря: 0.0275, Точность: 0.9966, Полнота(1): 1.0000, Точность(1): 0.9989, F1(1): 0.9995

Эпоха 32 Обучение: 100%|█| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=1.56, pr=1, res=1

Эпоха 32 Средняя потеря: 0.0121, Точность: 0.9998, Полнота(1): 0.9996, Точность(1): 1.0000, F1(1): 0.9998

Валидация: 100%|█| 46/46 [00:15<00:00, 3.02it/s, acc=0.998, f1=0.999, loss=9.68e-5, pr=0.999,

Валидация: Потеря: 0.0167, Точность: 0.9979, Полнота(1): 1.0000, Точность(1): 0.9989, F1(1): 0.9995

Эпоха 33 Обучение: 100%|█| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=0.00224, pr=1, re

Эпоха 33 Средняя потеря: 0.0005, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:15<00:00, 2.92it/s, acc=0.998, f1=0.999, loss=0.000117, pr=0.999,

Валидация: Потеря: 0.0152, Точность: 0.9979, Полнота(1): 1.0000, Точность(1): 0.9989, F1(1): 0.9995

Эпоха 34 Обучение: 100%|█| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=0.000455, pr=1, r

Эпоха 34 Средняя потеря: 0.0007, Точность: 0.9999, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:17<00:00, 2.67it/s, acc=0.997, f1=0.999, loss=8.92e-5, pr=0.999,

Валидация: Потеря: 0.0221, Точность: 0.9973, Полнота(1): 1.0000, Точность (1): 0.9989, F1(1): 0.9995

Эпоха 35 Обучение: 100%|█| 138/138 [01:03<00:00, 2.17it/s, acc=1, f1=1, loss=0.00274, pr=1, re

Эпоха 35 Средняя потеря: 0.0009, Точность: 0.9999, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:15<00:00, 2.93it/s, acc=0.998, f1=0.999, loss=8.14e-5, pr=0.999,

Валидация: Потеря: 0.0141, Точность: 0.9979, Полнота(1): 1.0000, Точность (1): 0.9989, F1(1): 0.9995

Эпоха 36 Обучение: 100%|█| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=0.00254, pr=1, re

Эпоха 36 Средняя потеря: 0.0016, Точность: 0.9999, Полнота(1): 1.0000, Точность(1): 0.9996, F1(1): 0.9998

Валидация: 100%|█| 46/46 [00:15<00:00, 2.96it/s, acc=0.998, f1=0.999, loss=6.1e-5, pr=0.999, r

Валидация: Потеря: 0.0143, Точность: 0.9979, Полнота(1): 1.0000, Точность (1): 0.9989, F1(1): 0.9995

Эпоха 37 Обучение: 100%|█| 138/138 [01:04<00:00, 2.15it/s, acc=1, f1=1, loss=0.00113, pr=1, re

Эпоха 37 Средняя потеря: 0.0016, Точность: 0.9999, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:16<00:00, 2.87it/s, acc=0.997, f1=0.999, loss=5.47e-5, pr=0.999,

Валидация: Потеря: 0.0229, Точность: 0.9973, Полнота(1): 1.0000, Точность (1): 0.9989, F1(1): 0.9995

Эпоха 38 Обучение: 100%|█| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=0.00157, pr=1, re

Эпоха 38 Средняя потеря: 0.0005, Точность: 0.9999, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:16<00:00, 2.86it/s, acc=0.998, f1=0.999, loss=8.24e-5, pr=0.999,

Валидация: Потеря: 0.0137, Точность: 0.9979, Полнота(1): 1.0000, Точность (1): 0.9989, F1(1): 0.9995

Эпоха 39 Обучение: 100%|█| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=0.0021, pr=1, rec

Эпоха 39 Средняя потеря: 0.0003, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:15<00:00, 3.05it/s, acc=0.998, f1=0.999, loss=6.7e-5, pr=0.999, r

Валидация: Потеря: 0.0153, Точность: 0.9976, Полнота(1): 1.0000, Точность (1): 0.9989, F1(1): 0.9995

Эпоха 40 Обучение: 100%|█| 138/138 [01:03<00:00, 2.17it/s, acc=1, f1=1, loss=0.00222, pr=1, re

Эпоха 40 Средняя потеря: 0.0003, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:16<00:00, 2.84it/s, acc=0.997, f1=0.999, loss=4.71e-5, pr=0.999,

Валидация: Потеря: 0.0210, Точность: 0.9969, Полнота(1): 1.0000, Точность (1): 0.9989, F1(1): 0.9995

Эпоха 41 Обучение: 100%|█| 138/138 [01:02<00:00, 2.19it/s, acc=0.999, f1=1, loss=0.000177, pr=

Эпоха 41 Средняя потеря: 0.0025, Точность: 0.9993, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:16<00:00, 2.86it/s, acc=0.998, f1=0.999, loss=4.29e-5, pr=0.999,

Валидация: Потеря: 0.0174, Точность: 0.9976, Полнота(1): 1.0000, Точность(1): 0.9989, F1(1): 0.9995

Эпоха 42 Обучение: 100%|█| 138/138 [01:02<00:00, 2.20it/s, acc=1, f1=1, loss=0.00521, pr=1, re

Эпоха 42 Средняя потеря: 0.0006, Точность: 0.9998, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:16<00:00, 2.79it/s, acc=0.998, f1=0.999, loss=3.82e-5, pr=0.999,

Валидация: Потеря: 0.0172, Точность: 0.9976, Полнота(1): 1.0000, Точность(1): 0.9989, F1(1): 0.9995

Эпоха 43 Обучение: 100%|█| 138/138 [01:02<00:00, 2.20it/s, acc=1, f1=1, loss=0.00191, pr=1, re

Эпоха 43 Средняя потеря: 0.0003, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:15<00:00, 2.88it/s, acc=0.998, f1=0.999, loss=5.06e-5, pr=0.999,

Валидация: Потеря: 0.0149, Точность: 0.9979, Полнота(1): 1.0000, Точность(1): 0.9989, F1(1): 0.9995

Эпоха 44 Обучение: 100%|█| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=0.000158, pr=1, r

Эпоха 44 Средняя потеря: 0.0002, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:15<00:00, 3.04it/s, acc=0.998, f1=0.999, loss=3.85e-5, pr=0.999,

Валидация: Потеря: 0.0165, Точность: 0.9976, Полнота(1): 1.0000, Точность(1): 0.9989, F1(1): 0.9995

Эпоха 45 Обучение: 100%|█| 138/138 [01:02<00:00, 2.20it/s, acc=1, f1=1, loss=0.00441, pr=1, re

Эпоха 45 Средняя потеря: 0.0013, Точность: 0.9998, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:16<00:00, 2.79it/s, acc=0.997, f1=1, loss=2.71e-5, pr=1, rec=1]

Валидация: Потеря: 0.0190, Точность: 0.9969, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Эпоха 46 Обучение: 100%|█| 138/138 [01:05<00:00, 2.12it/s, acc=1, f1=1, loss=0.305, pr=1, rec=

Эпоха 46 Средняя потеря: 0.0050, Точность: 0.9995, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:15<00:00, 2.93it/s, acc=0.997, f1=0.999, loss=2.61e-5, pr=0.999,

Валидация: Потеря: 0.0220, Точность: 0.9973, Полнота(1): 1.0000, Точность(1): 0.9989, F1(1): 0.9995

Эпоха 47 Обучение: 100%|█| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=0.000936, pr=1, r

Эпоха 47 Средняя потеря: 0.0006, Точность: 0.9999, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:15<00:00, 2.89it/s, acc=0.997, f1=0.999, loss=2.99e-5, pr=0.999,

Валидация: Потеря: 0.0190, Точность: 0.9973, Полнота(1): 1.0000, Точность (1): 0.9989, F1(1): 0.9995

Эпоха 48 Обучение: 100%|█| 138/138 [01:02<00:00, 2.20it/s, acc=1, f1=1, loss=0.000107, pr=1, r

Эпоха 48 Средняя потеря: 0.0011, Точность: 0.9999, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:16<00:00, 2.86it/s, acc=0.998, f1=1, loss=2.39e-5, pr=1, rec=1]

Валидация: Потеря: 0.0140, Точность: 0.9983, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 49 Обучение: 100%|█| 138/138 [01:02<00:00, 2.20it/s, acc=1, f1=1, loss=0.00704, pr=1, re

Эпоха 49 Средняя потеря: 0.0002, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:16<00:00, 2.71it/s, acc=0.998, f1=1, loss=2.43e-5, pr=1, rec=1]

Валидация: Потеря: 0.0164, Точность: 0.9983, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 50 Обучение: 100%|█| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=0.00012, pr=1, re

Эпоха 50 Средняя потеря: 0.0005, Точность: 0.9998, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:15<00:00, 3.06it/s, acc=0.998, f1=1, loss=2.06e-5, pr=1, rec=1]

Валидация: Потеря: 0.0124, Точность: 0.9983, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 51 Обучение: 100%|█| 138/138 [01:02<00:00, 2.20it/s, acc=1, f1=1, loss=0.0247, pr=1, rec

Эпоха 51 Средняя потеря: 0.0004, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:16<00:00, 2.80it/s, acc=0.998, f1=0.999, loss=2.23e-5, pr=0.999,

Валидация: Потеря: 0.0160, Точность: 0.9976, Полнота(1): 1.0000, Точность (1): 0.9989, F1(1): 0.9995

Эпоха 52 Обучение: 100%|█| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=0.000164, pr=1, r

Эпоха 52 Средняя потеря: 0.0002, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:17<00:00, 2.70it/s, acc=0.998, f1=1, loss=1.99e-5, pr=1, rec=1]

Валидация: Потеря: 0.0122, Точность: 0.9979, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 53 Обучение: 100%|█| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=0.000463, pr=1, r

Эпоха 53 Средняя потеря: 0.0005, Точность: 0.9999, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:15<00:00, 3.01it/s, acc=0.998, f1=1, loss=1.49e-5, pr=1, rec=1]

Валидация: Потеря: 0.0183, Точность: 0.9976, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 54 Обучение: 100%|█| 138/138 [01:02<00:00, 2.19it/s, acc=1, f1=1, loss=0.00263, pr=1, re

Эпоха 54 Средняя потеря: 0.0001, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:16<00:00, 2.83it/s, acc=0.998, f1=1, loss=1.4e-5, pr=1, rec=1]

Валидация: Потеря: 0.0182, Точность: 0.9976, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Эпоха 55 Обучение: 100%|██████| 138/138 [01:02<00:00, 2.19it/s, acc=1, f1=1, loss=0.000941, pr=1, r

Эпоха 55 Средняя потеря: 0.0003, Точность: 0.9999, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:15<00:00, 2.88it/s, acc=0.998, f1=1, loss=1.62e-5, pr=1, rec=1]

Валидация: Потеря: 0.0138, Точность: 0.9983, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Эпоха 56 Обучение: 100%|██████| 138/138 [01:02<00:00, 2.19it/s, acc=1, f1=1, loss=2.78e-5, pr=1, re

Эпоха 56 Средняя потеря: 0.0001, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:15<00:00, 2.88it/s, acc=0.998, f1=1, loss=1.37e-5, pr=1, rec=1]

Валидация: Потеря: 0.0150, Точность: 0.9983, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Эпоха 57 Обучение: 100%|██████| 138/138 [01:02<00:00, 2.20it/s, acc=1, f1=1, loss=0.0256, pr=1, rec

Эпоха 57 Средняя потеря: 0.0009, Точность: 0.9999, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:16<00:00, 2.83it/s, acc=0.998, f1=1, loss=1.69e-5, pr=1, rec=1]

Валидация: Потеря: 0.0146, Точность: 0.9983, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Эпоха 58 Обучение: 100%|██████| 138/138 [01:02<00:00, 2.20it/s, acc=1, f1=1, loss=0.000298, pr=1, r

Эпоха 58 Средняя потеря: 0.0001, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:15<00:00, 2.99it/s, acc=0.998, f1=1, loss=1.24e-5, pr=1, rec=1]

Валидация: Потеря: 0.0161, Точность: 0.9983, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Эпоха 59 Обучение: 100%|██████| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=0.000428, pr=1, r

Эпоха 59 Средняя потеря: 0.0001, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:14<00:00, 3.07it/s, acc=0.998, f1=1, loss=9.63e-6, pr=1, rec=1]

Валидация: Потеря: 0.0165, Точность: 0.9983, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Эпоха 60 Обучение: 100%|██████| 138/138 [01:02<00:00, 2.20it/s, acc=1, f1=1, loss=0.00405, pr=1, re

Эпоха 60 Средняя потеря: 0.0001, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:17<00:00, 2.66it/s, acc=0.998, f1=1, loss=1.05e-5, pr=1, rec=1]

Валидация: Потеря: 0.0161, Точность: 0.9983, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 61 Обучение: 100%|█| 138/138 [01:02<00:00, 2.20it/s, acc=1, f1=1, loss=0.000961, pr=1, r

Эпоха 61 Средняя потеря: 0.0001, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|███| 46/46 [00:16<00:00, 2.76it/s, acc=0.998, f1=1, loss=9.78e-6, pr=1, rec=1]

Валидация: Потеря: 0.0168, Точность: 0.9983, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 62 Обучение: 100%|█| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=0.000685, pr=1, r

Эпоха 62 Средняя потеря: 0.0001, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|███| 46/46 [00:16<00:00, 2.77it/s, acc=0.998, f1=1, loss=8.29e-6, pr=1, rec=1]

Валидация: Потеря: 0.0170, Точность: 0.9983, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 63 Обучение: 100%|█| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=0.000102, pr=1, r

Эпоха 63 Средняя потеря: 0.0001, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|███| 46/46 [00:15<00:00, 2.91it/s, acc=0.998, f1=1, loss=7.7e-6, pr=1, rec=1]

Валидация: Потеря: 0.0179, Точность: 0.9976, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 64 Обучение: 100%|█| 138/138 [01:02<00:00, 2.20it/s, acc=1, f1=1, loss=9.43e-5, pr=1, re

Эпоха 64 Средняя потеря: 0.0001, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:17<00:00, 2.65it/s, acc=0.997, f1=0.999, loss=1.26e-5, pr=0.999,

Валидация: Потеря: 0.0215, Точность: 0.9973, Полнота(1): 1.0000, Точность (1): 0.9989, F1(1): 0.9995

Эпоха 65 Обучение: 100%|█| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=0.000244, pr=1, r

Эпоха 65 Средняя потеря: 0.0001, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|███| 46/46 [00:15<00:00, 2.89it/s, acc=0.998, f1=1, loss=7.3e-6, pr=1, rec=1]

Валидация: Потеря: 0.0173, Точность: 0.9979, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 66 Обучение: 100%|█| 138/138 [01:02<00:00, 2.20it/s, acc=1, f1=1, loss=0.000514, pr=1, r

Эпоха 66 Средняя потеря: 0.0001, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|███| 46/46 [00:15<00:00, 2.93it/s, acc=0.998, f1=1, loss=7.43e-6, pr=1, rec=1]

Валидация: Потеря: 0.0162, Точность: 0.9983, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 67 Обучение: 100%|█| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=0.00039, pr=1, re

Эпоха 67 Средняя потеря: 0.0001, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:16<00:00, 2.86it/s, acc=0.998, f1=1, loss=7.97e-6, pr=1, rec=1]

Валидация: Потеря: 0.0171, Точность: 0.9979, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Эпоха 68 Обучение: 100%|██████| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=0.0342, pr=1, rec=1]

Эпоха 68 Средняя потеря: 0.0003, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:14<00:00, 3.07it/s, acc=0.998, f1=0.999, loss=5.83e-6, pr=0.999,

Валидация: Потеря: 0.0180, Точность: 0.9979, Полнота(1): 1.0000, Точность(1): 0.9989, F1(1): 0.9995

Эпоха 69 Обучение: 100%|██████| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=0.000414, pr=1, r=1]

Эпоха 69 Средняя потеря: 0.0001, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:15<00:00, 2.94it/s, acc=0.998, f1=1, loss=5.92e-6, pr=1, rec=1]

Валидация: Потеря: 0.0192, Точность: 0.9979, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Эпоха 70 Обучение: 100%|██████| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=0.000217, pr=1, r=1]

Эпоха 70 Средняя потеря: 0.0001, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:15<00:00, 3.01it/s, acc=0.998, f1=1, loss=6.22e-6, pr=1, rec=1]

Валидация: Потеря: 0.0207, Точность: 0.9979, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Эпоха 71 Обучение: 100%|██████| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=0.000223, pr=1, r=1]

Эпоха 71 Средняя потеря: 0.0005, Точность: 0.9999, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:17<00:00, 2.63it/s, acc=0.997, f1=0.999, loss=4.98e-6, pr=0.999,

Валидация: Потеря: 0.0267, Точность: 0.9973, Полнота(1): 1.0000, Точность(1): 0.9989, F1(1): 0.9995

Эпоха 72 Обучение: 100%|██████| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=7.88e-5, pr=1, rec=1]

Эпоха 72 Средняя потеря: 0.0003, Точность: 0.9999, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:16<00:00, 2.84it/s, acc=0.998, f1=1, loss=5.65e-6, pr=1, rec=1]

Валидация: Потеря: 0.0183, Точность: 0.9979, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Эпоха 73 Обучение: 100%|██████| 138/138 [01:03<00:00, 2.19it/s, acc=1, f1=1, loss=3.32e-5, pr=1, rec=1]

Эпоха 73 Средняя потеря: 0.0000, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:15<00:00, 3.03it/s, acc=0.998, f1=1, loss=4.66e-6, pr=1, rec=1]

Валидация: Потеря: 0.0179, Точность: 0.9983, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 74 Обучение: 100%|█| 138/138 [01:03<00:00, 2.18it/s, acc=1, f1=1, loss=0.000762, pr=1, r

Эпоха 74 Средняя потеря: 0.0003, Точность: 0.9999, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:15<00:00, 2.96it/s, acc=0.998, f1=1, loss=4.81e-6, pr=1, rec=1]

Валидация: Потеря: 0.0173, Точность: 0.9979, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 75 Обучение: 100%|█| 138/138 [01:02<00:00, 2.20it/s, acc=1, f1=1, loss=0.000633, pr=1, r

Эпоха 75 Средняя потеря: 0.0011, Точность: 0.9999, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:15<00:00, 2.91it/s, acc=0.998, f1=1, loss=5.32e-6, pr=1, rec=1]

Валидация: Потеря: 0.0182, Точность: 0.9979, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 76 Обучение: 100%|█| 138/138 [01:04<00:00, 2.13it/s, acc=1, f1=1, loss=1.33e-5, pr=1, re

Эпоха 76 Средняя потеря: 0.0000, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:15<00:00, 2.93it/s, acc=0.998, f1=1, loss=4.21e-6, pr=1, rec=1]

Валидация: Потеря: 0.0196, Точность: 0.9979, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 77 Обучение: 100%|█| 138/138 [01:03<00:00, 2.19it/s, acc=1, f1=1, loss=0.0107, pr=1, rec

Эпоха 77 Средняя потеря: 0.0001, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:17<00:00, 2.69it/s, acc=0.998, f1=1, loss=7.55e-6, pr=1, rec=1]

Валидация: Потеря: 0.0202, Точность: 0.9976, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 78 Обучение: 100%|█| 138/138 [01:03<00:00, 2.17it/s, acc=1, f1=1, loss=2.95e-5, pr=1, re

Эпоха 78 Средняя потеря: 0.0001, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:15<00:00, 2.90it/s, acc=0.998, f1=1, loss=3.82e-6, pr=1, rec=1]

Валидация: Потеря: 0.0179, Точность: 0.9979, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 79 Обучение: 100%|█| 138/138 [01:02<00:00, 2.19it/s, acc=1, f1=1, loss=0.000267, pr=1, r

Эпоха 79 Средняя потеря: 0.0000, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|█| 46/46 [00:15<00:00, 3.05it/s, acc=0.998, f1=1, loss=4.48e-6, pr=1, rec=1]

Валидация: Потеря: 0.0177, Точность: 0.9979, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 80 Обучение: 100%|█| 138/138 [01:02<00:00, 2.20it/s, acc=1, f1=1, loss=0.000422, pr=1, r

Эпоха 80 Средняя потеря: 0.0013, Точность: 0.9999, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:15<00:00, 3.03it/s, acc=0.998, f1=1, loss=3.86e-6, pr=1, rec=1]

Валидация: Потеря: 0.0176, Точность: 0.9983, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Эпоха 81 Обучение: 100%|██████| 138/138 [01:03<00:00, 2.18it/s, acc=1, f1=1, loss=0.000278, pr=1, re

Эпоха 81 Средняя потеря: 0.0000, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:16<00:00, 2.83it/s, acc=0.998, f1=1, loss=5.99e-6, pr=1, rec=1]

Валидация: Потеря: 0.0165, Точность: 0.9979, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Эпоха 82 Обучение: 100%|██████| 138/138 [01:03<00:00, 2.18it/s, acc=1, f1=1, loss=0.000721, pr=1, re

Эпоха 82 Средняя потеря: 0.0000, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:14<00:00, 3.07it/s, acc=0.998, f1=1, loss=3.73e-6, pr=1, rec=1]

Валидация: Потеря: 0.0177, Точность: 0.9983, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Эпоха 83 Обучение: 100%|██████| 138/138 [01:02<00:00, 2.20it/s, acc=1, f1=1, loss=0.00388, pr=1, re

Эпоха 83 Средняя потеря: 0.0001, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:16<00:00, 2.78it/s, acc=0.998, f1=1, loss=6.2e-6, pr=1, rec=1]

Валидация: Потеря: 0.0159, Точность: 0.9983, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Эпоха 84 Обучение: 100%|██████| 138/138 [01:03<00:00, 2.16it/s, acc=1, f1=1, loss=0.00125, pr=1, re

Эпоха 84 Средняя потеря: 0.0000, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:15<00:00, 2.94it/s, acc=0.998, f1=1, loss=3.22e-6, pr=1, rec=1]

Валидация: Потеря: 0.0188, Точность: 0.9979, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Эпоха 85 Обучение: 100%|██████| 138/138 [01:03<00:00, 2.17it/s, acc=1, f1=1, loss=3.47e-5, pr=1, re

Эпоха 85 Средняя потеря: 0.0000, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:16<00:00, 2.75it/s, acc=0.998, f1=1, loss=3.13e-6, pr=1, rec=1]

Валидация: Потеря: 0.0201, Точность: 0.9979, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Эпоха 86 Обучение: 100%|██████| 138/138 [01:05<00:00, 2.12it/s, acc=1, f1=1, loss=4.62e-5, pr=1, re

Эпоха 86 Средняя потеря: 0.0000, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:16<00:00, 2.86it/s, acc=0.998, f1=1, loss=3.21e-6, pr=1, rec=1]

Валидация: Потеря: 0.0195, Точность: 0.9979, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 87 Обучение: 100%|█| 138/138 [01:02<00:00, 2.20it/s, acc=1, f1=1, loss=0.000148, pr=1, r

Эпоха 87 Средняя потеря: 0.0000, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|███| 46/46 [00:14<00:00, 3.07it/s, acc=0.998, f1=1, loss=3.37e-6, pr=1, rec=1]

Валидация: Потеря: 0.0192, Точность: 0.9976, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 88 Обучение: 100%|█| 138/138 [01:03<00:00, 2.17it/s, acc=1, f1=1, loss=0.000228, pr=1, r

Эпоха 88 Средняя потеря: 0.0000, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|███| 46/46 [00:15<00:00, 3.05it/s, acc=0.998, f1=1, loss=3.9e-6, pr=1, rec=1]

Валидация: Потеря: 0.0206, Точность: 0.9976, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 89 Обучение: 100%|█| 138/138 [01:04<00:00, 2.16it/s, acc=1, f1=1, loss=0.00127, pr=1, re

Эпоха 89 Средняя потеря: 0.0000, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|███| 46/46 [00:17<00:00, 2.70it/s, acc=0.998, f1=1, loss=4.62e-6, pr=1, rec=1]

Валидация: Потеря: 0.0198, Точность: 0.9976, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 90 Обучение: 100%|█| 138/138 [01:04<00:00, 2.15it/s, acc=1, f1=1, loss=0.000582, pr=1, r

Эпоха 90 Средняя потеря: 0.0000, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|███| 46/46 [00:17<00:00, 2.67it/s, acc=0.998, f1=1, loss=4.77e-6, pr=1, rec=1]

Валидация: Потеря: 0.0181, Точность: 0.9976, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 91 Обучение: 100%|█| 138/138 [01:02<00:00, 2.20it/s, acc=1, f1=1, loss=0.000265, pr=1, r

Эпоха 91 Средняя потеря: 0.0000, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|███| 46/46 [00:15<00:00, 3.00it/s, acc=0.998, f1=1, loss=3.01e-6, pr=1, rec=1]

Валидация: Потеря: 0.0197, Точность: 0.9979, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 92 Обучение: 100%|█| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=3.25e-5, pr=1, re

Эпоха 92 Средняя потеря: 0.0000, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|███| 46/46 [00:15<00:00, 3.01it/s, acc=0.998, f1=1, loss=3.55e-6, pr=1, rec=1]

Валидация: Потеря: 0.0205, Точность: 0.9976, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 93 Обучение: 100%|█| 138/138 [01:03<00:00, 2.18it/s, acc=1, f1=1, loss=0.000144, pr=1, r

Эпоха 93 Средняя потеря: 0.0001, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:14<00:00, 3.07it/s, acc=0.998, f1=1, loss=2.99e-6, pr=1, rec=1]

Валидация: Потеря: 0.0189, Точность: 0.9979, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Эпоха 94 Обучение: 100%|██████| 138/138 [01:03<00:00, 2.19it/s, acc=1, f1=1, loss=0.00291, pr=1, re

Эпоха 94 Средняя потеря: 0.0001, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:16<00:00, 2.84it/s, acc=0.998, f1=1, loss=3.63e-6, pr=1, rec=1]

Валидация: Потеря: 0.0172, Точность: 0.9979, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Эпоха 95 Обучение: 100%|██████| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=3.29e-5, pr=1, re

Эпоха 95 Средняя потеря: 0.0000, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:15<00:00, 2.99it/s, acc=0.998, f1=1, loss=3.03e-6, pr=1, rec=1]

Валидация: Потеря: 0.0195, Точность: 0.9979, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Эпоха 96 Обучение: 100%|██████| 138/138 [01:03<00:00, 2.18it/s, acc=1, f1=1, loss=0.000302, pr=1, r

Эпоха 96 Средняя потеря: 0.0000, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:14<00:00, 3.08it/s, acc=0.998, f1=1, loss=6.1e-6, pr=1, rec=1]

Валидация: Потеря: 0.0215, Точность: 0.9976, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Эпоха 97 Обучение: 100%|██████| 138/138 [01:02<00:00, 2.20it/s, acc=1, f1=1, loss=0.000242, pr=1, r

Эпоха 97 Средняя потеря: 0.0000, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:15<00:00, 2.94it/s, acc=0.998, f1=1, loss=3.67e-6, pr=1, rec=1]

Валидация: Потеря: 0.0190, Точность: 0.9979, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Эпоха 98 Обучение: 100%|██████| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=3e-5, pr=1, rec=1]

Эпоха 98 Средняя потеря: 0.0000, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:16<00:00, 2.84it/s, acc=0.998, f1=1, loss=4e-6, pr=1, rec=1]

Валидация: Потеря: 0.0180, Точность: 0.9979, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Эпоха 99 Обучение: 100%|██████| 138/138 [01:02<00:00, 2.21it/s, acc=1, f1=1, loss=0.00144, pr=1, re

Эпоха 99 Средняя потеря: 0.0000, Точность: 1.0000, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

Валидация: 100%|██████| 46/46 [00:15<00:00, 3.00it/s, acc=0.998, f1=1, loss=5.5e-6, pr=1, rec=1]

Валидация: Потеря: 0.0173, Точность: 0.9979, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Эпоха 100 Обучение: 100%|█| 138/138 [01:02<00:00, 2.19it/s, acc=1, f1=1, loss=1.27, pr=1, rec=

Эпоха 100 Средняя потеря: 0.0092, Точность: 0.9999, Полнота(1): 0.9996, Точность(1): 1.0000, F1(1): 0.9998

Валидация: 100%|█| 46/46 [00:15<00:00, 3.04it/s, acc=0.998, f1=1, loss=5.36e-6, pr=1, rec=1]

Валидация: Потеря: 0.0175, Точность: 0.9983, Полнота(1): 1.0000, Точность (1): 1.0000, F1(1): 1.0000

Обучение завершено.

Лучшие метрики (по Полноте=1.0000): Потеря=0.2689, Точность=0.9901, Точность (1)=1.0000, F1(1)=1.0000

Training finished (from main).

2.9.3 Валидация: Потеря: 0.0175, Точность: 0.9983, Полнота(1): 1.0000, Точность(1): 1.0000, F1(1): 1.0000

In []: